

Basic Requirements Explained

08/07/2026 14:39:20

can u explain what the requirements for basics all mean

What “Basic” means overall To reach the Basic tier, your group must:...

give me that message as a pdf

Thinking 🌐 Reading PDF creation guidelines ● Created and verified PDF...

Sport-Analytics-Tool (3).zip now, make for me, using all git and project...

Thinking 🌐 Inspecting repository and methodology documents ●...

We want to do as much of this as possible in sprint 1, so, can u give me...

Thinking 🌐 Reading requirements issue backlog ● Organizing issue set...

restructure, give me all, we want to hit all the reqs for intermediate and...

Thinking 🌐 Inspecting repository structure and methodology docs ●...

so does mapping mean to make them in issues or what? also, if i have...

Correct — do not recreate existing issues. “Mapping” means matching a...

so what am i creating then from the catalogue? issues?

Yes. The catalogue is a list of Gitea issues for the entire project. For ever...

ok, for sake of time, give me the 'most urgent', first issues for now, and ...

Thinking ● Planning urgent issues I’m prioritizing foundational work first...

give me full Create and prioritise the complete project backlog issue...

Thinking ● Structuring issue details I’m organizing this as a...

give me markdown description again.. your markdown got cut off halfway

Copy everything inside this box into the Gitea issue description: ##...

ok, i've made that issue, what's the next step??

Thinking ● Planning the next issue I’m keeping the backlog issue open ...

332cb8d8-631f-4494-9da6-f897da591b97.png

Perfect — issue #36 is set up correctly. Just add the two missing labels:...

ok, let's do that one... idk what mapping you want me to do?? we will ha...

Thinking ● Mapping the backlog issue I’m treating issue #36 as a...

ok, next

Thinking ● Outlining decision records I’m preparing the next issue after...

summerize for me b4 i create this what it means? what does it require...

This issue is basically about keeping a formal record of what your tutor ...

is that not anywhere in our repo yet?

Thinking 🌐 Inspecting repository contents for decision logs ● Checking...

lets skipt this one for now! whats next?

Thinking ● Planning architecture next The next issue is the system...

give me the desc markdown in 1 thing to copy paste

Description Design and document the complete high-level...

wait, surely the issue description isnt that long? just make description a...

You're right. For now, the issue only needs enough detail for someone to...

ok, whats next? surely we start the project at some point?

Thinking ● Used personal_context tool ✓ Worked for 40s Yes. We have...

ok, now give me the next few titles and lemme see i agree with you, the...

Thinking 🌐 Inspecting Project Repository Structure ● Planning...

OK, give me these 12, with title LABELSD ETC AND THEN A Markdown to...

Thinking ● Writing the issue templates I'm structuring all 12 GitHub...

does gittea have a way where i can import a doc and they make the...

Thinking 🌐 Searching the web docs.gitea.com ● Checking import optio...

ok, so ive doen the first issue, give me process to make the next 11 usin...

Thinking 🌐 Searching the web...

64ea10f8-b9b5-47b9-846c-20a64cb46137.png no way!!! it worked can ...

Thinking ● Planning the importer I'm identifying which Sprint 1 issues...

F12 wanst me to log into my asus acount or something?

That means your ASUS laptop has F12 assigned to a laptop shortcut, no...

3e7423d3-1365-4fb9-bd0b-4aa6968638d2.png

Thinking 🌐 Searching the web github.com ● Explaining the failure I'm...

ok done, now thats all sprint 1, right?? what will we have at the end of...

Thinking ● Confirming sprint coverage Yes—Sprint 1 is covered, assumi...

is testing there as part of issues? shouldnt there be one to setup testing...

Thinking 🌐 Inspecting Repository Testing Setup ● Inspected repository...

so give me the issue to copy and paste

Issue details Title Complete the automated testing foundation and test...

give me the whole markdown as one thing, u got cut off midway

Copy everything inside this single box into the Gitea issue description: ...

e90173a7-f0ba-4627-9ab3-d3df9391028a.png bcead505-231c-40f9-9fb...

Thinking 🌐 Inspecting repository structure and documentation needs ●...

Pasted text.txt 49: Description Define an executable, version-controlled...

Thinking ● Narrowed Issue 49 and retained it alongside Issue 27 ✓...

Pasted markdown(2).md ok, now dodoc with me slowly 1 step at a time....

Thinking ● Reviewing document status I'm assessing #38 as an...

its already been closed, must i ope it with a comment, assign myself to i...

Thinking ● Choosing next steps I'd leave #38 closed and avoid assignin...

#73

Great - record: Architecture follow-up = #73 Paste this into issue #73: #...

ok, done, now, should we do doc on 73 or 36 first?

Do #73 first, then #36. The reason is that #36's backlog document...

ok, apparently we using supabase oauth now btw, whats next

Since the team has decided on Supabase Auth with Google OAuth, the...

ok, created branch, next

Next, update the authentication section first. Open:...

Sport-Analytics-Tool (4).zip should i maybe give u the updated repo b4 w...

Thinking 🌐 Inspecting repository contents from ZIP ● Reviewing the...

your markdown again cut off!!

Copy this entire block into docs/architecture/system-architecture.md....

ok, next

Thinking ● Planning the roadmap edit I'm replacing only the roadmap...

next

Next, update the authentication risk in the table. In...

next

Thinking ● Verifying terminology consistency Next, I'm checking the...

15013dbc-35f5-46a7-b528-f60047121bed.png

Those three matches are correct. They only state that Firebase was...

next

Thinking 🌐 Inspecting repository build and documentation scripts ●...

7a0ba8f0-8d9a-4188-9ff9-7a2b92342713.png

Thinking 🌐 Checking Repository Documentation Configuration ● Check...

+ mkdocs.cmd build --strict + ~~~~~ + CategoryInfo :...

That worked. The documentation build passed successfully. The MkDocs...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd ru...

Thinking ● Checking repository state I'm starting by confirming which...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --...

Thinking ● Checking repository state I'm identifying the relevant...

yes, next? can we commit, i have to leave in 1 minute and want to finish...

Thinking ● Finalizing documentation changes I'm checking the edited fil...

write proper, full pull request acc to methodology pls

Pull Request title docs(architecture): align roadmap and authentication...

k, now do for #36 docs etc, and woill do later

Thinking 🌐 Inspecting repository docs and planning files ● Inspected...

ok, go for it

Thinking ● Created project backlog and searched catalogue for issue #...

6eb9215e-6a2c-455a-8266-b29d226cdbb1.png da2a598c-7d10-4162-...

Thinking ● Mapping implementation work I'm mapping numbered work...

ok lets do it

Thinking ● Drafting sprint evidence I'm preparing a grounded sprint-...

next

Thinking ● Updated MkDocs navigation and finalized PR #36 ✓ Worked...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content...

Add the backlog page under Methodology. In mkdocs.yml, replace this: -...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> mkdocs buil...

Great — the MkDocs build passed. Now clean the generated site/ output...

yuo (note its 2026-08-06 not 07 so i changed it

Perfect — use the corrected evidence filename: evidence/sprints/sprint-...

what was issue 36? does it close or ref? i dont think we done, doesnt 36...

Thinking ● Clarifying backlog closure I'm distinguishing backlog setup... should we update the docs with sprint 2 and 3, but not the gittea bc we...

Thinking ● Reconciling backlog planning Yes, keeping the full roadmap i... ok, so give me steep by step on what to do so we can close 36 now,...

Thinking ● Clarifying issue scope I'm treating this as a planning change,... noo, but look at the rubric? dont we need a complete backlog...

Thinking ● Planning backlog documentation I'm confirming the... Pasted markdown(3).md seems wrong? give me the whole doc with the...

Thinking ● Planning document I'm preparing one clean document that... done

Great. Next, update issue #36 itself so the Gitea record matches the... my previous pr isnt merged yet, will it cause an issue if i pr again? shoul...

Thinking ● Checking branch ancestry I'm separating the pull requests... noo, i mean my #36 pr?

Thinking ● Clarifying pull request steps I'm understanding that PR #36... ## Related Issue Refs #36 ## Purpose Add a persistent project backlog...

Thinking ● Rewriting PR description I'm updating the PR to close issue... should i not also edit the ai decleration and ai docs stuff with this chat?...

Thinking ● Clarifying AI declarations I'm distinguishing document... Pasted text(1).txt i mean this doc?

Thinking ● Matching CSV entries Yes—this appears to be the usage... ive verified those stuff, fix the 2 csv entries for me

Thinking ● Updating verification status I'm marking both entries as...



can u explain what the rewuirments for basics all mean

8/6/2026, 9:31:33 AM



What “Basic” means overall

To reach the **Basic tier**, your group must:

1. Meet **all the requirements that apply to every project**.
2. Meet **all the Sport Analytics Basic requirements**.
3. Produce a **minimum viable product**, meaning a small but complete system that genuinely works from beginning to end. `project_briefs.pdf`

So you cannot choose only some of the Basic requirements. They all need to work.

A. Requirements that apply to every project

1. Version control

You must use Git through Gitea to manage your code.

In practice:

- Everyone should contribute through commits.
- Work should happen on branches.
- Branches should connect to issues.
- Changes should be merged through pull requests.
- Your repository should have a sensible structure and README.

You are already doing this.

2. Responsiveness and accessibility

Responsiveness

The website must work properly on different screen sizes:

- Desktop
- Laptop
- Tablet
- Mobile

Pages should not break, overlap or require strange horizontal scrolling.

Accessibility

People with disabilities should also be able to use the platform.

Examples include:

- Form fields have labels.
- Buttons clearly explain what they do.
- The website can be used with a keyboard.
- Images have alternative text where necessary.
- Text is readable.
- Colour is not the only way information is communicated.
- Error messages clearly explain the problem.

The requirement is not merely “make it look good”. It must also be usable.

`project_briefs.pdf`

3. CI/CD

Continuous Integration

Whenever someone pushes code or opens a pull request, your automated workflow should check things such as:

- Formatting
- Linting
- Type checking
- Tests
- Whether the frontend builds

- Whether the backend builds

Your current `npm run check` workflow is part of this.

Continuous Deployment

Your application should also have an automated or repeatable way of deploying the latest approved version.

For example:

- Merge into `main`
- CI checks pass
- Frontend deploys to Cloudflare Pages
- Backend deploys to Azure

You should not have to manually copy files every time you deploy.

4. Non-monolithic frontend and backend

Your frontend and backend must be separate applications or clearly separated systems.

For your project, this could mean:

```
frontend/  
backend/  
contracts/
```

The frontend:

- Displays pages
- Collects user input
- Sends requests to the API

The backend:

- Receives API requests
- Checks permissions

- Validates data
- Calculates statistics
- Communicates with the database

The requirement means you should not hide the entire backend inside the same framework that handles the frontend. The brief says a framework such as Next.js or SvelteKit may be used for either the frontend or backend, but not both together as one combined application. [project_briefs.pdf](#)

Your current monorepo can still satisfy this, as long as the frontend and backend remain separate projects.

5. Handwritten API

Your group must design and implement the API yourselves.

That means you must create endpoints such as:

```
GET /api/fixtures
GET /api/fixtures/:fixtureId/events
GET /api/players/:playerId/statistics
POST /api/submissions
```

You may use Supabase as a database, but you should not simply let Supabase automatically generate all your public API endpoints.

Your frontend should normally communicate with your backend through HTTP requests.

The important distinction is:

```
Acceptable:
Frontend → Your API → Supabase database
```

```
Potentially unacceptable:
Frontend → Automatically generated Supabase API for everything
```

You can still use Supabase for database storage and authentication, but the important Sport Analytics API must be designed and implemented by your team.

[project_briefs.pdf](#)

6. Authentication and security

Users must be able to:

- Sign up
- Sign in
- Reset their password
- Delete their account

You must not build password security yourselves.

Instead, use an established authentication system such as:

- Microsoft authentication
- Google authentication
- Supabase Auth
- Auth.js
- Another accepted authentication library

Your platform will also need roles because not every signed-in user should be allowed to submit sport data.

For example:

Role	Permission
Public user	View published statistics
Analyst	View and export data
Approved submitter	Upload event data
Administrator	Approve submitters and manage access

Authentication answers **“Who is this person?”**

Authorisation answers **“What is this person allowed to do?”**

You need both.

7. External API integration

You must use at least one relevant API created by someone outside your group.

For a Sport Analytics platform, possible examples include:

- A weather API to show weather during a fixture
- A venue or mapping API
- A professional league information API
- An external identity provider for OAuth
- A recognised sports reference-data API

The integration should make sense for your product. Adding a random API only to tick the requirement would be weak. `project_briefs.pdf`

8. Public documentation website

You need a separate, publicly accessible documentation website.

It could contain:

- Project overview
- System architecture
- Technology stack
- Setup instructions
- Developer guide
- API documentation
- Event schema
- Deployment guide
- Contribution process
- Git methodology
- Project methodology

The brief suggests tools such as:

- Docusaurus

- MkDocs
- mdBook

It can be hosted using something such as Cloudflare Pages. People should not need an account to read it. `project_briefs.pdf`

B. Sport Analytics Basic requirements

1. Choose a professional sport, league or tour

Your system is not meant to be a generic scoreboard for every sport.

You should choose a particular sport and define:

- What a fixture is
- Who the competitors are
- Which events occur
- Which statistics matter
- Who is allowed to submit data

For example, depending on the sport, a fixture might be:

- A football match
- A cricket match
- A tennis match
- A Formula 1 race
- A swimming heat

The data model will depend heavily on this choice.

2. Build the platform around event data

This is probably the **most important requirement**.

You should not store only final totals typed in by a person.

For example, do not only store:

Player A scored 24 points

Instead, store the individual events that produced that result:

```
10:02 - Player A scored 2 points
11:18 - Player A scored 3 points
14:40 - Player A scored 2 points
...
```

The platform then calculates the total from those events.

For football, events might include:

- Kick-off
- Goal
- Shot
- Foul
- Yellow card
- Substitution
- Offside
- Final whistle

For cricket, events might include:

- Delivery bowled
- Runs scored
- Wicket
- Wide
- No-ball
- Bye
- Leg bye

Every event should be connected to one fixture and stored in the order in which it happened. `sport_analytics.pdf`

3. Calculate statistics from events

Statistics must be derived automatically.

For example:

```
Goals = count of goal events
Yellow cards = count of yellow-card events
Shots on target = count of shot events where outcome = on-target
Possession = calculated from relevant event or tracking data
```

The user should not separately type:

```
Goals: 3
Shots: 12
Cards: 2
```

Those totals should come from the underlying event records.

This means your system needs a **derivation layer**, which is code that converts events into statistics.

4. Correcting an event must update the statistics

Suppose a submitter records:

```
Player 8 scored the goal
```

Later, it is corrected to:

```
Player 10 scored the goal
```

The system should automatically:

- Remove one goal from Player 8
- Add one goal to Player 10
- Update team and fixture statistics
- Update anything else that depends on that goal

Nobody should need to manually correct every affected statistic.

The source event is corrected, and everything derived from it should fall back into line.

`sport_analytics.pdf`

5. Event data must arrive through submissions

Events must enter your system as part of a submission.

A submission might be:

- A JSON file upload
- A CSV file upload
- Events submitted through a web form
- Events sent directly to an API endpoint

For example:

```
Submission
├─ submittedBy: User 42
├─ fixtureId: Fixture 19
├─ submittedAt: 2026-08-20
└─ events:
    ├─ Event 1
    ├─ Event 2
    └─ Event 3
```

You need to keep the submission itself, not merely copy its events into the database and forget where they came from.

6. Only approved submitters may submit

Not every registered account may upload official sport data.

A normal user may be able to view information, while an approved submitter can upload events.

A possible approval process is:

1. User creates an account.
2. Administrator approves them as a submitter.
3. Submitter is assigned to a defined part of the competition.

4. The backend checks their permissions when they submit.

This is an authorisation requirement, not merely a login requirement.

`sport_analytics.pdf`

7. Each submitter needs a defined scope

A submitter should only be allowed to submit within an assigned part of the competition.

For example:

- A submitter may be assigned to one team.
- A submitter may be assigned to one venue.
- A submitter may be assigned to a group of fixtures.
- A submitter may be assigned to one competition or season.

This makes it clear who was responsible for particular data and stops someone from editing fixtures outside their responsibility.

You may model this as something similar to:

```
Submitter Assignment
userId: 42
competitionId: League 2026
teamId: Team A
allowedFrom: 2026-08-01
allowedUntil: 2026-10-31
```

8. Create an event schema

An event schema defines exactly what valid event data must look like.

For example:

```
{
  "fixtureId": "fixture-123",
  "sequenceNumber": 17,
  "eventType": "goal",
  "competitorId": "team-4",
  "playerId": "player-22",
```



```
"occurredAt": "00:43:12"
}
```

The schema might specify:

- Required fields
- Allowed event types
- Data types
- Valid identifier formats
- Whether a field may be empty
- Sport-specific restrictions

This gives everyone a clear contract for submitting events.

9. Validate submissions before accepting them

Every submission must be checked against the event schema.

The system should reject things such as:

- Missing fixture ID
- Invalid player ID
- Unknown event type
- Incorrect date format
- Missing sequence number
- Text where a number is expected
- Event assigned to the wrong fixture
- Submitter working outside their assigned scope

Invalid data should not be silently accepted.

10. Rejections must explain what is wrong

It is not enough to return:

Upload failed

The submitter should receive useful feedback, for example:

Submission rejected.

Event 14:

- playerId is required for event type "goal"

Event 21:

- eventType "redcard" is invalid
- expected "red_card"

Event 30:

- sequenceNumber 18 has already been used

The requirement specifically says a submission should not fail quietly.

sport_analytics.pdf

11. Published statistics must be traceable

For every statistic, you should be able to identify:

Statistic

↓

Events used in the calculation

↓

Submission containing those events

↓

Submitter who provided the submission

For example, when displaying:

Player A: 7 goals

your system should be able to find the seven goal events used to calculate that figure.

This is sometimes called **data lineage** or **provenance**.

It makes your numbers explainable and auditable.

12. The API is the main product

The web interface is useful, but the brief says the API should be treated as the platform's actual product.

That means another developer should be able to use your system without using your frontend.

For example:

```
GET /api/v1/fixtures
GET /api/v1/events
GET /api/v1/statistics
```

The API should be properly designed, documented and externally usable.

`sport_analytics.pdf`

13. The API must expose fixtures

A fixture is one competition event, such as a match, race or heat.

The API should allow another platform to retrieve fixtures.

For example:

```
GET /api/v1/fixtures
```

Possible result:

```
{
  "id": "fixture-123",
  "competitionId": "league-2026",
  "startedAt": "2026-08-20T17:00:00Z",
  "status": "completed"
}
```

14. The API must expose individual events

Consumers should be able to retrieve the event records behind the statistics.

For example:

```
GET /api/v1/fixtures/fixture-123/events
```

This is important because analysts may want to perform calculations that your platform does not already provide.

15. The API must expose derived statistics

The API should return calculated statistics, not only raw events.

For example:

```
GET /api/v1/players/player-22/statistics
```

Possible result:

```
{
  "playerId": "player-22",
  "appearances": 12,
  "goals": 7,
  "assists": 4
}
```

Those values must still be calculated from event data.

16. API requests must support filtering

A consumer should be able to request only the data they need.

For example:

```
GET /api/v1/events?fixtureId=123
GET /api/v1/events?eventType=goal
GET /api/v1/fixtures?season=2026
GET /api/v1/statistics?playerId=22
```

Without filters, analysts would need to download the entire database to find a small amount of information.

17. The API must support pagination

Some requests may return thousands of records. You should not return everything in one enormous response.

Instead, divide the response into pages.

For example:

```
GET /api/v1/events?page=2&pageSize=100
```

Or use cursor-based pagination:

```
GET /api/v1/events?limit=100&cursor=abc123
```

The response should tell the consumer how to retrieve the next page.

18. Identifiers must be stable

Fixtures, players, teams, submissions and events need permanent identifiers.

For example:

```
fixture-01J8XQZ8...  
player-01J9ABCD...
```

A fixture ID from the previous season should still point to the same fixture later.

Do not create IDs based on unstable information such as:

```
team-name-date
```

because team names and other descriptive fields may change.

The requirement means that updating a record must not give it a completely new identity. `sport_analytics.pdf`

19. Analysts must be able to export filtered data

Users should be able to select part of the dataset and download it as a file.

For example:

- All goal events from the 2026 season
- All events for one team
- All fixtures during a date range
- All statistics for selected players

Possible formats include:

- CSV
- JSON

For example:

```
GET /api/v1/exports/events.csv?season=2026&eventType=goal
```

The export should respect the same filtering rules as the API.

The simplest complete Basic workflow

Your Basic product should be able to demonstrate this full process:

1. User signs up and signs in
2. Administrator approves the user as a submitter
3. Administrator assigns the submitter a competition scope
4. Submitter uploads a fixture event file
5. Backend validates the submission against the event schema
6. Invalid submissions receive detailed errors
7. Valid events are stored in their original order
8. Statistics are calculated from those events
9. Statistics appear in the frontend
10. Statistics and events are available through the API
11. API results can be filtered and paginated
12. A filtered dataset can be exported
13. Correcting an event updates the relevant statistics
14. Every statistic can be traced to its source events and submission

That is the **minimum complete Sport Analytics system** described by the Basic requirements. `sport_analytics.pdf`

8/6/2026, 9:31:34 AM



give me that message as a pdf

8/6/2026, 9:36:18 AM



Thinking



Reading PDF creation guidelines

- **Created and verified PDF requirements guide with corrected numbering**

✓ Worked for 3m 52s

[Download the PDF: Sport Analytics Basic Requirements Explained](#)

8/6/2026, 9:36:19 AM



Sport-Analytics-Tool (3).zip

now, make for me, using all git and rpject methodology, all the issues needed, with labels etc for the basic reqs (include sign in buttons, blah blah)

8/6/2026, 10:00:15 AM



Thinking



Inspecting repository and methodology documents

- **Preparing issue set**

I'm organizing actionable, non-duplicative GitHub issues around the remaining architecture, sport modules, shared services, testing, accessibility, security, and deployment gaps reflected in the project materials.

- **Prepared issue set**

- **Structuring remaining work**

I'm grouping remaining work into focused GitHub issues covering shared architecture, sport analytics, data modeling, APIs, frontend workflows, accessibility, integrations, documentation, testing, and deployment rather than repeating completed foundations.

- **Structured the remaining work**



Creating Gitea Basic Requirements Issue Plan

- **Inspected repository documentation and created Gitea requirements plan**

✓ Worked for 16m 30s

I reviewed the current repository and created a **complete copy-and-paste Gitea backlog of 50 issues** for the remaining Basic requirements.

Each issue includes:

- Exact Gitea title



- Feature, task or bug template
- `type:` and `area:` labels
- Priority label where appropriate
- Sprint milestone
- Suggested owner role
- Suggested branch name
- Description and motivation
- Included and excluded scope
- Detailed acceptance criteria
- Dependencies
- Testing and evidence requirements
- Security, accessibility and responsiveness checks
- AI declaration

It covers sign-up, **Sign in and Create account buttons**, sign-out, password reset, account deletion, administrator approval, submitter scopes, fixtures, event submission, validation, statistics, correction, traceability, export, external API integration, testing, documentation and final acceptance.

I also accounted for work already completed in the repository, rather than duplicating it. I included a high-priority issue to repair the current Azure workflows because they still refer to the old `frontend/` and `backend/` paths instead of `apps/frontend` and `apps/backend`.

[Download the complete Gitea Basic Requirements issue backlog](#)

The backlog follows the Basic project requirements, the all-project technical requirements and the AI attribution rules. `sport_analytics.pdf` `project_briefs.pdf` `ai_policy_20_jul.pdf`

8/6/2026, 10:00:17 AM



We want to do as much of this as possible in sprint 1, so, can u gived me all the titles and labels etc, ill put them in and sent u numbers, and then we can do all the descriptions etc? does that work?

8/6/2026, 10:24:14 AM



Thinking



Reading requirements issue backlog

- **Organizing issue setup**

I'm separating the full backlog from Sprint 1 commitments. I'll provide titles, labels, and suggested milestones first, with foundational issues as Sprint 1 targets and later features marked as stretch work.

- **Organized issue setup**

- **Read requirements issue backlog**

✓ Worked for 26s

Yes, that works well.

Create the issues first using only the **title, template, labels, milestone and board column** below. Send me the Gitea issue numbers afterward in the same order, and I will produce each full description with the correct dependencies, branch name, acceptance criteria, testing, documentation and AI declaration.

Do **not** assign all 50 to Sprint 1. Create all of them now, but use:

- **Sprint 1 Core:** issues 1-21
- **Sprint 1 Stretch:** issues 22-32 and 40, pulled into Sprint 1 only when dependencies and capacity allow
- **Later milestones:** the remaining issues

This keeps your methodology honest while still allowing you to complete more than the minimum during Sprint 1.

Labels that should exist

Type labels

- `type: feature`
- `type: bug`
- `type: documentation`
- `type: testing`
- `type: research`
- `type: infrastructure`

Area labels

- `area: frontend`
- `area: api`
- `area: database`
- `area: data`
- `area: infrastructure`
- `area: documentation`

Other labels

- `priority: high`
- `status: blocked`

Set every new issue's initial board column to **Backlog**.

Sprint 1 Core

1. Confirm the sport, professional competition scope, event vocabulary and required statistics

Template: Task

Labels: `type: research`, `area: data`, `area: documentation`, `priority: high`

Milestone: Sprint 1

2. Define user roles, permissions, submitter approval, scopes and account lifecycle

Template: Task

Labels: type: research, area: frontend, area: api, area: database, area: documentation, priority: high

Milestone: Sprint 1

3. Produce the Basic information architecture, user journeys and responsive wireframes

Template: Task

Labels: type: documentation, area: frontend, area: documentation, priority: high

Milestone: Sprint 1

4. Align Gitea branch protection and Pull Request review enforcement with the Git methodology

Template: Task

Labels: type: infrastructure, area: infrastructure, area: documentation, priority: high

Milestone: Sprint 1

5. Repair Azure deployment workflows for the current monorepo and verify deployed smoke checks

Template: Bug

Labels: type: bug, area: infrastructure, priority: high

Milestone: Sprint 1

6. Define and implement shared API identifiers, errors, filtering, sorting and pagination contracts

Template: Feature

Labels: type: feature, area: api, area: data, area: documentation, priority: high

Milestone: Sprint 1

7. Implement repeatable database migration and development seed tooling

Template: Task

Labels: type: infrastructure, area: database, area: infrastructure, priority:

high

Milestone: Sprint 1

8. Define the executable sport-specific event schema and example submissions

Template: Feature

Labels: type: feature , area: data , area: api , area: documentation , priority: high

Milestone: Sprint 1

9. Implement the competition, season, competitor, participant and fixture database schema

Template: Feature

Labels: type: feature , area: database , area: data , priority: high

Milestone: Sprint 1

10. Implement application accounts, roles, submitter approval and scope database schema

Template: Feature

Labels: type: feature , area: database , area: api , priority: high

Milestone: Sprint 1

11. Implement submission, ordered event and provenance database schema

Template: Feature

Labels: type: feature , area: database , area: data , area: api , priority: high

Milestone: Sprint 1

12. Implement the backend database access, repository and transaction foundation

Template: Feature

Labels: type: feature , area: api , area: database , priority: high

Milestone: Sprint 1

13. Configure managed Firebase providers for sign-up, sign-in and password-reset requirements

Template: Task

Labels: type: infrastructure, area: infrastructure, area: frontend, area: api, priority: high

Milestone: Sprint 1

14. Implement application account synchronisation and the authenticated profile API

Template: Feature

Labels: type: feature, area: api, area: database, priority: high

Milestone: Sprint 1

15. Implement frontend authentication state, protected routing and authenticated API requests

Template: Feature

Labels: type: feature, area: frontend, area: api, priority: high

Milestone: Sprint 1

16. Build sign-up, sign-in, sign-out and authentication navigation controls

Template: Feature

Labels: type: feature, area: frontend, priority: high

Milestone: Sprint 1

This includes the visible **Create account**, **Sign in**, **Account** and **Sign out** buttons.

17. Build the forgotten-password and password-reset request flow

Template: Feature

Labels: type: feature, area: frontend, priority: high

Milestone: Sprint 1

18. Implement secure account deletion across the frontend, API, Firebase and application data

Template: Feature

Labels: type: feature, area: frontend, area: api, area: database, priority: high

Milestone: Sprint 1

19. Implement reusable role, approval and competition-scope authorisation middleware

Template: Feature

Labels: type: feature, area: api, priority: high

Milestone: Sprint 1

20. Implement administrator APIs for submitter approval, revocation and scope assignment

Template: Feature

Labels: type: feature, area: api, area: database, priority: high

Milestone: Sprint 1

21. Build the administrator submitter-approval and scope-management interface

Template: Feature

Labels: type: feature, area: frontend, priority: high

Milestone: Sprint 1

Sprint 1 Stretch Features

Create these under **Sprint 2 initially**. Move them into Sprint 1 only when their dependencies are ready and someone genuinely has capacity.

22. Implement public competition, season, competitor and fixture read APIs

Template: Feature

Labels: type: feature, area: api, area: data, priority: high

Milestone: Sprint 2

23. Implement administrator CRUD APIs for competitions, seasons, competitors and fixtures

Template: Feature

Labels: type: feature, area: api, area: database, priority: high

Milestone: Sprint 2

24. Build public competition, competitor and fixture browsing pages

Template: Feature

Labels: type: feature , area: frontend

Milestone: Sprint 2

25. Build administrator competition, competitor and fixture management pages

Template: Feature

Labels: type: feature , area: frontend

Milestone: Sprint 2

26. Implement direct JSON event submission with validation, authorisation and provenance

Template: Feature

Labels: type: feature , area: api , area: data , priority: high

Milestone: Sprint 2

27. Implement JSON file-upload submission using the shared validation pipeline

Template: Feature

Labels: type: feature , area: api , area: data , area: infrastructure

Milestone: Sprint 2

28. Implement submission list, detail and validation-result APIs

Template: Feature

Labels: type: feature , area: api , area: data

Milestone: Sprint 2

29. Build the approved-submitter event submission and result interface

Template: Feature

Labels: type: feature , area: frontend , priority: high

Milestone: Sprint 2

30. Implement the public events API with filtering, deterministic order and pagination

Template: Feature

Labels: type: feature, area: api, area: data, priority: high

Milestone: Sprint 2

31. Implement the Basic sport-specific statistic derivation engine

Template: Feature

Labels: type: feature, area: data, area: api, priority: high

Milestone: Sprint 2

32. Implement derived fixture-statistics API endpoints

Template: Feature

Labels: type: feature, area: api, area: data, priority: high

Milestone: Sprint 2

Remaining Sprint 2 Issues

33. Build fixture and competitor statistics views

Template: Feature

Labels: type: feature, area: frontend

Milestone: Sprint 2

34. Implement authorised event correction with automatic statistic refresh

Template: Feature

Labels: type: feature, area: api, area: data, priority: high

Milestone: Sprint 2

35. Build the approved event-correction interface

Template: Feature

Labels: type: feature, area: frontend

Milestone: Sprint 2

36. Implement statistic provenance and “how calculated” traceability

Template: Feature

Labels: type: feature, area: api, area: frontend, area: data, priority: high

Milestone: Sprint 2

37. Implement filtered dataset export through the API and frontend

Template: Feature

Labels: type: feature, area: api, area: frontend, area: data

Milestone: Sprint 2

38. Select a relevant external API service and record the integration decision

Template: Task

Labels: type: research, area: api, area: documentation

Milestone: Sprint 2

39. Implement the selected external API integration in the backend and frontend

Template: Feature

Labels: type: feature, area: api, area: frontend, area: infrastructure

Milestone: Sprint 2

40. Maintain an OpenAPI specification and publish complete Basic API documentation

Template: Task

Labels: type: documentation, area: api, area: documentation, priority: high

Milestone: Sprint 2

This can begin during Sprint 1 as the API contracts are designed.

41. Add backend integration tests for the complete Basic data workflow

Template: Task

Labels: type: testing, area: api, area: database, area: data, priority: high

Milestone: Sprint 2

42. Add frontend integration tests for the main Basic user workflows

Template: Task

Labels: type: testing, area: frontend, area: api

Milestone: Sprint 2

Sprint 3 Issues

43. Add a browser-level end-to-end Basic acceptance workflow

Template: Task

Labels: type: testing, area: frontend, area: api, area: database, area: data, priority: high

Milestone: Sprint 3

44. Conduct formal user testing and integrate prioritised feedback

Template: Task

Labels: type: testing, area: frontend, area: documentation

Milestone: Sprint 3

45. Complete the Basic accessibility and responsive-design audit and fixes

Template: Task

Labels: type: testing, area: frontend, area: documentation, priority: high

Milestone: Sprint 3

46. Complete the Basic security, privacy and dependency hardening review

Template: Task

Labels: type: testing, area: api, area: database, area: infrastructure, priority: high

Milestone: Sprint 3

47. Complete database schema documentation, ERD and data dictionary

Template: Task

Labels: type: documentation, area: database, area: data, area: documentation

Milestone: Sprint 3

48. Complete Basic user, developer, deployment and operations documentation

Template: Task

Labels: type: documentation, area: documentation, area: infrastructure

Milestone: Sprint 3

49. Create the Basic requirements traceability matrix

Template: Task

Labels: type: documentation, area: documentation, area: data, priority: high

Milestone: Sprint 3

Final Submission Issue

50. Perform the final Basic acceptance audit and prepare the reproducible demonstration dataset

Template: Task

Labels: type: testing, area: data, area: documentation, area: infrastructure, priority: high

Milestone: Final Submission

Create them in this exact order. Then send me something like:

```
B01 = #18
B02 = #19
B03 = #20
...
B50 = #67
```

I will replace all the temporary B-numbers with the real Gitea issue references and prepare the descriptions in manageable batches.

8/6/2026, 10:24:15 AM



restructure, give me all, we want to hit all the reqs for intermediate and advanced too by sprint 3 end
lets do: all issues for entire project with tags etc, then we can expand sprint 1 stuff, is that better? whats the best way to do it?

8/6/2026, 10:28:35 AM



Thinking



Inspecting repository structure and methodology docs

- **Reviewed repository structure and project methodology documentation**

- **Planning the backlog structure**

I'm organizing the work into milestones and issue-sized tasks, while keeping labels minimal. I'll suggest a staged approach: establish the full backlog now, then expand Sprint 1 with detailed acceptance criteria and dependencies.

- **Planned the backlog structure**
- **Planned sport analytics project backlog and requirements strategy**



Inspecting Repository Documentation and Package Configuration

- **Designing the backlog**

I'm planning a complete master backlog organized by milestone and tier, with Sprint 1 fully specified first. I'll keep implementation realistic by treating advanced features as gated targets while prioritizing reliable foundations.

- **Designed the backlog**



Reviewing Git history to map existing project issues

- **Inspected repository documentation, issues, history, references, and workflows**

✓ Worked for 5m 55s

Yes - this is the better approach

Create the **complete project backlog now**, but do not fully expand every issue yet.

The best workflow is:

1. **Create every issue title, label and milestone** so the complete Basic, Intermediate and Advanced roadmap is visible.
2. Put new issues in **Backlog**, not automatically in Ready.
3. Map work already completed to existing issue numbers instead of creating duplicates.
4. Send me the resulting issue numbers.
5. We then fully write the **Sprint 1 descriptions first**, including acceptance criteria, dependencies, testing, evidence, branch names and AI declarations.
6. Sprint 2 and Sprint 3 descriptions are refined one sprint ahead as stakeholder decisions and technical discoveries become clearer.

This follows your methodology because an issue may only move to **Ready** once it has clear acceptance criteria, dependencies, verification and an assignee. It also avoids pretending that distant Advanced work is already fully defined.

Project_Methodology.md

Recommended target

Your practical target should be:

- **Sprint 1:** Working vertical slice and foundations.
- **Sprint 2:** Complete Basic and most or all Intermediate capabilities.
- **Sprint 3:** Advanced feature completeness.
- **Final Submission:** Hardening, defects, production verification, documentation, evidence and presentation preparation.

The brief describes Sprint 3 as near-complete and the final submission as the point where the application must work well. The Advanced standard is effectively market-ready, so aiming for Advanced **feature completeness** by Sprint 3 while reserving the final milestone for release work is the strongest realistic interpretation.

project_briefs.pdf

The Advanced requirements are substantial: custom statistic definitions, live feeds, out-of-order processing, temporal queries, dataset comparisons, asynchronous API jobs, change feeds, API deprecation, anomaly detection and submitter reconciliation. Therefore, some Advanced architecture and high-risk research begins in Sprint 1 rather than being left until Sprint 3. `sport_analytics.pdf`

Complete catalogue

I created a catalogue containing:

- **122 total entries**
- **8 existing foundation items to map**
- **114 new issues to create**
- Template type
- All labels
- Tier
- Milestone
- Initial board position
- Basic, Intermediate and Advanced coverage
- Final submission and reporting work

[Download the complete Gitea issue catalogue](#)

Four labels to add

Your existing type and area labels remain unchanged. Add only:

```
tier: common
tier: basic
tier: intermediate
tier: advanced
```

These are worthwhile because they let you filter the backlog by requirement tier without adding unnecessary label clutter.

What to do now

Create or map the entries in catalogue order. Do not write the descriptions yet. Send the numbers back like this:

```
EX01 = #2
EX02 = #7
EX03 = #10
...

S1-01 = #27
S1-02 = #28
S1-03 = #29
...
```

We will then expand Sprint 1 into complete, copy-ready Gitea issues and identify which ones can safely be worked on in parallel by the six group members. Each implementation will retain the required issue → branch → commits → Pull Request → merge traceability. `Git_Methodology.md`

8/6/2026, 10:28:36 AM



so does mapping mean to make them in issues or what? also, if i have existing issues im not recreating them, right?

8/6/2026, 10:39:59 AM



Correct — **do not recreate existing issues.**

“Mapping” means matching an item in my catalogue to the Gitea issue that already covers that work.

For example:

```
EX03 = #10
```

means:

```
Catalogue item EX03  
"Configure the CI quality gate..."  
is already covered by Gitea issue #10
```

You do not create a new issue for EX03. You simply tell me which existing issue number matches it.

Use these rules:

- **Existing issue covers the same work:** map it to that issue number.
- **Existing issue covers most of the work:** map it, and we can update its description or acceptance criteria.
- **Existing issue only covers a small part:** keep it, then create a separate issue for the missing work.
- **Work is already completed but has an existing closed issue:** map it to the closed issue.
- **Work was completed without an issue:** ideally create a retrospective issue only if you need traceability for the rubric. Do not falsely present it as future work.

Example:

```
EX01 = #1  
EX02 = #7  
EX03 = #10  
EX04 = #12  
  
S1-01 = #18  
S1-02 = #19
```

Here, the **EX** items point to issues you already had, while the **S1** items are newly created issues.

So your process now is:

1. Check the **EX01-EX08** entries against your current issues.
2. Send me the existing issue numbers that match them.
3. Create only catalogue items that are not already covered.
4. Send me all the final mappings and new issue numbers.
5. We then expand the Sprint 1 descriptions.

Also avoid creating two issues merely because their titles are worded differently. The deciding factor is whether their actual **scope and acceptance criteria** are the same.

8/6/2026, 10:40:08 AM



so what am i creating then from the catalogue? issues?

8/6/2026, 10:40:41 AM



Yes. The catalogue is a list of **Gitea issues for the entire project.**

For every row marked **Create**, create one new issue in Gitea using:

- **Title**
- **Template:** Feature, Task or Bug
- **Labels**
- **Milestone**
- **Board column:** Backlog

For every row marked **Map existing**, do **not** create anything. Match it to an issue you already have.

Example:

```
EX03 – Map existing  
Configure the CI quality gate...
```

You would send:

```
EX03 = #10
```

But:

```
S1-01 – Create  
Confirm the sport, professional competition scope...
```

means you create a new Gitea issue, then send:

```
S1-01 = #18
```

Also, when a row says **Create** but one of your existing issues already fully covers it, do not duplicate it. Map it to the existing issue instead.

So the catalogue works like this:

```
Map existing = find the matching current issue  
Create = make a new Gitea issue
```

For now, only add the title, labels, milestone and template. Leave the description blank or write:

```
Full issue specification to be added during backlog refinement.
```

Once you send me the issue numbers, we will fully expand the Sprint 1 issues first.

8/6/2026, 10:40:48 AM



ok, for sake of time, give me the 'most urgent', first issues for now, and ill add the rest later
do i need to create an issue called making the backlog or something??

8/6/2026, 1:52:01 PM



Thinking

- **Planning urgent issues**

I'm prioritizing foundational work first: scope and requirements, backlog setup, architecture, CI reliability, branch protection, API contracts, database migrations, authentication, event validation, and one complete vertical slice. I'll flag duplicates so existing issues aren't recreated unnecessarily.

- **Planned urgent issues**

- **Structured sport analytics project scope, priorities, templates, and milestones**

✓ Worked for 45s

Yes - create **one backlog-planning issue**, unless an existing issue already covers setting up and populating the Gitea work tracker.

Use:

Title: Create and prioritise the complete project backlog

Template: Task

Labels: type: documentation , area: documentation , priority: high

Milestone: Sprint 1

Board: Backlog

This gives evidence that the work tracker is being actively used, which is assessed in Sprint 1. Your methodology also states that Gitea is the project's source of truth and meaningful work must be recorded as issues. `Project_Methodology.md`
`project_briefs.pdf`

If you already have an issue called something like “**Set up Gitea project board**” or “**Create project work tracker**”, update that issue to include backlog creation instead of making another one.

Most urgent issues to create now

Create these in this order. Every issue starts in **Backlog** until we add the full description and acceptance criteria.

1. Create and prioritise the complete project backlog

Template: Task

Labels: type: documentation, area: documentation, priority: high

Milestone: Sprint 1

Only create this if an existing planning or project-board issue does not already cover it.

2. Confirm the sport, professional competition scope, event vocabulary and required statistics

Template: Task

Labels: type: research, area: data, area: documentation, priority: high

Milestone: Sprint 1

This must happen first because the database, event schema, statistics and UI all depend on the sport you select.

3. Record stakeholder decisions for the Basic, Intermediate and Advanced requirements

Template: Task

Labels: type: research, area: documentation, priority: high

Milestone: Sprint 1

This should capture questions asked, answers received, disputed interpretations and what the tutor considers complete.

4. Produce the complete system architecture, data flow and development roadmap

Template: Task

Labels: type: documentation, area: api, area: database, area: infrastructure,

area: documentation, priority: high

Milestone: Sprint 1

This supports the heavily weighted Sprint 1 design and development-plan criterion.

project_briefs.pdf

5. Align Gitea branch protection and Pull Request review enforcement with the Git methodology

Template: Task

Labels: type: infrastructure, area: infrastructure, area: documentation, priority: high

Milestone: Sprint 1

Do not recreate this if you already made the branch-protection issue.

6. Repair Azure deployment workflows for the current monorepo and verify deployed smoke checks

Template: Bug

Labels: type: bug, area: infrastructure, priority: high

Milestone: Sprint 1

Do not recreate this if the Azure person already has the corresponding issue. Use the existing issue number.

7. Define stable identifiers and shared API response, error, filtering and pagination contracts

Template: Feature

Labels: type: feature, area: api, area: data, area: documentation, priority: high

Milestone: Sprint 1

The API is the main product, so its contracts should be agreed before several people independently create inconsistent endpoints. sport_analytics.pdf

8. Create the OpenAPI baseline and API versioning conventions

Template: Task

Labels: type: documentation, area: api, area: documentation, priority: high

Milestone: Sprint 1

This can be completed alongside the API-contract issue.

9. Implement repeatable database migrations and development seed tooling

Template: Task

Labels: type: infrastructure, area: database, area: infrastructure, priority: high

Milestone: Sprint 1

This allows all team members and CI to construct the same database reliably.

10. Implement the competition, season, competitor, participant and fixture database schema

Template: Feature

Labels: type: feature, area: database, area: data, priority: high

Milestone: Sprint 1

11. Define the executable sport-specific event schema and example submissions

Template: Feature

Labels: type: feature, area: data, area: api, area: documentation, priority: high

Milestone: Sprint 1

This should define event order, types, required fields and valid and invalid examples.

12. Implement the account, role, approved-submitter and competition-scope schema

Template: Feature

Labels: type: feature, area: database, area: api, priority: high

Milestone: Sprint 1

The project requires approved submitters whose permissions are restricted to a defined part of the competition. `sport_analytics.pdf`

13. Configure sign-up, sign-in, sign-out and password-reset authentication

Template: Feature

Labels: type: feature, area: frontend, area: api, area: infrastructure, priority: high

Milestone: Sprint 1

Do not recreate this if your Firebase authentication issue already includes these actual user flows rather than only token-verification setup.

14. Build the Create Account, Sign In, Account and Sign Out navigation controls

Template: Feature

Labels: type: feature, area: frontend, priority: high

Milestone: Sprint 1

This includes the visible buttons and pages users actually interact with.

15. Implement event validation with useful rejection messages

Template: Feature

Labels: type: feature, area: api, area: data, priority: high

Milestone: Sprint 1

Invalid submissions must explain exactly what is wrong rather than failing silently.

`sport_analytics.pdf`

16. Implement direct fixture-event submission with authorisation and provenance

Template: Feature

Labels: type: feature, area: api, area: database, area: data, priority: high

Milestone: Sprint 1

This should record who submitted the events, which fixture they relate to and their original order.

17. Implement the event-driven statistic derivation engine and fixture-statistics API

Template: Feature

Labels: type: feature , area: api , area: data , priority: high

Milestone: Sprint 1

This is the core of the project: statistics must be calculated from events rather than manually entered totals. `sport_analytics.pdf`

18. Build the first public fixture, event and statistics browsing pages

Template: Feature

Labels: type: feature , area: frontend , priority: high

Milestone: Sprint 1

This gives you a demonstrable end-to-end vertical slice.

19. Add automated tests for the Sprint 1 vertical slice

Template: Task

Labels: type: testing , area: frontend , area: api , area: database , area: data , priority: high

Milestone: Sprint 1

20. Publish Sprint 1 setup, architecture, API and testing documentation

Template: Task

Labels: type: documentation , area: documentation , area: api , area: infrastructure , priority: high

Milestone: Sprint 1

Do not recreate this if your documentation-site issue already includes producing real content. A deployed but nearly empty documentation site only earns limited credit.

Important rule

Before creating each issue, search your existing issues by keyword.

Already covered fully → use the existing issue

Partly covered → update the existing issue or create only the missing follow-up

Not covered → create the new issue

These 20 issues give you the planning evidence, remove the main technical blockers and establish the core event → submission → statistic → API → frontend workflow that the Sport Analytics brief is built around. `sport_analytics.pdf`

Send the issue numbers back in this order, including existing ones that you mapped, and we will expand the descriptions starting with the issues that can be worked on immediately.

8/6/2026, 1:52:20 PM



give me full Create and prioritise the complete project backlog iddue desxc etc b4 we do the next

8/6/2026, 2:01:06 PM



Thinking

- **Structuring issue details**

I'm organizing this as a documentation task for Sprint 1, with clear scope boundaries, deliverables, acceptance criteria, ownership, workflow states, and review evidence. I'm keeping feature implementation separate from this issue.

- **Structured issue details**

- **Created Sprint 1 work tracker setup and documentation**

✓ Worked for 48s

Yes. This issue is specifically for **creating, organising and prioritising the project's Gitea backlog**. It does **not** mean that all the project features must be completed before this issue can close.

The Sprint 1 rubric rewards an actively used work tracker, a documented and followed methodology, and a comprehensive development roadmap. Your own methodology also requires meaningful work to be recorded as labelled, assigned and prioritised Gitea issues. `project_briefs.pdf` `Project_Methodology_v1.1.md`

Gitea issue details

Title

Create and prioritise the complete project backlog

Issue template: Task

Labels:

type: documentation
area: documentation

```
tier: common
priority: high
```

Milestone: Sprint 1

Initial board column: Backlog

Suggested assignee: Project lead or backlog coordinator

Suggested branch after the issue number is created:

```
docs/<issue-number>-project-backlog
```

For example, if it becomes issue #24 :

```
docs/24-project-backlog
```

Copy and paste this into the issue description

Description

Create, organise and prioritise the complete Gitea backlog for the Sport Analytics Tool project.

The backlog must cover:

- the requirements that apply to every COMS3011A project;
- the Sport Analytics Basic requirements;
- the Sport Analytics Intermediate requirements;
- the Sport Analytics Advanced requirements;
- testing, documentation, stakeholder interaction and project methodology work;
- deployment, security, accessibility and performance work;
- final submission, reporting and presentation work.

Existing Gitea issues must be reviewed before new issues are created. Work that is already represented by an existing issue must be mapped to that issue rather than duplicated.

The backlog will be created progressively. The urgent Sprint 1 issues must be created and refined first, followed by the remaining Sprint 1, Sprint 2, Sprint 3 and Final Submission issues.

Motivation

The project requires a clear and actively maintained work tracker.

A complete backlog will allow the team to:

- understand the full project scope;
- identify the work needed for Basic, Intermediate and Advanced achievement;
- prioritise the most important and highest-risk work;
- identify dependencies and blockers;
- distribute work between team members;
- avoid forgotten requirements;
- demonstrate consistent use of the agreed project methodology;
- maintain traceability between requirements, issues, branches, commits and Pull Requests;
- provide evidence of planning for the Sprint 1 milestone.

The backlog is a planning tool and must remain current as requirements, stakeholder feedback and technical decisions develop.

Scope

Included

- Review all current Gitea issues.
- Identify which existing issues already cover project requirements.
- Map existing issues to the relevant project requirements.
- Avoid creating duplicate issues.
- Create missing issues for the complete project lifecycle.
- Include project-wide, Basic, Intermediate and Advanced requirements.
- Create separate issues for meaningful units of work.
- Assign the correct issue template to each issue.
- Add applicable type, area, tier and priority labels.
- Assign each issue to the appropriate milestone.
- Place newly created issues in the Backlog board column.
- Identify dependencies between major issues.
- Identify blockers and high-risk work.
- Prioritise urgent Sprint 1 foundation and vertical-slice work.
- Record stakeholder questions where a requirement needs clarification.
- Create or update a repository backlog and requirements-traceability document.
- Review the initial backlog with the team.

- Update the backlog after relevant stakeholder feedback.

Excluded

- Implementing the features represented by the backlog issues.
- Completing all Basic, Intermediate or Advanced functionality.
- Assigning every future issue to a team member before it is ready.
- Moving every issue into Sprint 1.
- Creating duplicate issues for work already represented in Gitea.
- Fully refining every distant Sprint 2 and Sprint 3 issue before its requirements are sufficiently understood.
- Treating the initial backlog as permanently fixed.

Backlog Structure

Each issue must contain, at minimum:

- a clear title;
- an appropriate issue template;
- applicable labels;
- an appropriate milestone;
- an initial board position.

Issues being prepared for immediate implementation must also contain:

- a clear description;
- motivation;
- included and excluded scope;
- acceptance criteria;
- dependencies;
- testing or verification requirements;
- documentation and evidence requirements;
- an assignee;
- a suggested branch name.

Later-sprint issues may initially contain a concise scope summary. They must be fully refined before being moved from `Backlog` to `Ready`.

Prioritisation Order

The backlog must be prioritised in the following order:

1. Project blockers and unresolved stakeholder decisions.
2. Sprint 1 setup and methodology evidence.
3. The event-driven Basic vertical slice.
4. Remaining Basic requirements.
5. Intermediate batch, review, aggregation, performance and API-consumer requirements.
6. Advanced statistic-definition, live-feed, temporal and reconciliation requirements.
7. Final hardening, documentation, demonstration and submission work.

Security, testing, documentation and accessibility must be included throughout the project rather than left entirely until the final milestone.

Acceptance Criteria

- [] All existing Gitea issues have been reviewed before new issues are created.
- [] Existing issues covering catalogue work have been mapped rather than recreated.
- [] Duplicate or substantially overlapping issues have been removed, combined or clearly distinguished.
- [] All requirements applying to every COMS3011A project are represented in the backlog.
- [] All Sport Analytics Basic requirements are represented in the backlog.
- [] All Sport Analytics Intermediate requirements are represented in the backlog.
- [] All Sport Analytics Advanced requirements are represented in the backlog.
- [] Testing, documentation, accessibility, security, deployment and stakeholder-interaction work are represented.
- [] Final submission, reporting and presentation work are represented.
- [] Every issue has an appropriate title, template, labels and milestone.
- [] Newly created issues initially appear in the Backlog board column.
- [] Urgent Sprint 1 issues have complete descriptions and acceptance criteria before implementation begins.
- [] Later issues have enough information to identify their purpose and requirement coverage.
- [] High-priority and high-risk issues are clearly labelled.
- [] Major dependencies and blockers are recorded.
- [] The backlog supports a Basic vertical slice during Sprint 1.
- [] The backlog targets Basic and Intermediate completion before Advanced work is considered complete.
- [] Advanced architectural risks are scheduled early enough that they are not

all deferred until Sprint 3.

- [] A backlog or requirements-traceability document is added to the repository or documentation site.
- [] The backlog has been reviewed by the group.
- [] Relevant stakeholder questions are recorded for the next stakeholder meeting.
- [] Changes resulting from stakeholder feedback are reflected in the relevant issues.
- [] The work is reviewed and merged through a Pull Request where repository documentation is changed.

Suggested Repository Deliverable

Create or update:

```
```text
docs/planning/project-backlog.md
```

The document should contain:

- the issue number;
- issue title;
- requirement tier;
- milestone;
- priority;
- current status;
- dependencies;
- the requirement or rubric criterion covered.

This document must reference Gitea as the authoritative live work tracker. It should not become a second independent backlog that is allowed to contradict Gitea.

## Dependencies

This issue depends on access to:

- the COMS3011A Project Brief;
- the Sport Analytics project-specific brief;

- the COMS3011A AI Policy;
- the approved Project Methodology;
- the approved Git Methodology;
- the existing Gitea issue list;
- the Gitea Project board and milestones.

There are no implementation dependencies.

## Testing and Verification

The backlog must be verified by:

1. Comparing all created or mapped issues against the project-wide requirements.
2. Comparing all created or mapped issues against the Sport Analytics Basic requirements.
3. Comparing all created or mapped issues against the Sport Analytics Intermediate requirements.
4. Comparing all created or mapped issues against the Sport Analytics Advanced requirements.
5. Checking that no requirement is represented only by an untracked conversation or informal note.
6. Checking that issues can be filtered by milestone, type, area and requirement tier.
7. Checking that urgent Sprint 1 issues have sufficient acceptance criteria to move to Ready.
8. Checking that existing completed work points to its original issue rather than a duplicate retrospective issue.

## Documentation and Evidence

Evidence for this issue must include:

- links to the created and mapped Gitea issues;
- the populated Gitea Project board;
- the completed backlog or requirements-traceability document;
- labels and milestones visible on the issues;

- a record of the team backlog review;
- stakeholder questions or decisions arising from backlog refinement;
- the linked Pull Request for any repository documentation changes.

## Definition of Done

This issue is complete when:

- the complete initial project backlog exists;
- existing issues have been mapped;
- missing issues have been created;
- all major project requirements are traceable to issues;
- the urgent Sprint 1 backlog is fully refined;
- later work is assigned to an appropriate milestone;
- the group has reviewed the backlog;
- the repository backlog or traceability document has been merged;
- the Gitea board accurately reflects the agreed priorities.

Completing this issue does not mean that the individual backlog issues have been implemented.

## Git and Pull Request Requirements

After the issue number is assigned:

1. Create a branch using:

```
docs/<issue-number>-project-backlog
```

2. Use a commit such as:

```
docs(project): add complete project backlog plan
```

3. Reference the issue in the commit body:

```
Refs #<issue-number>
```

4. Open a Pull Request linked to this issue.
5. Include the following in the Pull Request:
  - purpose;
  - backlog summary;
  - requirement coverage;
  - verification performed;
  - known gaps or stakeholder questions;
  - documentation changes.
6. Close the issue through the Pull Request using:

```
Closes #<issue-number>
```

## AI Declaration

*The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].*

The AI declaration identifies the tool and model as required by the course policy. Your group remains responsible for reviewing and verifying the issue before using it. `ai\_policy\_20\_jul.pdf`

8/6/2026, 2:01:26 PM



give me markdown description again.. your markdown got cut off halfway

8/6/2026, 2:05:24 PM



Copy everything inside this box into the **Gitea issue description**:

## ## Description

Create, organise and prioritise the complete Gitea backlog for the Sport Analytics Tool project.

The backlog must cover:

- the requirements that apply to every COMS3011A project;
- the Sport Analytics Basic requirements;
- the Sport Analytics Intermediate requirements;
- the Sport Analytics Advanced requirements;
- testing, documentation, stakeholder interaction and project methodology work;
- deployment, security, accessibility and performance work;
- final submission, reporting and presentation work.

Existing Gitea issues must be reviewed before new issues are created. Work that is already represented by an existing issue must be mapped to that issue rather than duplicated.

The backlog will be created progressively. The urgent Sprint 1 issues must be created and refined first, followed by the remaining Sprint 1, Sprint 2, Sprint 3 and Final Submission issues.

## ## Motivation

The project requires a clear and actively maintained work tracker.

A complete backlog will allow the team to:

- understand the full project scope;
- identify the work needed for Basic, Intermediate and Advanced achievement;

- prioritise the most important and highest-risk work;
- identify dependencies and blockers;
- distribute work between team members;
- avoid forgotten requirements;
- demonstrate consistent use of the agreed project methodology;
- maintain traceability between requirements, issues, branches, commits and Pull Requests;
- provide evidence of planning for the Sprint 1 milestone.

The backlog is a planning tool and must remain current as requirements, stakeholder feedback and technical decisions develop.

## ## Scope

### ### Included

- Review all current Gitea issues.
- Identify which existing issues already cover project requirements.
- Map existing issues to the relevant project requirements.
- Avoid creating duplicate issues.
- Create missing issues for the complete project lifecycle.
- Include project-wide, Basic, Intermediate and Advanced requirements.
- Create separate issues for meaningful units of work.
- Assign the correct issue template to each issue.
- Add applicable type, area, tier and priority labels.
- Assign each issue to the appropriate milestone.
- Place newly created issues in the Backlog board column.
- Identify dependencies between major issues.
- Identify blockers and high-risk work.
- Prioritise urgent Sprint 1 foundation and vertical-slice work.
- Record stakeholder questions where a requirement needs clarification.
- Create or update a repository backlog and requirements-traceability document.
- Review the initial backlog with the team.
- Update the backlog after relevant stakeholder feedback.

### ### Excluded

- Implementing the features represented by the backlog issues.
- Completing all Basic, Intermediate or Advanced functionality.
- Assigning every future issue to a team member before it is ready.
- Moving every issue into Sprint 1.
- Creating duplicate issues for work already represented in Gitea.

- Fully refining every distant Sprint 2 and Sprint 3 issue before its requirements are sufficiently understood.
- Treating the initial backlog as permanently fixed.

## ## Backlog Structure

Each issue must contain, at minimum:

- a clear title;
- an appropriate issue template;
- applicable labels;
- an appropriate milestone;
- an initial board position.

Issues being prepared for immediate implementation must also contain:

- a clear description;
- motivation;
- included and excluded scope;
- acceptance criteria;
- dependencies;
- testing or verification requirements;
- documentation and evidence requirements;
- an assignee;
- a suggested branch name.

Later-sprint issues may initially contain a concise scope summary. They must be fully refined before being moved from `Backlog` to `Ready`.

## ## Prioritisation Order

The backlog must be prioritised in the following order:

1. Project blockers and unresolved stakeholder decisions.
2. Sprint 1 setup and methodology evidence.
3. The event-driven Basic vertical slice.
4. Remaining Basic requirements.
5. Intermediate batch, review, aggregation, performance and API-consumer requirements.
6. Advanced statistic-definition, live-feed, temporal and reconciliation requirements.
7. Final hardening, documentation, demonstration and submission work.

Security, testing, documentation and accessibility must be included throughout the project rather than left entirely until the final milestone.

## ## Acceptance Criteria

- [ ] All existing Gitea issues have been reviewed before new issues are created.
- [ ] Existing issues covering catalogue work have been mapped rather than recreated.
- [ ] Duplicate or substantially overlapping issues have been removed, combined or clearly distinguished.
- [ ] All requirements applying to every COMS3011A project are represented in the backlog.
- [ ] All Sport Analytics Basic requirements are represented in the backlog.
- [ ] All Sport Analytics Intermediate requirements are represented in the backlog.
- [ ] All Sport Analytics Advanced requirements are represented in the backlog.
- [ ] Testing, documentation, accessibility, security, deployment and stakeholder-interaction work are represented.
- [ ] Final submission, reporting and presentation work are represented.
- [ ] Every issue has an appropriate title, template, labels and milestone.
- [ ] Newly created issues initially appear in the Backlog board column.
- [ ] Urgent Sprint 1 issues have complete descriptions and acceptance criteria before implementation begins.
- [ ] Later issues have enough information to identify their purpose and requirement coverage.
- [ ] High-priority and high-risk issues are clearly labelled.
- [ ] Major dependencies and blockers are recorded.
- [ ] The backlog supports a Basic vertical slice during Sprint 1.
- [ ] The backlog targets Basic and Intermediate completion before Advanced work is considered complete.
- [ ] Advanced architectural risks are scheduled early enough that they are not all deferred until Sprint 3.
- [ ] A backlog or requirements-traceability document is added to the repository or documentation site.
- [ ] The backlog has been reviewed by the group.
- [ ] Relevant stakeholder questions are recorded for the next stakeholder meeting.
- [ ] Changes resulting from stakeholder feedback are reflected in the relevant issues.
- [ ] The work is reviewed and merged through a Pull Request where repository



documentation is changed.

## ## Suggested Repository Deliverable

Create or update:

```
```text
docs/planning/project-backlog.md
```
```

The document should contain:

- the issue number;
- issue title;
- requirement tier;
- milestone;
- priority;
- current status;
- dependencies;
- the requirement or rubric criterion covered.

This document must reference Gitea as the authoritative live work tracker. It should not become a second independent backlog that is allowed to contradict Gitea.

## ## Dependencies

This issue depends on access to:

- the COMS3011A Project Brief;
- the Sport Analytics project-specific brief;
- the COMS3011A AI Policy;
- the approved Project Methodology;
- the approved Git Methodology;
- the existing Gitea issue list;
- the Gitea Project board and milestones.

There are no implementation dependencies.

## ## Testing and Verification

The backlog must be verified by:

1. Comparing all created or mapped issues against the project-wide requirements.
2. Comparing all created or mapped issues against the Sport Analytics Basic requirements.
3. Comparing all created or mapped issues against the Sport Analytics Intermediate requirements.
4. Comparing all created or mapped issues against the Sport Analytics Advanced requirements.
5. Checking that no requirement is represented only by an untracked conversation or informal note.
6. Checking that issues can be filtered by milestone, type, area and requirement tier.
7. Checking that urgent Sprint 1 issues have sufficient acceptance criteria to move to `Ready`.
8. Checking that existing completed work points to its original issue rather than a duplicate retrospective issue.

## ## Documentation and Evidence

Evidence for this issue must include:

- links to the created and mapped Gitea issues;
- the populated Gitea Project board;
- the completed backlog or requirements-traceability document;
- labels and milestones visible on the issues;
- a record of the team backlog review;
- stakeholder questions or decisions arising from backlog refinement;
- the linked Pull Request for any repository documentation changes.

## ## Definition of Done

This issue is complete when:

- the complete initial project backlog exists;
- existing issues have been mapped;
- missing issues have been created;
- all major project requirements are traceable to issues;
- the urgent Sprint 1 backlog is fully refined;
- later work is assigned to an appropriate milestone;
- the group has reviewed the backlog;
- the repository backlog or traceability document has been merged;

- the Gitea board accurately reflects the agreed priorities.

Completing this issue does not mean that the individual backlog issues have been implemented.

## ## Git and Pull Request Requirements

After the issue number is assigned:

1. Create a branch using:

```
```text
docs/<issue-number>-project-backlog
```
```

2. Use a commit such as:

```
```text
docs(project): add complete project backlog plan
```
```

3. Reference the issue in the commit body:

```
```text
Refs #<issue-number>
```
```

4. Open a Pull Request linked to this issue.

5. Include the following in the Pull Request:

- purpose;
- backlog summary;
- requirement coverage;
- verification performed;
- known gaps or stakeholder questions;
- documentation changes.

6. Close the issue through the Pull Request using:

```
```text
Closes #<issue-number>
```
```

...

## ## AI Declaration

> *The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].*

8/6/2026, 2:05:29 PM



k, ive made that issue, whats the next step??

8/6/2026, 2:07:35 PM



## Thinking

- **Planning the next issue**

I'm keeping the backlog issue open as the planning container. Next, I'm creating the first concrete task: confirm the MVP sport, competition, and schedule scope.

- **Planned the next issue**

✓ Worked for 18s

Good. **Do not close the backlog issue yet.** It stays open while you create and organise the project backlog.

## Do this now

1. Confirm it has:

- Milestone: `Sprint 1`
- Labels: `type: documentation`, `area: documentation`, `tier: common`, `priority: high`
- An assignee, preferably the project lead or backlog coordinator
- Board position: move it to `Ready`, then `In Progress` once you begin creating the other issues

2. Send me its Gitea issue number, for example:

Backlog issue = #18

3. As we create the remaining issues, add a checklist or comment to the backlog issue:

## ## Backlog Creation Progress

### ### Sprint 1

- [ ] #XX Confirm the sport and competition scope
- [ ] #XX Record stakeholder requirement decisions
- [ ] #XX Produce the system architecture and roadmap
- [ ] #XX Define the API contracts

That makes the issue act as the tracker for the backlog-building work.

4. Do **not** create a branch yet unless someone is also creating:

```
docs/planning/project-backlog.md
```

Creating issues directly in Gitea does not itself require a Git branch or Pull Request.

## Next issue

The next issue should be:

```
Confirm the sport, professional competition scope, event vocabulary and
required statistics
```

This comes next because your event schema, database, API and calculated statistics all depend on the chosen sport. `sport_analytics.pdf`

Send me the backlog issue number, and I'll give you the full copy-and-paste markdown for this next issue.

*8/6/2026, 2:07:45 PM*



332cb8d8-631f-4494-9da6-f89...

8/6/2026, 2:08:56 PM



Perfect — issue **#36** is set up correctly.

Just add the two missing labels:

```
tier: common
priority: high
```

Everything else shown is right:

- **Milestone:** Sprint 1
- **Project:** Sport Analytics project board
- **Assignee:** Shayna
- **Status:** Open
- **No Branch/Tag Specified:** fine for now

On the **Projects** board, move issue **#36** to:

In Progress

because you are actively creating and prioritising the backlog. Keep it open until the urgent issues are created, duplicates are checked, requirements are mapped, and the team reviews the backlog.

You do **not** need a branch yet unless you are also creating the repository file:

```
docs/planning/project-backlog.md
```

Record this mapping:

```
Backlog planning = #36
```

The next issue is:

Confirm the sport, professional competition scope, event vocabulary and required statistics

*8/6/2026, 2:09:09 PM*





ok, lets do that one... idk what mapping you want me to do?? we will have to do that next i guess? or must we do it as we go?

8/6/2026, 2:11:04 PM



## Thinking

- **Mapping the backlog issue**

I'm treating issue #36 as a research-and-definition task. I'll create copy-paste Markdown covering scope, decision criteria, terminology, data needs, acceptance criteria, dependencies, and evidence, while keeping implementation out of scope.

- **Mapped the backlog issue**

- **Built the next sport analytics issue and defined event mapping validation**

✓ Worked for 25s

You do **not** need to complete all the mapping now.

The easiest process is:

- As we create each new issue, record its number.
- When you recognise that an existing issue already covers something, tell me that number instead of recreating it.
- After the urgent issues are created, we will do one proper mapping pass through your older issues.

For now, the only mapping is:

Backlog planning = #36

The next issue is important because the chosen sport determines what counts as a fixture, which events must be recorded and which statistics can be derived.

sport\_analytics.pdf

# Issue details

## Title

Confirm the sport, professional competition scope, event vocabulary and required statistics

**Template:** Task

## Labels

type: research  
area: data  
area: documentation  
tier: basic  
priority: high

**Milestone:** Sprint 1

**Initial board column:** Backlog

**Suggested assignee:** Person leading requirements, data modelling or stakeholder communication

Once the issue is fully reviewed and assigned, it can move to **Ready**. Your methodology says a research issue may use a documented recommendation or decision as its output. `Project_Methodology_v1.1.md`

## Copy-and-paste description

### `## Description`

Select and formally define the professional sport, league or tour that the Sport Analytics Tool will support.

The team must define the sport-specific domain clearly enough for the database, event schema, statistic derivation engine, API and user interface to be designed consistently.

The outcome of this issue must identify:

- the selected sport;
- the professional league, competition or tour represented by the platform;

- what constitutes a fixture;
- whether competitors are teams, individual athletes or both;
- the important entities involved in the sport;
- the events that may occur during a fixture;
- the order and timing information required for those events;
- the fixture, competitor and participant statistics that must be calculated;
- which statistics are directly derived from recorded events;
- who is expected to submit fixture data;
- how submitters may be restricted to a defined part of the competition;
- any sport-specific rules that affect validation or correction.

This issue is a domain-definition and requirements issue. It does not implement the database, API, event schema or statistic calculations.

## ## Motivation

The Sport Analytics Tool must be built around ordered event data rather than manually entered totals.

The selected sport determines:

- the structure of a fixture;
- the competitors and participants that must be stored;
- the event vocabulary;
- the fields required for each event;
- the validation rules;
- the statistics that can be derived;
- the type and scale of the test data;
- the API resources;
- the frontend pages and visualisations;
- the type of professional competition the platform serves.

Without a confirmed sport and domain scope, different members of the team may design incompatible database tables, API endpoints, frontend pages and statistic calculations.

This decision must therefore be completed before detailed implementation of the event model and derivation engine begins.

## ## Scope

### ### Included

- Identify a shortlist of suitable sports where necessary.
- Compare the shortlisted sports using clear selection criteria.
- Select one professional sport.
- Select or define the professional league, competition or tour that the system represents.
- Define what a fixture means for the selected sport.
- Define the competitors and participants involved.
- Define a controlled vocabulary of fixture events.
- Define the minimum data required for each event type.
- Define how events are ordered within a fixture.
- Define the important fixture-level statistics.
- Define the important competitor and participant statistics.
- Identify how each required statistic is derived from event data.
- Identify statistics that cannot reasonably be supported by the proposed event data.
- Define the initial approved-submitter scope model.
- Identify sport-specific impossible or conflicting event conditions.
- Identify representative fixture data needed for development and testing.
- Record stakeholder questions and decisions.
- Store the final decision in the repository documentation.
- Review the decision with the team and stakeholder.

### ### Excluded

- Implementing the event schema.
- Creating database migrations.
- Implementing API endpoints.
- Building frontend pages.
- Implementing statistic calculations.
- Importing a complete historical dataset.
- Building batch processing.
- Implementing live-event ingestion.
- Implementing custom analyst-defined statistics.
- Supporting multiple unrelated sports.
- Attempting to model every possible rule or statistic before development begins.

### ## Sport Selection Criteria

Where the sport has not already been selected, the team must compare suitable options using criteria such as:

- events can be represented individually and chronologically;
- meaningful statistics can be derived from those events;
- the fixture structure is clearly understood;
- sufficient domain information is available;
- representative test data can be created or obtained legally;
- the event volume is suitable for Intermediate performance testing;
- corrections can be demonstrated clearly;
- season and career aggregates are meaningful;
- live and out-of-order events can eventually be demonstrated;
- the team can explain the sport confidently;
- the scope is achievable within the project timeline.

The final choice must include a short written motivation.

## ## Required Domain Decisions

The final domain definition must answer the following questions.

### ### Competition

- What sport is supported?
- What professional league, competition or tour is represented?
- Is the system intended for one competition or multiple competitions of the same sport?
- What is a season or competition period?
- What is a fixture?

### ### Competitors and Participants

- Are competitors teams, individual athletes or both?
- Which participant roles must be represented?
- Can a participant change teams or competitors between seasons?
- Which identifiers need to remain stable across seasons?

### ### Event Vocabulary

For each event type, define:

- event name;
- plain-language meaning;
- required fields;

- optional fields;
- involved competitor;
- involved participant or participants;
- fixture time or sequence information;
- valid values;
- relevant sport-specific restrictions;
- statistics affected by the event.

### ### Statistics

For each required statistic, define:

- statistic name;
- level of calculation, such as fixture, participant, team, season or career;
- events used in the calculation;
- calculation rule;
- treatment of corrections;
- treatment of missing or invalid data;
- whether the statistic is required for Basic, Intermediate or Advanced work.

### ## Suggested Initial Deliverable

Create:

```
```text
docs/requirements/sport-domain-definition.md
```
```

The document should contain:

1. Selected sport and professional competition.
2. Motivation for the selection.
3. Competition and fixture definitions.
4. Competitor and participant definitions.
5. Event vocabulary table.
6. Required-event field table.
7. Statistic catalogue.
8. Event-to-statistic mapping.
9. Submitter scope proposal.
10. Sport-specific validation rules.
11. Representative fixture example.
12. Open stakeholder questions.

### 13. Final decisions and approval record.

#### ## Suggested Event Vocabulary Table

| Event Type    | Meaning     | Required Data   | Optional Data   | Statistics Affected |
|---------------|-------------|-----------------|-----------------|---------------------|
| Example event | Explanation | Required fields | Optional fields | Derived statistics  |

#### ## Suggested Statistic Catalogue

| Statistic         | Level                                  | Source Events   | Calculation      | Requirement Tier                |
|-------------------|----------------------------------------|-----------------|------------------|---------------------------------|
| Example statistic | Fixture, participant, season or career | Relevant events | Calculation rule | Basic, Intermediate or Advanced |

#### ## Acceptance Criteria

- [ ] One professional sport has been selected.
- [ ] The selected professional league, competition or tour has been identified or clearly defined.
- [ ] The choice includes a written motivation.
- [ ] A fixture has been defined precisely for the selected sport.
- [ ] Competitors and participants have been identified.
- [ ] The important domain entities have been listed.
- [ ] A controlled initial event vocabulary has been created.
- [ ] Every event type has a clear meaning.
- [ ] Every event type identifies its required data.
- [ ] Event ordering and timing requirements have been defined.
- [ ] The required Basic fixture statistics have been listed.
- [ ] The required Basic competitor or participant statistics have been listed.
- [ ] Each Basic statistic is mapped to the events used to derive it.
- [ ] Intermediate season, career and competition aggregates have been identified.
- [ ] Advanced custom-statistic and live-feed considerations have been recorded without over-designing them.
- [ ] Statistics unsupported by the proposed event data are identified as exclusions or future decisions.
- [ ] A proposed submitter scope has been defined.
- [ ] Initial sport-specific impossible or conflicting conditions have been identified.
- [ ] A representative fixture example has been documented.

- [ ] Open questions for the stakeholder have been recorded.
- [ ] The domain-definition document has been added to the repository.
- [ ] The team has reviewed the proposed domain.
- [ ] The stakeholder has reviewed the decision or unresolved questions have been clearly recorded.
- [ ] Any changes resulting from stakeholder feedback have been documented.
- [ ] The documentation change has been reviewed and merged through a Pull Request.

## ## Dependencies

Related issue:

- #36 Create and prioritise the complete project backlog

This issue should be completed before:

- the final event schema is implemented;
- the main sport database schema is finalised;
- the statistic derivation engine is implemented;
- sport-specific frontend pages are designed in detail;
- representative performance data is generated.

## ## Testing and Verification

Verify the domain definition by selecting one representative fixture and confirming that:

1. Its competitors and participants can be represented.
2. Its important events can be recorded in order.
3. Each event can be represented using the proposed data fields.
4. The required fixture statistics can be calculated from those events.
5. At least one event correction would automatically affect a derived statistic.
6. Each calculated statistic can be traced back to its contributing events.
7. The same domain model could support multiple fixtures in a season.
8. The model does not depend on manually typing final totals.

The team must also review whether the selected sport can reasonably support:

- whole-season batch submissions;
- season and career aggregates;



- representative-scale performance testing;
- live fixture feeds;
- late or out-of-order events;
- custom statistic definitions.

## ## Documentation and Evidence

Evidence must include:

- the completed sport-domain definition;
- the comparison or motivation used to select the sport;
- the event vocabulary;
- the statistic catalogue;
- the event-to-statistic mapping;
- the representative fixture walkthrough;
- stakeholder questions and responses;
- team review evidence;
- the linked Pull Request.

## ## Definition of Done

This issue is complete when:

- the sport and professional competition scope are confirmed;
- the fixture, competitor and participant definitions are clear;
- the initial event vocabulary is agreed;
- required statistics are mapped to event data;
- the representative fixture walkthrough succeeds;
- stakeholder decisions and open questions are recorded;
- the domain-definition document is reviewed and merged;
- dependent database, API and event-schema work can proceed without inventing incompatible domain assumptions.

## ## Git and Pull Request Requirements

After the issue number is assigned, create a branch using:

```
```text
docs/<issue-number>-sport-domain-definition
```
```

Example:

```
```text
docs/37-sport-domain-definition
```
```

Use a commit such as:

```
```text
docs(requirements): define sport analytics domain
```
```

Reference the issue in the commit body:

```
```text
Refs #<issue-number>
```
```

Open a Pull Request that contains:

- the selected sport and competition;
- the selection motivation;
- the event and statistic scope;
- the representative fixture verification;
- stakeholder decisions;
- open questions;
- known limitations.

Close the issue through the Pull Request using:

```
```text
Closes #<issue-number>
```
```

## ## AI Declaration

> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Create it, add it to the project in **Backlog**, and send me its issue number. We will then record:

Sport and domain definition = #XX

That is all “mapping” needs to mean for now.

*8/6/2026, 2:11:26 PM*



ok, next

8/6/2026, 2:15:57 PM



## Thinking

- **Outlining decision records**

I'm preparing the next issue after the sport-domain requirements: a traceable decision record covering requirement tiers, stakeholder input, evidence, unresolved conflicts, and later validation. I'll format it for direct issue creation.

- **Outlined decision records**

✓ Worked for 16s

The next issue is the **stakeholder requirements and decision record**. It is important because your tutor acts as the project's client, and the brief expects evidence that requirements were discussed and stakeholder feedback influenced the project. `project_briefs.pdf`

## Issue details

### Title

Record stakeholder decisions for the Basic, Intermediate and Advanced requirements

**Template:** Task

### Labels

type: research  
area: documentation  
tier: common  
priority: high

**Milestone:** Sprint 1

**Initial board column:** Backlog

**Suggested assignee:** Project lead, Scrum facilitator or stakeholder liaison

Move it to **Ready** once the issue has been reviewed and someone has taken responsibility for the next stakeholder discussion.

## Copy-and-paste description

### ## Description

Create and maintain a formal record of stakeholder questions, discussions, decisions and requirement clarifications for the Sport Analytics Tool project.

The project brief defines Basic, Intermediate and Advanced requirements, but several details depend on the selected sport, the intended users and the stakeholder's interpretation of what constitutes a complete feature.

The team must identify unclear or high-risk requirements, discuss them with the assigned tutor acting as the project client, record the responses and update the relevant backlog issues and project documentation.

This issue establishes an ongoing stakeholder decision process. It is not limited to one meeting and should remain open until the initial high-priority requirement decisions have been recorded and incorporated into the Sprint 1 plan.

### ## Motivation

The project is evaluated not only on implemented features but also on:

- stakeholder interaction;
- requirement identification;
- evidence of discussion;
- evaluation of stakeholder feedback;
- integration of relevant feedback into the product;
- consistent use of the chosen project methodology.

The project brief gives the tutor the role of client and first point of contact for project questions.

Several requirements may be interpreted differently depending on the selected sport and architecture. Examples include:

- what counts as a professional league or tour;
- the minimum event vocabulary;
- which statistics are considered important;
- what counts as an approved submitter;
- how submitter scopes should work;
- what evidence is needed to prove traceability;
- acceptable approaches to batch processing;
- expected fixture and event volumes;
- acceptable response-time targets;
- the permitted design of custom statistic definitions;
- acceptable live-feed simulation;
- what is required for Advanced market-ready quality.

Unrecorded verbal decisions can easily be forgotten or interpreted differently by different team members. A formal decision record will provide one shared source of truth.

## ## Scope

### ### Included

- Review the project-wide requirements.
- Review the Sport Analytics Basic requirements.
- Review the Sport Analytics Intermediate requirements.
- Review the Sport Analytics Advanced requirements.
- Identify requirements that are unclear, risky or open to interpretation.
- Create a prioritised stakeholder-question register.
- Discuss relevant questions with the assigned tutor or lecturer.
- Record the date, participants, question, response and resulting decision.
- Record any conditions, limitations or follow-up actions.
- Link decisions to the affected Gitea issues.
- Update acceptance criteria when stakeholder decisions change the meaning of a feature.
- Update architecture, domain and planning documentation where necessary.
- Record decisions made during stand-ups, reviews, demonstrations and stakeholder meetings.
- Record when a decision is provisional and still requires lecturer confirmation.
- Maintain a requirement change log.
- Communicate important decisions to the full group.

### ### Excluded

- Implementing the features discussed with the stakeholder.
- Treating informal assumptions as approved decisions.
- Asking the stakeholder to design the entire technical solution.
- Replacing the project brief with stakeholder preferences.
- Ignoring written project requirements because of an undocumented verbal comment.
- Making retrospective records that falsely imply a meeting took place.
- Closing every project ambiguity during a single meeting.
- Recording routine implementation details that do not affect requirements or acceptance.

## ## Stakeholder Roles

### ### Assigned Tutor or Client

The assigned tutor acts as the project client and should be the first point of contact for:

- requirement interpretation;
- sport-specific scope;
- feature completeness;
- stakeholder expectations;
- feedback on proposed workflows;
- review of demonstrations and prototypes.

### ### Lecturer

The lecturer should be asked where:

- the project-wide requirements are unclear;
- a proposed technology may conflict with the brief;
- the tutor cannot provide a final ruling;
- the AI policy requires clarification;
- the proposed solution may rely on prohibited automatic API generation;
- frontend and backend separation is uncertain;
- a requirement needs an official course-level interpretation.

### ### Team

The team remains responsible for:

- preparing useful questions;
- presenting possible options;
- recording decisions accurately;
- evaluating feedback rather than accepting it without thought;
- implementing agreed changes;
- maintaining the decision record.

## ## Initial Stakeholder Questions

The team must review and adapt the following questions.

### ### Sport and Competition Scope

- Is the selected sport and professional competition appropriate?
- May the platform support a modelled professional competition rather than importing official league data?
- Is supporting one professional competition sufficient?
- How much real historical data is expected?
- May representative or synthetic fixture data be used for development and demonstration?

### ### Event Data and Statistics

- What is the expected minimum number of event types?
- Which statistics must be included for the selected sport?
- Must every published statistic be derived entirely from recorded events?
- May fixture metadata such as venue and scheduled time be entered directly?
- What level of event correction history is expected for Basic?
- What evidence will demonstrate that statistics are traceable to events and submissions?

### ### Submitters and Review

- What qualifies a user as an approved submitter?
- Must administrators approve submitters manually?
- What is an acceptable defined competition scope?
- May scope be defined by competition, team, venue or fixture?
- Must Basic submissions be reviewed before publication, or is review only an Intermediate requirement?

### ### API and Datasets



- Which filters are essential for the Basic API?
- Is offset pagination acceptable, or is cursor pagination expected?
- Which export formats are required?
- What level of public API documentation is expected?
- Does the API need anonymous public access, authenticated access or both?
- What qualifies as a relevant external API integration?

### ### Intermediate Requirements

- What size should the representative load test contain?
- Must “hundreds of fixtures with hundreds of events” be literal test data?
- What response-time target would be considered reasonable?
- What behaviour must be resumable after a failed batch?
- What counts as sufficient idempotency evidence?
- What reference results should be used to verify derived statistics?
- Is a scheduled background worker sufficient for batch processing?
- What level of API rate limiting and quota management is expected?
- What information must appear in a versioned dataset release?

### ### Advanced Requirements

- What format may be used for analyst-defined statistics?
- Is a constrained expression language acceptable?
- Is simulated live event ingestion acceptable?
- What transport is expected for the live feed?
- How should late or out-of-order events be demonstrated?
- What degree of temporal history is required for as-of-date queries?
- What constitutes an acceptable dataset release comparison?
- What size qualifies as a large asynchronous API request?
- Is a polling-based change feed acceptable?
- What evidence is expected for API deprecation and contract testing?
- What level of anomaly detection is considered sufficient?
- How should conflicting submitters be reconciled?
- What does “market-ready” mean for the Advanced standard?

### ### Project-Wide Requirements

- Is the proposed frontend and backend separation acceptable?
- Is the proposed authentication provider acceptable?
- Is the planned external API relevant enough?
- Is the documentation hosting approach acceptable?
- Is the planned CI/CD process sufficient?

- Are the selected accessibility standards appropriate?
- Is the proposed deployment architecture acceptable?

## ## Decision Statuses

Each question or decision must have one of the following statuses:

- `Open`
- `Proposed`
- `Confirmed by Tutor`
- `Escalated to Lecturer`
- `Confirmed by Lecturer`
- `Superseded`
- `Rejected`
- `Deferred`

A decision must not be marked as confirmed unless the relevant stakeholder actually approved it.

## ## Suggested Repository Deliverables

Create:

```
```text
docs/requirements/stakeholder-questions.md
```
```

and:

```
```text
docs/requirements/decision-log.md
```
```

The stakeholder-question register should include:

| ID  | Requirement | Question | Priority | Raised By | Status | Target Meeting |
|-----|-------------|----------|----------|-----------|--------|----------------|
| --- | ---         | ---      | ---      | ---       | ---    | ---            |

The decision log should include:

| Decision ID | Date | Participants | Requirement | Decision | Reason |
|-------------|------|--------------|-------------|----------|--------|
|-------------|------|--------------|-------------|----------|--------|

Affected Issues | Status |  
|---|---|---|---|---|---|---|---|

A decision entry should contain enough context that a team member who did not attend the meeting can understand:

- what was asked;
- why it mattered;
- what alternatives were considered;
- what the stakeholder decided;
- which issues or documents must change.

## ## Requirement Change Process

When stakeholder feedback changes a requirement:

1. Record the original requirement or assumption.
2. Record the stakeholder's response.
3. Explain the resulting interpretation or change.
4. Identify the affected issues.
5. Update the relevant acceptance criteria.
6. Update architecture or domain documentation where necessary.
7. Re-evaluate milestone, priority and dependencies.
8. Inform the group.
9. Record the change in the decision log.
10. Do not silently rewrite history or delete the original decision.

## ## Acceptance Criteria

- [ ] All project-wide requirements have been reviewed for ambiguity.
- [ ] All Sport Analytics Basic requirements have been reviewed for ambiguity.
- [ ] All Sport Analytics Intermediate requirements have been reviewed for ambiguity.
- [ ] All Sport Analytics Advanced requirements have been reviewed for ambiguity.
- [ ] A stakeholder-question register has been created.
- [ ] Questions are linked to the relevant requirements.
- [ ] Questions are prioritised by urgency and project risk.
- [ ] Questions affecting the Sprint 1 vertical slice are clearly identified.
- [ ] Sport and competition scope questions have been recorded.
- [ ] Event vocabulary and required-statistic questions have been recorded.
- [ ] Submitter approval and scope questions have been recorded.

- [ ] API, filtering, pagination and export questions have been recorded.
- [ ] Intermediate batch and performance questions have been recorded.
- [ ] Advanced custom-statistic and live-feed questions have been recorded.
- [ ] Project-wide architecture and technology questions have been recorded.
- [ ] At least one formal stakeholder discussion has taken place.
- [ ] The date and participants of each formal discussion are recorded.
- [ ] Stakeholder responses are recorded accurately.
- [ ] Decisions distinguish between tutor confirmation and lecturer confirmation.
- [ ] Provisional decisions are clearly marked.
- [ ] A decision log has been added to the repository.
- [ ] Each confirmed decision identifies the affected Gitea issues.
- [ ] Affected issue descriptions or acceptance criteria have been updated.
- [ ] Significant changes have been communicated to the group.
- [ ] Superseded decisions remain visible in the history.
- [ ] Relevant evidence is linked from the issue.
- [ ] The documentation has been reviewed and merged through a Pull Request.

## ## Dependencies

Related issues:

- #36 Create and prioritise the complete project backlog
- The issue for confirming the sport, professional competition scope, event vocabulary and required statistics

Replace the second item with its actual issue number once known.

This issue should begin immediately and continue alongside:

- domain definition;
- system architecture;
- API contract design;
- database modelling;
- backlog refinement.

Unresolved questions that block implementation must receive:

```
```text
status: blocked
```
```

on the affected issue until a safe decision is made.

## ## Testing and Verification

Verify the stakeholder process by checking that:

1. Each major unclear requirement has a recorded question.
2. Every confirmed answer has a date and identified stakeholder.
3. Each decision links to the issues it affects.
4. Relevant issue acceptance criteria match the recorded decision.
5. The decision log does not contradict the current project brief.
6. Tutor-level and lecturer-level decisions are clearly distinguished.
7. Superseded decisions remain available for traceability.
8. Team members can explain the current agreed interpretation.
9. No major design decision relies only on an unrecorded chat or verbal conversation.
10. Open blocking questions are visible on the Gitea board.

## ## Documentation and Evidence

Evidence must include:

- the stakeholder-question register;
- the stakeholder decision log;
- meeting or stand-up notes;
- dated stakeholder responses;
- issue comments linking decisions to implementation work;
- updated issue acceptance criteria;
- updated architecture or requirements documentation;
- the linked Pull Request;
- evidence that the team reviewed the decisions.

Unedited AI transcripts must be retained where AI was used during formal assessment work, in accordance with the course AI policy.

## ## Definition of Done

This issue is complete for Sprint 1 when:

- the initial question register exists;
- all high-priority Sprint 1 ambiguities have been identified;
- the first stakeholder discussion has taken place;

- available decisions have been recorded;
- blocking questions have been escalated;
- affected Sprint 1 issues have been updated;
- the decision log has been reviewed and merged;
- the group has been informed of the confirmed decisions.

The decision log itself remains a living project document and must continue to be updated after this issue closes.

## ## Git and Pull Request Requirements

After the issue number is assigned, create a branch using:

```
```text
docs/<issue-number>-stakeholder-decisions
```
```

Example:

```
```text
docs/38-stakeholder-decisions
```
```

Use a commit such as:

```
```text
docs(requirements): add stakeholder question and decision logs
```
```

Reference the issue in the commit body:

```
```text
Refs #<issue-number>
```
```

The Pull Request must include:

- the purpose of the registers;
- the initial questions;
- decisions already received;
- outstanding blockers;
- affected Gitea issues;

- verification performed.

Close the issue through the Pull Request using:

```
```text
```

```
Closes #<issue-number>
```

```
```
```

## ## AI Declaration

*> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].*

The requirement decisions must still follow the written project brief. Where the course policy, project-wide brief and project-specific brief conflict or remain unclear, record the question and request clarification rather than silently choosing an interpretation.

ai\_policy\_20\_jul.pdf sport\_analytics.pdf

8/6/2026, 2:16:11 PM



summerize for me b4 i create this what it means? what does it require doing?

8/6/2026, 2:18:18 PM



This issue is basically about **keeping a formal record of what your tutor or lecturer decides about unclear project requirements.**

It does **not** require you to build any features.

To complete it, someone in the group must:

1. Read through the Basic, Intermediate and Advanced requirements.
2. Identify anything that is unclear or could be interpreted in different ways.
3. Write those points as questions.
4. Ask the tutor or lecturer during meetings.
5. Record:
  - what was asked;
  - who answered;
  - the date;
  - what decision was made;
  - which Gitea issues are affected.
6. Update relevant issues if the answer changes their scope or acceptance criteria.
7. Save the questions and decisions in the documentation repository.

For example, you may need to ask:

- Is synthetic sport data acceptable?
- How many event types are expected?
- What size must the Intermediate performance test be?
- Is a simulated live feed acceptable for Advanced?



- What exactly counts as an approved submitter?
- What does “market-ready” mean for Advanced?

The main deliverables are two simple documents:

```
docs/requirements/stakeholder-questions.md
docs/requirements/decision-log.md
```

The first document lists questions still needing answers. The second records confirmed decisions.

In practical terms, this issue gives you evidence that you are regularly communicating with your stakeholder and using their feedback to guide the project, which is part of the marking rubric.

You do not need all the answers immediately. For Sprint 1, you mainly need:

- a list of important questions;
- at least one discussion with the tutor;
- the answers you received recorded;
- affected issues updated.

So the issue is really: **“Track what we need to ask the tutor and record what they tell us.”**

*8/6/2026, 2:18:26 PM*



is that not anywhere in our repo yet?

8/6/2026, 2:19:42 PM



## Thinking



### Inspecting repository contents for decision logs

- **Checking repository coverage**

I'm finding that the repository already defines stakeholder-meeting evidence and methodology, but it doesn't contain populated question registers or decision logs. The issue is still valid, though it should focus on populating and maintaining existing structures rather than inventing a new process.

- **Checked repository coverage**

- **Inspected repository and documentation for stakeholder review evidence**

✓ Worked for 47s

## Partly — the system is already set up in your repo.

I checked the uploaded repository. You already have:

- `docs/project_methodology.md` — explains weekly stakeholder meetings, what must be discussed, and provides a meeting-note template.
- `evidence/stakeholder-meetings/README.md` — tells you to save dated meeting records with requirements, feedback, decisions, actions and linked Gitea issues.
- `evidence/README.md` — identifies the stakeholder-meetings folder as the place for agendas, minutes, feedback and decisions.

So you **should not create another stakeholder-process or meeting-template system.**

What is currently missing is the actual populated evidence:

- dated stakeholder meeting notes;

- a list of unanswered requirement questions;
- confirmed decisions from the tutor or lecturer;
- links from those decisions to affected Gitea issues;
- updates showing how feedback changed the backlog.

Therefore, the issue is still useful, but I would rename it to:

Populate and maintain stakeholder questions and requirement decisions

The work would be:

1. Use the existing meeting template in `docs/project_methodology.md`.
2. Save each genuine meeting as:

`evidence/stakeholder-meetings/YYYY-MM-DD-topic.md`

3. Record questions before the meeting.
4. Record the tutor's actual answers afterward.
5. Link the decisions to relevant Gitea issues.
6. Update those issues where the decision affects their scope.

You do **not necessarily need** two new files called `stakeholder-questions.md` and `decision-log.md`. A central register may help later, but your existing dated meeting-record structure already satisfies much of the purpose.

So: **the process exists; the real stakeholder records and decisions do not yet appear to exist.** We should shorten and correct the issue description before you create it.

*8/6/2026, 2:19:43 PM*



lets skipt this one for now!  
whats next?

8/6/2026, 2:22:45 PM



## Thinking

- **Planning architecture next**

The next issue is the system architecture and development roadmap. I'm outlining the major components, data flow, integrations, deployment path, and implementation phases so development has a shared technical plan.

- **Planned the next architecture steps**

✓ Worked for 12s

The next urgent issue is:

## Produce the complete system architecture, data flow and development roadmap

**Template:** Task

**Labels:**

```
type: documentation
area: api
area: database
area: infrastructure
area: documentation
tier: common
priority: high
```

**Milestone:** Sprint 1

**Board:** Backlog

**What this issue means**

Your group must agree on and document **how the whole system will fit together before everyone starts building separate pieces.**

It should show:

```
Frontend
 ↓ HTTP requests
Handwritten Backend API
 ↓
PostgreSQL Database
```

It should also show where these fit:

- Firebase authentication
- Approved submitter permissions
- Event submission and validation
- Statistic calculation
- File uploads
- Dataset exports
- External API integration
- CI/CD
- Azure deployment
- Cloudflare documentation
- Later batch processing
- Caching and background jobs
- Advanced live event processing

The issue also requires a development roadmap showing what is planned for:

- Sprint 1
- Sprint 2
- Sprint 3
- Final submission

## What you actually need to produce

Probably one main document:

```
docs/architecture/system-architecture.md
```

It should contain:

- A high-level architecture diagram
- A data-flow diagram
- A list of the main system components
- An explanation of what each component does
- The chosen technologies
- How frontend and backend remain separate
- How the handwritten API works
- How authentication and authorisation work
- How events become calculated statistics
- Deployment architecture
- Sprint roadmap
- Important risks and unresolved decisions

This is mainly a **planning and documentation issue**, not a coding issue.

It is especially urgent because several people may soon work on frontend, backend, database and deployment simultaneously. Without a shared architecture, they may build incompatible systems.

The sport-domain issue should ideally be completed or at least mostly agreed before this architecture is finalised. However, both issues can begin at the same time.

I'll give you the complete copy-and-paste description for this architecture issue next.

*8/6/2026, 2:22:56 PM*



give me the desc markdown in 1 thing to copy paste

8/6/2026, 2:24:52 PM



## ## Description

Design and document the complete high-level architecture, data flow and development roadmap for the Sport Analytics Tool.

The architecture must explain how the frontend, handwritten backend API, database, authentication provider, event-processing logic, external services, deployment platforms and documentation website work together.

The document must cover the planned system from the Basic requirements through to the Intermediate and Advanced requirements. It should provide enough direction for different team members to work on the frontend, backend, database, infrastructure and documentation without creating incompatible solutions.

This is primarily a planning and documentation issue. It does not require every architectural component to be implemented before the issue can be completed.

## ## Motivation

The project contains several connected systems:

- a separate frontend application;
- a handwritten backend API;
- a PostgreSQL database;
- Firebase authentication;
- approved submitter roles and scopes;
- event submission and validation;
- event-derived statistics;
- file uploads and dataset exports;
- batch processing;
- API keys, quotas and caching;
- versioned dataset releases;
- custom statistic definitions;

- live event ingestion;
- Azure deployment;
- Cloudflare-hosted documentation;
- CI/CD through Gitea Actions.

Without a shared architecture, team members may make conflicting decisions about:

- database structure;
- API contracts;
- authentication;
- permissions;
- frontend and backend responsibilities;
- event processing;
- deployment;
- background jobs;
- caching;
- file storage;
- testing;
- Advanced feature support.

A documented architecture will provide a shared technical direction and support the Sprint 1 requirement for an initial design and development plan.

## ## Scope

### ### Included

- Document the overall system architecture.
- Define the responsibility of each major system component.
- Show that the frontend and backend are separate applications.
- Show that the frontend communicates with the backend through HTTP API requests.
- Show that the backend owns the handwritten public API.
- Show how the backend communicates with PostgreSQL.
- Show how Firebase authentication integrates with the frontend and backend.
- Define the difference between authentication and application authorisation.
- Show how user roles, submitter approval and competition scopes are enforced.
- Show how event data enters the system.
- Show how submissions are validated.
- Show how ordered events are stored.
- Show how statistics are derived from event data.



- Show how corrections cause affected statistics to be updated.
- Show how statistics remain traceable to events and submissions.
- Show how filtered data is exposed through the API.
- Show how filtered datasets are exported.
- Include the planned external API integration point.
- Include the documentation website and its deployment.
- Include CI/CD and application deployment.
- Include the planned Intermediate batch-processing architecture.
- Include the planned caching, rate-limiting and dataset-release components.
- Include the planned Advanced custom-statistic and live-feed architecture.
- Identify major architectural risks and unresolved decisions.
- Create a development roadmap for Sprint 1, Sprint 2, Sprint 3 and Final Submission.
- Record important architecture decisions and their motivation.
- Review the architecture with the group.

### ### Excluded

- Fully implementing the architecture.
- Writing all API endpoints.
- Creating all database migrations.
- Completing the frontend.
- Completing authentication flows.
- Implementing batch processing.
- Implementing live feeds.
- Implementing custom statistic execution.
- Producing low-level class diagrams for every file.
- Finalising details that depend on unresolved stakeholder decisions.
- Creating a separate microservice for every minor responsibility.
- Changing the agreed technology stack without recording and motivating the decision.

### ## Proposed High-Level Architecture

The planned request flow should follow this general structure:

```
```text
```

```
User Browser
```

```
|
```

```
v
```

```
Frontend Application
```

```
|
```

```
| HTTPS requests
v
Handwritten Backend API
|
+---- Firebase token verification
|
+---- Authorisation and submitter-scope checks
|
+---- Submission and event validation
|
+---- Statistic derivation
|
+---- Export and dataset services
|
v
PostgreSQL Database
...
```

Supporting services may include:

```
```text
Gitea Repository
|
v
Gitea Actions CI/CD
|
+---- Frontend deployment
+---- Backend deployment
+---- Automated tests
+---- Documentation deployment
...
```

Intermediate and Advanced components may include:

```
```text
Batch Upload
|
v
Staging and Validation
|
v
Background Processing
```

```
|
v
Events, Statistics and Dataset Releases
...

```text
Live Event Feed
|
v
Ordered and Replayable Event Pipeline
|
v
Derived Statistics and Change Feed
...

```

These diagrams are starting points and must be adjusted to match the final approved architecture.

## ## Required Architecture Components

The document must explain the purpose and responsibility of the following components.

### ### Frontend Application

The frontend is responsible for:

- public fixture and statistic pages;
- sign-up and sign-in interfaces;
- account management;
- administrator interfaces;
- approved submitter interfaces;
- event-upload and validation-result pages;
- statistic and provenance views;
- dataset export and release interfaces;
- accessible and responsive presentation;
- sending authenticated requests to the backend API.

The frontend must not directly replace the handwritten backend API.

### ### Backend API

The backend is responsible for:

- handwritten HTTP endpoints;
- request validation;
- Firebase token verification;
- role and permission checks;
- approved submitter scope enforcement;
- competition and fixture management;
- event submission;
- event validation;
- correction handling;
- statistic derivation;
- provenance;
- filtering;
- sorting;
- pagination;
- data export;
- external API integration;
- batch processing;
- API keys and quotas;
- asynchronous jobs;
- change feeds;
- custom statistic execution;
- live-feed processing.

### ### PostgreSQL Database

The database is responsible for storing:

- application accounts;
- roles and permissions;
- submitter approvals and scopes;
- competitions;
- seasons;
- competitors;
- participants;
- fixtures;
- submissions;
- ordered events;
- corrections;
- derived statistics or calculation state where appropriate;
- audit history;

- API consumers and keys;
- batch-processing state;
- dataset releases;
- custom statistic definitions;
- temporal history;
- processing jobs.

### ### Authentication

Firebase should provide established authentication functionality such as:

- account creation;
- sign-in;
- sign-out;
- password reset;
- identity tokens;
- supported OAuth or OIDC providers where used.

The backend must verify Firebase identity tokens.

Application roles and permissions must remain under the control of the Sport Analytics application.

### ### Authorisation

The architecture must distinguish between:

```text

Authentication:

Who is the user?

```

and:

```text

Authorisation:

What is the user allowed to do?

```

The authorisation design must support roles such as:

- public user;

- registered user;
- analyst or API consumer;
- approved submitter;
- administrator.

An approved submitter must only be able to submit or correct data within their assigned competition scope.

### ### Event Processing

The architecture must show the following flow:

```
```text
Submission
  |
  v
Authentication and Scope Check
  |
  v
Schema and Sport-Rule Validation
  |
  +---- Invalid: structured rejection report
  |
  v
Store Ordered Events and Provenance
  |
  v
Derive Statistics
  |
  v
Publish Through API and Frontend
```
```

A correction should follow this flow:

```
```text
Correct Source Event
  |
  v
Record Correction History
  |
  v
```

Identify Affected Statistics

|

v

Recalculate Affected Results

|

v

Update API, Views and Dataset Outputs

...

Intermediate Architecture Planning

The architecture must explain the planned approach for:

- staging whole-season and back-catalogue batches;
- validating batches before publication;
- partial acceptance and rejection reporting;
- idempotent resubmission;
- resumable processing;
- review before publication;
- correction history;
- season, career and competition aggregates;
- selective recomputation;
- reference-result verification;
- representative-scale load testing;
- database indexing;
- API versioning;
- API key management;
- rate limits and quotas;
- caching;
- versioned dataset releases;
- schema and field documentation;
- release checksums.

The implementation details may still change, but the document must identify where these responsibilities will exist.

Advanced Architecture Planning

The architecture must explain the planned approach for:

- analyst-defined statistics;
- validation of statistic definitions;

- versioning of statistic definitions;
- safe or sandboxed execution;
- evaluation across historical event data;
- traceability to event and definition versions;
- live fixture-event ingestion;
- duplicate, late and out-of-order events;
- deterministic ordering;
- replayable processing;
- temporal or as-of-date statistic queries;
- dataset release comparisons;
- aggregate API queries;
- asynchronous large-query jobs;
- change feeds;
- API deprecation;
- contract testing;
- per-consumer usage information;
- anomaly detection;
- conflicting-submitter reconciliation;
- downstream correction propagation.

High-risk Advanced decisions must be identified early rather than being left entirely until Sprint 3.

Deployment Architecture

The document must identify the planned hosting location for:

- the frontend application;
- the backend API;
- PostgreSQL;
- background workers where required;
- file or object storage where required;
- caching where required;
- the documentation website.

The deployment diagram must show:

```
```text
```

```
Developer Push or Pull Request
```

```
|
```

```
v
```

```
Gitea Actions
```



```
|
+---- Install dependencies
+---- Lint and type-check
+---- Run tests
+---- Build applications
+---- Deploy approved changes
+---- Run smoke checks
...
```

The architecture must also identify:

- production and development environments;
- environment-variable handling;
- secret management;
- database migrations;
- health checks;
- deployment rollback considerations;
- public API availability;
- documentation-site availability.

## ## Required Documentation Deliverable

Create or update:

```
```text
docs/architecture/system-architecture.md
```
```

The document should contain:

1. Architecture overview.
2. System context diagram.
3. Component diagram.
4. Main data-flow diagram.
5. Authentication and authorisation flow.
6. Event-submission and validation flow.
7. Event-to-statistic derivation flow.
8. Correction and recomputation flow.
9. API architecture.
10. Database responsibility overview.
11. Intermediate processing architecture.
12. Advanced processing architecture.

13. Deployment architecture.
14. CI/CD flow.
15. Technology stack.
16. Architecture decision summary.
17. Development roadmap.
18. Risks and unresolved questions.
19. Links to relevant Gitea issues.

Diagrams may be created using Mermaid so that they remain version-controlled and editable.

## ## Development Roadmap

The document must contain a roadmap with the following intended outcomes.

### ### Sprint 1

Target a working vertical slice containing:

- confirmed sport and competition scope;
- system architecture;
- database foundation;
- authentication;
- approved submitter permissions;
- event schema;
- event validation;
- direct event submission;
- ordered event storage;
- Basic statistic derivation;
- first API endpoints;
- first public frontend views;
- CI/CD and deployed environments;
- initial automated tests;
- public documentation.

### ### Sprint 2

Target:

- complete Basic functionality;
- file uploads;
- corrections and provenance;

- exports;
- external API integration;
- full Basic testing;
- batch staging and processing;
- idempotent and resumable batches;
- review and publication workflow;
- aggregate statistics;
- selective recomputation;
- performance testing;
- API keys and quotas;
- caching;
- dataset releases;
- most or all Intermediate functionality.

### ### Sprint 3

Target:

- custom statistic definitions;
- safe custom-statistic execution;
- live event feeds;
- out-of-order event handling;
- replayable pipelines;
- temporal queries;
- release comparison;
- asynchronous API jobs;
- change feeds;
- API compatibility and deprecation;
- usage reporting;
- anomaly detection;
- submitter reconciliation;
- downstream correction propagation;
- Advanced testing and documentation;
- near-complete Advanced product.

### ### Final Submission

Reserve this period for:

- release-blocking defect resolution;
- security review;
- accessibility review;

- performance verification;
- production deployment verification;
- final documentation;
- traceability evidence;
- reproducible demonstration data;
- reports;
- presentation;
- release tagging and submission packaging.

Major unplanned features should not be intentionally deferred to the Final Submission period.

## ## Architecture Decisions

Important architectural choices must be recorded with:

- the decision;
- the context;
- alternatives considered;
- the motivation;
- consequences;
- affected components;
- date;
- status.

Where appropriate, create Architecture Decision Records in:

```
```text
docs/architecture/decisions/
```
```

Example filename:

```
```text
docs/architecture/decisions/0001-separate-frontend-and-backend.md
```
```

Likely decisions include:

- separate frontend and backend applications;
- handwritten backend API;
- PostgreSQL as the primary data store;

- Firebase as the identity provider;
- approach to background jobs;
- approach to file storage;
- approach to caching;
- approach to live-feed transport;
- approach to custom-statistic definitions;
- approach to API versioning;
- approach to temporal history.

## ## Acceptance Criteria

- [ ] A high-level system architecture document exists.
- [ ] The frontend and backend are shown as separate applications.
- [ ] The frontend communicates with the backend through HTTP requests.
- [ ] The backend is identified as the owner of the handwritten API.
- [ ] The relationship between the backend and PostgreSQL is documented.
- [ ] Firebase authentication is shown in the architecture.
- [ ] Authentication and authorisation are clearly distinguished.
- [ ] User roles and approved submitter scopes are documented.
- [ ] The event-submission flow is documented.
- [ ] Event-schema validation is documented.
- [ ] Structured submission rejection is documented.
- [ ] Ordered event storage is documented.
- [ ] Event-derived statistic calculation is documented.
- [ ] Event correction and affected-statistic recalculation are documented.
- [ ] Statistic provenance is documented.
- [ ] API filtering, sorting and pagination responsibilities are documented.
- [ ] Dataset export responsibilities are documented.
- [ ] The external API integration point is identified.
- [ ] The documentation website is included in the architecture.
- [ ] CI/CD is included in the architecture.
- [ ] Frontend and backend deployment are included.
- [ ] Database migration responsibilities are documented.
- [ ] Intermediate batch staging and processing are represented.
- [ ] Idempotency and resumability responsibilities are represented.
- [ ] Review and publication workflow responsibilities are represented.
- [ ] Aggregate and selective-recomputation responsibilities are represented.
- [ ] API keys, quotas and rate-limiting responsibilities are represented.
- [ ] Caching responsibilities are represented.
- [ ] Dataset releases are represented.
- [ ] Advanced custom-statistic components are represented.
- [ ] Advanced live-feed components are represented.

- [ ] Late and out-of-order event handling is represented.
- [ ] Replayable processing is represented.
- [ ] Temporal queries and release comparison are represented.
- [ ] Asynchronous jobs and change feeds are represented.
- [ ] API deprecation and contract testing are represented.
- [ ] Anomaly detection and submitter reconciliation are represented.
- [ ] At least one system context diagram has been created.
- [ ] At least one component diagram has been created.
- [ ] At least one main data-flow diagram has been created.
- [ ] A deployment diagram has been created.
- [ ] The technology stack is listed.
- [ ] Important technology choices are motivated.
- [ ] Major architectural risks are recorded.
- [ ] Unresolved stakeholder or lecturer questions are recorded.
- [ ] A Sprint 1 to Final Submission roadmap is included.
- [ ] Relevant Gitea issues are linked from the document.
- [ ] The architecture has been reviewed by the group.
- [ ] Feedback from the group has been incorporated or rejected with motivation.
- [ ] The document has been reviewed and merged through a Pull Request.

## ## Dependencies

Related issues:

- #36 Create and prioritise the complete project backlog
- Confirm the sport, professional competition scope, event vocabulary and required statistics

Replace the second item with its actual Gitea issue number.

The architecture issue may begin before the sport-domain issue is complete, but sport-specific sections must not be finalised using unsupported assumptions.

This issue should be sufficiently complete before major independent implementation begins in:

- frontend application structure;
- API design;
- database schema;
- event processing;
- batch processing;

- live-feed processing;
- custom statistic execution.

## ## Testing and Verification

Verify the architecture by walking through the following scenarios.

### ### Public Read Scenario

1. A public user opens the frontend.
2. The frontend requests fixture data from the backend API.
3. The backend reads the relevant data.
4. The API returns filtered and paginated results.
5. The frontend displays the result.

### ### Authentication Scenario

1. A user signs in through Firebase.
2. The frontend receives an identity token.
3. The frontend sends the token to the backend.
4. The backend verifies the token.
5. The backend loads the application account and role.
6. The backend enforces the requested permission.

### ### Event Submission Scenario

1. An approved submitter sends fixture events.
2. The backend verifies identity.
3. The backend verifies submitter approval and scope.
4. The submission is validated against the event schema.
5. Invalid data receives structured errors.
6. Valid events are stored in order.
7. Statistics are derived.
8. The results become available through the API.

### ### Correction Scenario

1. An authorised user corrects an event.
2. The original history remains traceable.
3. Affected statistics are identified.
4. Only dependent results are recalculated where appropriate.
5. Updated results appear in the API and frontend.

### ### Batch Scenario

1. A whole-season batch is uploaded.
2. It is staged.
3. Validation occurs before publication.
4. Accepted and rejected items are reported.
5. Repeated submission does not duplicate events.
6. Failed processing can resume.

### ### Live-Feed Scenario

1. Events arrive during a fixture.
2. Some events arrive late or out of order.
3. The pipeline creates a deterministic order.
4. Replaying the same events produces the same statistics.
5. Results remain traceable to source events.

The architecture is acceptable only if each scenario has a clear path through the documented components.

## ## Documentation and Evidence

Evidence must include:

- the architecture document;
- architecture diagrams;
- data-flow diagrams;
- deployment diagram;
- development roadmap;
- technology-stack motivation;
- architecture decision records where applicable;
- risk and unresolved-question register;
- evidence of group review;
- links to affected Gitea issues;
- the linked Pull Request.

## ## Definition of Done

This issue is complete when:

- the complete high-level architecture is documented;



- the major components and responsibilities are agreed;
- frontend and backend separation is clear;
- authentication and authorisation are clear;
- the event-to-statistic flow is clear;
- Basic, Intermediate and Advanced architecture needs are represented;
- the deployment and CI/CD approach is documented;
- the roadmap is agreed;
- major risks and unresolved questions are visible;
- team members can begin implementation without inventing conflicting architecture;
- the documentation has been reviewed and merged.

The architecture remains a living document and may be updated through later issues as the project develops.

## ## Git and Pull Request Requirements

After the issue number is assigned, create a branch using:

```
```text
docs/<issue-number>-system-architecture
```
```

Example:

```
```text
docs/38-system-architecture
```
```

Use a commit such as:

```
```text
docs(architecture): add system design and roadmap
```
```

Reference the issue in the commit body:

```
```text
Refs #<issue-number>
```
```

The Pull Request must include:

- the architecture purpose;
- diagrams added;
- major technology decisions;
- event and statistic data flow;
- deployment approach;
- development roadmap;
- risks and unresolved questions;
- verification scenarios performed.

Close the issue through the Pull Request using:

```
```text
```

```
Closes #<issue-number>
```

```
```
```

### ## AI Declaration

> *The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].*

8/6/2026, 2:25:02 PM



wait, surely the issue description isnt that long? just make description and someone will do it

8/6/2026, 2:26:34 PM



You're right. For now, the issue only needs enough detail for someone to understand and complete it.

## ## Description

Create a high-level architecture and development roadmap for the Sport Analytics Tool.

The architecture should show how the following parts connect:

- frontend application;
- handwritten backend API;
- PostgreSQL database;
- Firebase authentication;
- user roles and approved submitter permissions;
- event submission and validation;
- event-derived statistics;
- file uploads and dataset exports;
- external API integration;
- CI/CD and deployment;
- future Intermediate and Advanced components such as batch processing, caching, live events and custom statistics.

Create the architecture documentation in:

``docs/architecture/system-architecture.md``

Include:

- a high-level architecture diagram;
- the responsibility of each main component;
- the main data flow from event submission to calculated statistics;
- authentication and authorisation flow;

- deployment and CI/CD overview;
- planned work for Sprint 1, Sprint 2, Sprint 3 and Final Submission;
- major risks or unresolved decisions.

## ## Acceptance Criteria

- [ ] The frontend and backend are shown as separate applications.
- [ ] The main system components and responsibilities are documented.
- [ ] The event submission, validation and statistic calculation flow is shown.
- [ ] Authentication and user permissions are explained.
- [ ] The deployment and CI/CD approach is documented.
- [ ] Basic, Intermediate and Advanced architecture needs are considered.
- [ ] A development roadmap for all milestones is included.
- [ ] The document is reviewed by the group and merged through a Pull Request.

## ## Dependencies

- #36 Create and prioritise the complete project backlog
- Confirm the sport, professional competition scope, event vocabulary and required statistics

## ## AI Declaration

> *The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].*

8/6/2026, 2:26:42 PM



ok, whats next? surely we start the project at some point?

8/6/2026, 2:27:46 PM



### Thinking

- **Used personal\_context tool**

✓ Worked for 40s

Yes. We have done enough planning to start implementation.

The architecture issue should be created, but it does **not** need to be completed before everyone starts coding. The sport-domain work and architecture can continue while other members work on independent foundations.

## Next issue: first visible implementation

### Title

Build frontend authentication pages and navigation controls

**Template:** Feature

### Labels

```
type: feature
area: frontend
tier: common
priority: high
```

**Milestone:** Sprint 1

**Board:** Backlog, then move to Ready once assigned

Before creating it, check whether your existing Firebase issue already includes the actual frontend forms and buttons. If it only sets up Firebase and backend token verification, this is a separate issue.

## Copy-and-paste description

### ## Description

Build the user-facing authentication pages and navigation controls for the Sport Analytics Tool.

When a user is signed out, the navigation must display:

- Create Account
- Sign In

When a user is signed in, the navigation must display:

- Account
- Sign Out

Create the required sign-up and sign-in forms and connect them to the existing Firebase authentication setup.

Also include:

- useful validation and error messages;
- a forgotten-password link;
- responsive layouts;
- accessible form labels and keyboard navigation;
- loading states while authentication requests are processing.

### ## Acceptance Criteria

- [ ] Signed-out users can see Create Account and Sign In buttons.
- [ ] Users can create an account through the frontend.
- [ ] Users can sign in through the frontend.
- [ ] Signed-in users can see Account and Sign Out controls.
- [ ] Users can sign out successfully.
- [ ] A forgotten-password link is available.
- [ ] Authentication errors are shown clearly.
- [ ] Forms work on desktop and mobile screen sizes.
- [ ] Form fields have accessible labels.
- [ ] Automated tests cover the main authentication interface behaviour.
- [ ] The work is reviewed and merged through a Pull Request.

## ## Dependencies

- Existing Firebase authentication foundation
- Existing frontend application

## ## AI Declaration

*> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].*

This is genuine project implementation. In parallel, another person should finish the sport-domain decision because the event schema, database and statistic engine cannot be finalised until the sport is confirmed. The project specifically requires accounts to support sign-up, sign-in, password reset and deletion, so this work directly satisfies a mandatory project-wide requirement. `project_briefs.pdf`

8/6/2026, 2:27:57 PM



ok, now give me the next few titles and lemme see i agree with you, then we will do them all at once. lets do the next 10?

also note: statistics qwill be viewable by all wothout signing in, signing in only required to upload stats

8/6/2026, 2:32:35 PM



## Thinking



### Inspecting Project Repository Structure

- **Planning implementation issues**

I'm aligning the roadmap around public read access and protected write operations. I'll propose ten sequenced titles covering authorization, public pages, submission workflows, storage, moderation, and testing without overcomplicating the foundation.

- **Planned implementation issue resolution**

- **Inspected project structure, application files, and authentication documentation**

✓ Worked for 1m 20s

Yes - that changes the access model clearly:

Public, no sign-in:

Fixtures, events, competitors, statistics, public API and dataset browsing

Signed-in:

Account management

Approved submitter only:

Upload fixture events and view their submissions

Administrator only:

Approve submitters and assign submission scopes



Only approved submitters should be able to submit event data, while reading fixtures, events and derived statistics can remain public. `sport_analytics.pdf`

Add these two acceptance criteria to the authentication issue:

- [ ] Public fixture and statistics pages remain accessible without signing in.
- [ ] A signed-in user who is not an approved submitter cannot upload statistics or event data.

## Proposed next 10 issues

### 1. Implement the application account, role, approved-submitter and scope database schema

**Labels:** `type: feature`, `area: database`, `area: api`, `tier: basic`, `priority: high`

**Milestone:** Sprint 1

Stores the Firebase user link, application role, submitter approval status and which competition or fixtures they may upload.

---

### 2. Implement account synchronisation, profile API and role-based authorisation middleware

**Labels:** `type: feature`, `area: api`, `area: database`, `tier: basic`, `priority: high`

**Milestone:** Sprint 1

Connects authenticated Firebase identities to application accounts and lets the backend check whether someone is an approved submitter or administrator.

---

### 3. Build the administrator submitter-approval and scope-management interface

**Labels:** `type: feature`, `area: frontend`, `area: api`, `tier: basic`, `priority: high`

**Milestone:** Sprint 1

Allows an administrator to approve a user and assign the competition, team or fixtures they may submit.

---

### 4. Implement the competition, season, competitor, participant and fixture database schema

**Labels:** type: feature , area: database , area: data , tier: basic , priority: high

**Milestone:** Sprint 1

Creates the main sport entities that events and statistics will belong to.

---

## 5. Implement public competition, fixture and participant read APIs

**Labels:** type: feature , area: api , area: data , tier: basic , priority: high

**Milestone:** Sprint 1

These endpoints must work without authentication and support filtering and pagination.

---

## 6. Build public competition, fixture and participant browsing pages

**Labels:** type: feature , area: frontend , tier: basic , priority: high

**Milestone:** Sprint 1

Anyone should be able to browse the sport data without creating an account.

---

## 7. Define the executable sport-specific event schema and example submissions

**Labels:** type: feature , area: data , area: api , area: documentation , tier: basic , priority: high

**Milestone:** Sprint 1

Defines what every event looks like, including event types, required fields, ordering and valid examples.

This depends on confirming the sport.

---

## 8. Implement event validation with detailed submission rejection messages

**Labels:** type: feature , area: api , area: data , tier: basic , priority: high

**Milestone:** Sprint 1

Invalid submissions must explain exactly which events and fields are incorrect rather than simply saying the upload failed.

---

## 9. Implement authenticated event submission with provenance and scope enforcement

**Labels:** type: feature , area: api , area: database , area: data , tier: basic , priority: high

**Milestone:** Sprint 1

Only approved submitters may submit. Every accepted event must record its fixture, order, submission and submitter.

I suggest calling it **event submission**, not “statistics upload”, because users upload the underlying events and the system calculates the statistics.

---

## 10. Implement event-derived fixture statistics and the public statistics API

**Labels:** type: feature , area: api , area: data , tier: basic , priority: high

**Milestone:** Sprint 1

Calculates statistics from accepted events and exposes them publicly without requiring authentication.

---

### Immediately after these

The next two would naturally be:

Build the approved-submitter event upload and validation-results interface

and:

Build public fixture and competitor statistics pages

That gives you the full first workflow:

```
Administrator approves submitter
↓
Approved submitter uploads events
↓
Backend checks scope and validates events
↓
Events are stored in order
↓
```

Statistics are calculated



Everyone can view the statistics publicly

These ten are the right next group, although issues 1-3 can run in parallel with issues 4-7 once the sport is settled.

*8/6/2026, 2:32:41 PM*



OK, give me these 12, with title  
LABELSD ETC  
AND THEN A Markdown to paste into discription  
for each of the 12  
remeber project methodologies, ai declerations etc

8/6/2026, 3:40:13 PM



## Thinking

- **Writing the issue templates**

I'm structuring all 12 GitHub issues with exact labels, objectives, specifications, and acceptance criteria. I'm also keeping naming consistent with the project's methodology and organizing work into actionable, copy-ready sections.

- **Wrote the issue templates**



- **Inspecting repository contents and methodologies**

- **Creating the issue set**

I'm preparing 12 copy-ready GitHub issues with consistent titles, labels, objectives, specifications, acceptance criteria, and dependencies. I'm leaving dependency links explicit so issue numbers can be added after creation.

- **Created the issue set**

- **Inspected repository documentation, methodologies, and templates**

✓ Worked for 1m 25s

I prepared all 12 as **copy-ready Gitea issues** in one Markdown file:

[Download the 12 Sprint 1 Gitea issues](#)

Each issue includes:

- Title
- Feature template
- Labels

- Sprint 1 milestone
- Initial `Backlog` position
- Suggested branch name
- Concise description
- Scope
- Acceptance criteria
- Dependencies
- Testing and evidence
- Security, accessibility and responsiveness
- AI declaration

## The 12 issues

1. **Implement the application account, role, approved-submitter and scope database schema**
2. **Implement account synchronisation, profile API and role-based authorisation**
3. **Build administrator submitter approval and scope management**
4. **Implement the competition, season, competitor, participant and fixture database schema**
5. **Implement public competition, fixture and participant read APIs**
6. **Build public competition, fixture and participant browsing pages**
7. **Define the executable sport-specific event schema and example submissions**
8. **Implement event validation with detailed rejection messages**
9. **Implement approved-submitter event submission with provenance and scope enforcement**
10. **Implement event-derived fixture statistics and the public statistics API**
11. **Build the approved-submitter event upload and validation-results interface**
12. **Build public fixture and competitor statistics pages**

The access rules are applied consistently throughout:

No sign-in required:

- Browse competitions and fixtures
- View competitors and participants
- View events and derived statistics
- Use the public read API

Sign-in and approval required:

- Submit event data

Administrator required:

- Approve submitters
- Assign or change submission scopes

The project brief requires submissions to come from approved submitters with defined scopes, while other platforms must be able to read fixtures, events and statistics through the API. [sport\\_analytics.pdf](#)

Create each issue in **Backlog**. Only move it to **Ready** once it is assigned and its dependencies are available, following your Scrumban methodology. The included AI declarations follow the course attribution requirements. [ai\\_policy\\_20\\_jul.pdf](#)

8/6/2026, 3:40:26 PM



does gittea have a way where i can import a doc and they make the issues or soething?

8/6/2026, 3:45:46 PM



## Thinking



### Searching the web

[docs.gitea.com](https://docs.gitea.com)

- **Checking import options**

Gitea doesn't generally convert a Markdown document directly into issues. The cleanest route is an API script that reads structured issue data, creates titles and bodies, and resolves labels, milestones, and assignees automatically. A one-time token with issue-writing access is typically needed.

- **Checked import options**

- **Searched 1 website**

✓ Worked for 32s

Not directly. **Gitea does not have a normal button where you upload a Markdown or Word document and it automatically turns each section into an issue.**

Its built-in import feature is mainly for migrating an entire repository from services such as GitHub, GitLab or another Gitea instance, including their existing issues. It is not intended for importing an arbitrary planning document. ([Gitea Documentation](https://docs.gitea.com))

## The fastest option for your 12 issues

Use a small script that reads a structured file and creates the issues through the **Gitea API**.

The API can create an issue with:

- title;
- Markdown description;



- assignees;
- labels;
- milestone.

Gitea provides the endpoint:

```
POST /api/v1/repos/{owner}/{repo}/issues
```

([Gitea Documentation](#))

The process would be:

```
Our 12 issue descriptions
 ↓
issues.json
 ↓
Node script
 ↓
Gitea API
 ↓
12 issues created automatically
```

The script can also:

- find labels by their names;
- find the `sprint 1` milestone;
- avoid creating a duplicate when an issue with the same title already exists;
- print the new issue numbers;
- stop safely if a label or milestone is missing.

You would need to create a personal access token under **Gitea → Settings → Applications** with issue-writing permission. Gitea's current documentation identifies `write:issue` as the scope covering issues, labels and milestones, although the exact token options shown may depend on the version used by Wits. ([Gitea Documentation](#))

## My recommendation

Do **not** manually create all 12. We should convert the issue pack into:

```
scripts/create-gitea-issues.mjs
scripts/issues/sprint-1-issues.json
```

You would then run one command, such as:

```
GITEA_TOKEN="your-token" node scripts/create-gitea-issues.mjs
```

The token must remain private and must **never** be committed to the repository. The project board position may still need to be set manually afterward, depending on which project-board API features are available on your Wits Gitea version.

## Sources

[Migration](https://docs.gitea.com) docs.gitea.com

8/6/2026, 3:45:48 PM



ok, so ive doen the first issue, give me process to make the next 11 using that instead of manual!

8/6/2026, 3:47:30 PM



## Thinking



### Searching the web

[docs.gitea.com](https://docs.gitea.com)

[dictionary.cambridge.org](https://dictionary.cambridge.org)

[sbbit.jp](https://sbbit.jp)

[github.com](https://github.com)

[about.gitea.com](https://about.gitea.com)

[en.wikipedia.org](https://en.wikipedia.org)

- Searched 6 websites



### Searching the web

- Searching 13 websites



### Inspecting Project Archive Contents

- Inspected project archive, Git remotes, and public API version endpoint



### Searching the web

- Planning issue creation

I'm preparing to parse the existing markdown, generate JSON for issues 2-12, and create a script that assigns labels and a milestone. Project-board placement may require separate configuration.

- Planned issue creation



### Searching Gitea API add issue to project board column

- Searching 10 websites

- Preparing automation

I'm designing an idempotent script that parses the remaining markdown issues, creates labels and milestones as needed, skips duplicate titles, and records created issue numbers while leaving board placement configurable.



- **Prepared automation**
  - **Used container tool**
- ✓ Worked for 2m 13s

I made the importer for the **remaining 11 issues**. The first issue is excluded because you already created it.

### Download the Gitea issue importer

Gitea supports creating issues through its API, including the title, Markdown body, labels, milestone and assignees. Personal access tokens are created under **User Settings → Applications**, and the token is normally displayed only once. ([Gitea Documentation](#))

## 1. Download and extract the ZIP

Extract:

```
gitea-next-11-importer.zip
```

The folder contains:

```
create-gitea-issues.mjs
issues-next-11.json
README.md
.gitignore
```

## 2. Create a Gitea access token

In Gitea:

1. Click your profile picture.
2. Open **Settings**.
3. Open **Applications**.
4. Create a new personal access token.
5. Give it **Issues: Read and Write** permission, or `write:issue` if that is how your version displays it.
6. Copy the token immediately.

Do not send the token to me, put it in the JSON file, or commit it to Git.

### 3. Open PowerShell in the extracted folder

For example:

```
cd "$HOME\Downloads\gitea-next-11-importer"
```

### 4. Temporarily set your token

```
$env:GITEA_TOKEN = "PASTE_YOUR_TOKEN_HERE"
```

The script is already configured for:

```
Gitea: https://sdp.ms.wits.ac.za
Owner: git-push-pray
Repository: Sport-Analytics-Tool
Milestone: Sprint 1
```

Leave the issues unassigned for now. They can be assigned during backlog refinement.

### 5. Run the safe test first

```
node .\create-gitea-issues.mjs
```

This is a **dry run**. It creates nothing.

It checks:

- whether your token works;
- whether the repository is accessible;
- whether the `Sprint 1` milestone exists;
- whether all required labels exist;
- whether any of the issue titles already exist;
- which 11 issues would be created.

If labels are missing, it will stop and tell you exactly which labels to create.

### 6. Create the 11 issues

Once the dry run looks correct:

```
node .\create-gitea-issues.mjs --confirm
```

The script will:

- create the issue titles;
- paste in the full Markdown descriptions;
- apply the labels;
- apply the `Sprint 1` milestone;
- skip exact-title duplicates;
- print every new issue number;
- create a report called:

```
created-issues.json
```

The official Gitea API uses authenticated requests to create repository issues; the token is passed in the authorization header. ([Gitea Documentation](#))

## 7. Add them to the project board

The importer deliberately does not alter your project board.

Using the numbers in `created-issues.json`:

1. Open each created issue.
2. Add **Sport Analytics project board**.
3. Place it in **Backlog**.
4. Keep it there until the issue is assigned and dependency-ready.
5. Move it to **Ready** only when the team intends to begin it.

This follows your Scrumban methodology rather than placing all 11 immediately into active work.

## 8. Remove the token afterward

Remove-Item Env:GITEA\_TOKEN

You may also delete the access token in Gitea after the import if it was created only for this task.

## Sources

[API Usage | Gitea Documentation](https://docs.gitea.com) docs.gitea.com

8/6/2026, 3:47:31 PM



64ea10f8-b9b5-47b9-846c-20...

no way!!! it worked  
can u do that for the rest of them, just add project field  
start by doing all sprint 1 ones, then we will later do  
sprint 1 stretch and then 2 etc

8/6/2026, 3:59:37 PM



## Thinking

- **Planning the importer**

I'm identifying which Sprint 1 issues already exist, then I'll build a new importer for the remaining issues. I'm also checking how to assign projects automatically while avoiding duplicates.

- **Planned the importer**



- **Searching the web**

[docs.gitea.com](https://docs.gitea.com)

[swagger.io](https://swagger.io)

- **Searched 2 websites**



- **Searching the web**

- **Searching 5 websites**



- **Inspecting Catalogue and Generated Issues**

- **Reconciling remaining work**

I'm matching the catalogue against issues already created, including combined or renamed items. I'll include unfinished Sprint 1 work while preserving the earlier decision to defer stakeholder interviews, then prepare project assignment automatically.

- **Reconciled remaining work**

- **Inspected catalogue and generated issues**







## Searching Gitea API add issue to project endpoint CreateProjectIssue official source

[docs.gitea.com](https://docs.gitea.com)

[dictionary.cambridge.org](https://dictionary.cambridge.org)

[github.com](https://github.com)

[pmi.org](https://pmi.org)

[en.wikipedia.org](https://en.wikipedia.org)

[rdrr.io](https://rdrr.io)

[projectmanager.com](https://projectmanager.com)

[postman.com](https://postman.com)

[apis.io](https://apis.io)

[projectmanagementsa.co.za](https://projectmanagementsa.co.za)

[go-gitea-gitea.mintlify.app](https://go-gitea-gitea.mintlify.app)

[arxiv.org](https://arxiv.org)

- **Choosing project assignment**

The catalogue comparison is complete. I'm treating project assignment as a separate step so the importer can create the remaining Sprint 1 issues cleanly and avoid relying on uncertain interface behavior.

- **Chose project assignment**



## Searching Gitea batch issue handling assign project multiple issues UI

- **Searched 12 websites**



## Extracting Remaining Sprint 1 Catalogue Issues

- **Extracted remaining Sprint 1 catalogue issues**



## Searching the web

[github.com](https://github.com)

- **Searched 1 website**



## Inspecting Repository Methodology Guidelines

- **Reviewed repository methodology guidelines and templates**

✓ Worked for 5m 29s

Excellent! I created the importer for the **17 remaining Sprint 1 issues**:

[Download the remaining Sprint 1 importer](#)

It excludes work already covered by your existing issues and the previous import. It also continues to exclude the stakeholder-clarification issue that you chose to skip.

## Run it

Extract the ZIP, open PowerShell in the folder and use the same token:



```
$env:GITEA_TOKEN = "PASTE_YOUR_TOKEN_HERE"
```

Run the dry test:

```
node .\create-sprint1-issues.mjs
```

When the list looks correct:

```
node .\create-sprint1-issues.mjs --confirm
```

The importer will:

- create all 17 issues;
- add their descriptions;
- apply all type, area, tier and priority labels;
- assign the `Sprint 1` milestone;
- skip exact-title duplicates;
- create `created-issues.json`;
- create `assign-project-in-browser.js`.

## Add the Project field

Gitea does not currently expose repository project-board assignment through its normal REST API, although its web interface has an internal project-assignment route. ([GitHub](#))

After the issues are created:

1. Open issue **#36** in Gitea while signed in.
2. Press **F12** and open **Console**.
3. Open `assign-project-in-browser.js` in Notepad.
4. Copy the entire script into the console.
5. Press **Enter**.

It will find:

and assign all imported issues to that project in one operation. Gitea's normal interface similarly assigns issues through the Projects field in the issue sidebar. ([Gitea Blog](#))

Refresh the board afterward and confirm they are in **Backlog**. Keep them there until each has an assignee, clear dependencies and meets your Definition of Ready.

## Sources

[feat: REST API for repository project boards · Issue #36824](#) github.com

[Supercharge Your Issue Management Workflow: Introducing Org/User Level Projects, Scoped Labels & Advanced Label Templates | Gitea Blog](#) blog.gitea.com

8/6/2026, 3:59:55 PM



F12 wanst me to log into my asus account or something?

8/6/2026, 4:15:28 PM



That means your ASUS laptop has **F12 assigned to a laptop shortcut**, not the browser developer tools. You do **not** need to log into an ASUS account.

In Chrome or Edge, use:

```
Ctrl + Shift + J
```

That opens the **Console** directly.

Other options:

```
Ctrl + Shift + I
```

then click the **Console** tab, or:

1. Right-click anywhere on the Gitea page.
2. Choose **Inspect**.
3. Open **Console**.

You can also try:

```
Fn + F12
```

Once the Console is open, do not paste anything yet—tell me what you see, and I'll guide you through the project-assignment step safely.

8/6/2026, 4:15:40 PM



3e7423d3-1365-4fb9-bd0b-4a...

8/6/2026, 4:17:31 PM



## Thinking



### Searching the web

[github.com](https://github.com)

- **Explaining the failure**

I'm seeing a known Gitea 1.21.2 issue: project assignment after issue creation returns 404, so the importer's workflow cannot finish. The safest fix is upgrading Gitea, then rerunning only project assignment or assigning projects manually; avoid deleting existing issues.

- **Explained the failure**



### Searching the web

- **Choosing a workaround**

I'm revising the approach: existing issues may remain unassigned because post-creation project updates are broken on this Gitea version. I'd upgrade first; afterward, I'd retry assignment or create future issues with their project selected during creation.

- **Chose a workaround**



### Searching the web

- **Searched 1 website**

✓ Worked for 1m 20s

That 404 is **not your fault**. The issue importer worked; only the automatic **Project field assignment** failed.

The failed request is:

```
POST /git-push-pray/Sport-Analytics-Tool/issues/projects
```



That exact route is associated with a known Gitea bug where assigning an already-created issue to a project returns `404 Not Found`. ([GitHub](#)) Gitea also does not yet provide a supported repository-project REST API that we can safely use instead. ([GitHub](#))

## What happened

- The Sprint 1 issues were still created.
- Their titles, descriptions, labels and milestones should still be correct.
- No issues were deleted or damaged.
- They simply were not added to `Sport-Analytics-Tool-Proj`.

Close the Console now.

## Fastest recovery

Open:

```
Projects → Sport-Analytics-Tool-Proj
```

In the **Backlog** column, look for either:

```
+
Add issues
Add item
```

The wording depends on your Gitea version.

Use that option to search for and add the newly created issue numbers from:

```
created-issues.json
```

You should be able to select several issues before confirming.

## Check before doing all of them

First add just **one** imported issue through the project board.

Then open that issue and check whether this appears:

```
Projects
```

If that works, add the remaining imported issues through the project board.

## If the board also gives a 404

Leave the issues as they are for now. They still have:

- Sprint 1 milestone
- Tier labels
- Type labels
- Area labels
- Priority labels

That means they remain organised and filterable even without the Project field.

For the next importer batch, the project must be attached **when each issue is initially created**, rather than trying to assign it afterward, because post-creation project assignment is what this server is rejecting.

### Sources

[Cannot read properties of undefined \(reading 'error'\) · Issue #28538 · go-gitea/gitea · GitHub](#) [github.com](#)

8/6/2026, 4:17:33 PM



ok done, now thats all sprint 1, right??  
what will we have at the end of them?

8/6/2026, 4:20:28 PM



## Thinking

- **Confirming sprint coverage**

Yes—Sprint 1 is covered, assuming the imported issues and manual additions are included. I'm excluding the stakeholder-clarification item and any tickets already mapped elsewhere. I'll verify the final catalogue against the project brief and sports-domain requirements.

- **Confirmed sprint coverage**

- **Clarifying sprint boundaries**

Yes—the Sprint 1 plan is complete, with advanced capabilities intentionally deferred. Sprint 2 covers batch seasons, publication review, aggregates, API operations, caching, and releases; Sprint 3 covers live feeds, custom statistics, time travel, anomaly detection, and reconciliation.

✓ Worked for 30s

Yes - **that is now the complete planned Sprint 1 backlog**, provided the earlier foundation issues are also still on the board, such as CI, Firebase, PostgreSQL/Supabase, Azure deployment, Cloudflare documentation, Git methodology, project methodology and AI evidence.

The only deliberately skipped item is the separate stakeholder-decision issue. Your repository already contains the stakeholder-meeting process, so you can record actual meeting notes as you go rather than creating that extra issue now.

A complete backlog does **not** mean every Sprint 1 issue must immediately move to **Ready** or **In Progress**. Keep most in **Backlog**, assign work according to dependencies and your work-in-progress limit, and only move issues forward when someone can genuinely begin them.

## What you should have at the end of Sprint 1



## 1. A properly planned project

You should have:

- A complete and prioritised backlog
- A confirmed sport and professional competition
- A defined fixture, event vocabulary and statistic catalogue
- High-level architecture diagrams
- Frontend user journeys and wireframes
- A Sprint 1-3 development roadmap
- Recorded architecture decisions
- Working Git and project methodologies
- Clear links between issues, branches, commits and Pull Requests

This directly supports the Sprint 1 focus on project setup, removing blockers and establishing a strong direction for the semester. [project\\_briefs.pdf](#)

## 2. A deployed technical foundation

You should have:

```
Frontend application
 ↓ HTTP
Handwritten backend API
 ↓
PostgreSQL database
```

Together with:

- Firebase authentication
- Gitea Actions CI
- Automated linting, formatting, type checking, testing and builds
- Azure frontend deployment
- Azure backend deployment
- Public MkDocs documentation on Cloudflare

- Repeatable database migrations
- Development seed data
- Deployment smoke checks

### 3. Public access without signing in

Anyone should be able to:

- Browse competitions and seasons
- Browse fixtures
- View competitors and participants
- View ordered fixture events
- View derived statistics
- Use the public read API

A visitor should **not** be forced to create an account merely to view statistics.

### 4. Complete account functionality

Users should be able to:

- Create an account
- Sign in
- Sign out
- Reset a forgotten password
- View their account
- Delete their account

Public pages must remain available while signed out.

### 5. Approved-submitters and administrators

An administrator should be able to:

- View registered users
- Approve or revoke submitter access

- Assign a submitter to a defined competition scope
- Prevent submitters from working outside that scope

A normal signed-in user should **not** automatically be allowed to upload data.

## 6. The core event-submission workflow

An approved submitter should be able to:

1. Sign in.
2. Select an assigned fixture.
3. Submit ordered event data.
4. Receive detailed validation errors when something is wrong.
5. Successfully submit valid events.
6. Receive a submission reference.

Every accepted event should record:

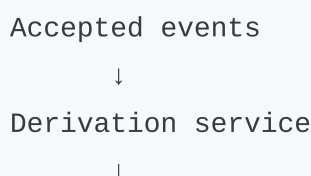
- Its stable identifier
- Its order within the fixture
- The fixture it belongs to
- The submission it came from
- The approved submitter who supplied it
- Its schema version

The project brief requires the platform to be based on individual fixture events rather than manually entered totals. `sport_analytics.pdf`

## 7. Event-derived statistics

Your system should calculate the first important sport statistics from the stored events.

For example:



Fixture statistics



Public API



Public statistics pages

Submitters should upload **events**, not type final statistics manually.

At least one reference fixture should prove that:

- The event order is preserved
- The expected statistics are calculated correctly
- The same events always produce the same results
- Each statistic can be traced back to contributing events

## 8. A usable public API

The handwritten API should provide at least the initial public endpoints for:

- Competitions
- Seasons
- Fixtures
- Competitors
- Participants
- Events
- Fixture statistics

It should include:

- Stable identifiers
- Filtering
- Deterministic sorting
- Pagination
- Clear error responses
- OpenAPI documentation

The project brief treats the API as the platform's central product rather than merely something supporting the website. `sport_analytics.pdf`

## 9. A usable frontend demonstration

By the Sprint 1 demonstration, you should be able to show:

```
Public visitor browses a fixture
 ↓
Views its ordered events
 ↓
Views statistics calculated from those events
```

And separately:

```
Administrator approves a submitter
 ↓
Submitter signs in
 ↓
Submitter uploads events
 ↓
Invalid data receives useful errors
 ↓
Valid data produces public statistics
```

That is a genuine end-to-end vertical slice rather than isolated setup work.

## 10. Testing and evidence

You should have automated tests for:

- Public versus protected access
- Administrator permissions
- Approved-submitter permissions
- Competition-scope enforcement
- Valid event submissions
- Invalid event submissions
- Transaction rollback
- Event ordering and provenance

- Statistic derivation
- Public API access
- Main frontend states

You should also have:

- A representative seed dataset
- One manually verified reference fixture
- Passing `npm run check`
- Gitea Actions evidence
- Updated documentation
- Pull Request evidence
- AI declarations and usage records

## 11. Sprint close-out

At the end of the sprint, you should:

- Demonstrate the product
- Record stakeholder feedback
- Hold a retrospective
- Return unfinished work to the backlog honestly
- Record defects and limitations
- Confirm CI passes on `main`
- Create the annotated `sprint-1` Git tag

## What Sprint 1 will not yet fully include

These are mainly for later sprints:

- Whole-season batch ingestion
- Resumable and idempotent batch processing
- Review before publication
- Season and career aggregates

- API keys, quotas and rate limits
- Caching
- Versioned dataset releases
- Analyst-defined custom statistics
- Live fixture feeds
- Late and out-of-order live-event handling
- As-of-date statistics
- Dataset-release comparisons
- Anomaly detection
- Conflicting-submitter reconciliation

So, at the end of Sprint 1, you should have a **small but real working Sport Analytics platform**: an approved user submits events, the platform validates them, derives statistics, and anyone can view those statistics publicly. That is the correct foundation for completing Intermediate in Sprint 2 and Advanced in Sprint 3.

*8/6/2026, 4:20:40 PM*



is testing there as part of issues? shouldnt there be one to setup testing? how is testing working?

8/6/2026, 4:26:21 PM



## Thinking



### Inspecting Repository Testing Setup

- **Inspected repository testing setup, packages, workflows, documentation, and tests**
- **Assessing testing coverage**

I'm confirming that the repository already has a solid testing foundation and that testing appears throughout the issue plan. One gap remains: database integration, coverage reporting, and browser E2E setup are documented but not fully configured, so I'm proposing a focused Sprint 1 infrastructure issue.
- **Assessed testing coverage**
- **Planning test setup**

I'm seeing testing across feature acceptance criteria, the vertical-slice issue, and later integration, E2E, accessibility, security, and performance work. The repository has core tools, but test databases, coverage, browser automation, and CI wiring still need one focused Sprint 1 setup issue.
- **Planned test setup**
- **Evaluated testing infrastructure and designed a documented vertical-slice strategy**

✓ Worked for 2m 1s

Yes - testing is already partly set up and partly represented in the issues, but there is **one gap** in the Sprint 1 backlog I gave you: a dedicated issue for completing the shared test environment.

## What is already set up in the repo

I checked the uploaded repository. You already have:



- **Vitest** for frontend, backend and shared contracts.
- **React Testing Library + jsdom** for frontend component tests.
- **Supertest** for backend API tests.
- Existing backend tests for:
  - the health endpoint;
  - Firebase authentication behaviour.
- An existing frontend rendering/API-state test.
- A root command:

```
npm test
```

- A complete local/CI check:

```
npm run check
```

That runs structure checks, formatting, linting, type checking, tests and builds. Your Gitea CI workflow already runs this on pushes to `main` and Pull Requests. The repository testing strategy also defines unit, API integration, UI component, end-to-end, accessibility, performance and user testing layers. `Pasted markdown.md`

So the **basic test runners are already wired**. You should not create an issue simply called “Install Vitest.”

## Testing that is already in your issues

Testing appears in two ways.

### 1. Tests inside every feature issue

The person implementing a feature must also test that feature in the same branch.

For example:

```
Implement event validation
+
Write validation tests
+
```

```
Update documentation
 =
One completed issue
```

A feature should not be considered complete when the code works manually but its required tests are missing.

## 2. Dedicated testing issues

You already imported:

```
Add backend and frontend tests for the Basic vertical slice
```

That issue is intended to check that the different features work together, not merely that each file works alone. `COMS3011A_Complete_Gitea_Issue_Catalogue.md`

Later backlogs also include separate work for:

- Backend integration tests
- Frontend integration tests
- Browser end-to-end tests
- Accessibility testing
- Performance testing
- API contract tests
- User testing
- Security and failure testing

This is important because Sprint 2 awards **10% for automated testing**, with the strongest level requiring useful UI and API tests, and the final project also includes testing as a marked category. `project_briefs.pdf` `project_briefs.pdf`

## What is not fully set up yet

Your repository currently has the initial runners, but it does not yet appear to have a complete shared testing environment for:

- An isolated PostgreSQL test database

- Automatic test database migration, reset and seed
- Test data builders or fixtures
- Coverage reporting
- A browser end-to-end test runner
- Automated accessibility checks
- Performance-test tooling
- Full Firebase emulator integration across frontend, API and test users

The `tests/e2e`, `tests/accessibility` and `tests/performance` folders currently describe what should eventually go there, but they are not yet complete executable suites.

## Add this Sprint 1 issue

This should be added before the larger vertical-slice testing issue.

### Title

Complete the automated testing foundation and test-environment setup

**Template:** Task

### Labels

```
type: testing
area: frontend
area: api
area: database
area: infrastructure
tier: common
priority: high
```

**Milestone:** Sprint 1

**Board:** Backlog

Its purpose should be to:

- configure a safe test database;
- automate migrations and seed data for tests;
- define test environment variables;

- support Firebase emulator test identities;
- add coverage commands and reporting;
- choose and configure the browser end-to-end runner;
- ensure CI can run the relevant tests;
- document where each type of test belongs.

This is the **testing infrastructure issue**.

The existing issue:

```
Add backend and frontend tests for the Basic vertical slice
```

then uses that infrastructure to write the actual product tests.

## How testing should work during development

For each feature:

```
Issue assigned
 ↓
Branch created
 ↓
Feature code written
 ↓
Relevant tests written on the same branch
 ↓
Developer runs targeted tests
 ↓
Developer runs npm run check
 ↓
Pull Request opened
 ↓
Gitea Actions runs all checks again
 ↓
Failure = cannot merge
 ↓
Review passes and code is merged
```

Examples:

- Statistic formula → unit tests
- API endpoint → Supertest integration tests
- Sign-in form → React Testing Library tests
- Database transaction → test-database integration test
- Full submit-and-view journey → browser end-to-end test
- Responsive page → accessibility and viewport checks

The dedicated Sprint 1 testing issue then proves the combined workflow:

```
Administrator approves submitter
↓
Submitter uploads valid events
↓
Invalid events are rejected correctly
↓
Valid events are stored
↓
Statistics are calculated
↓
Public user views statistics without signing in
```

So the correct answer is: **testing is already built into the feature issues and CI, but yes, we should add one dedicated testing-foundation issue before relying on the complete vertical-slice test issue.**

*8/6/2026, 4:26:23 PM*



so give me the issue to copy and paste

8/6/2026, 4:31:02 PM



## Issue details

### Title

Complete the automated testing foundation and test environment setup

**Template:** Task

### Labels

```
type: testing
area: frontend
area: api
area: database
area: infrastructure
tier: common
priority: high
```

**Milestone:** Sprint 1

**Initial board column:** Backlog

**Suggested branch:**

test/<issue-number>-testing-foundation

## Copy into the Gitea description

### ## Description

Complete the shared automated testing foundation for the frontend, backend API, database and browser-level workflows.

The repository already uses Vitest, React Testing Library and Supertest. This issue must complete the supporting test environment so that feature teams can write reliable tests and Gitea Actions can run them consistently.

This issue sets up testing infrastructure. Tests for individual features must still be included in their own implementation issues.

## ## Scope

### ### Included

- Confirm the test structure for unit, component, API integration and end-to-end tests.
- Configure an isolated PostgreSQL test database.
- Add commands to migrate, reset and seed the test database safely.
- Create reusable test fixtures, builders or helper functions.
- Configure test environment variables without committing secrets.
- Configure Firebase authentication testing using emulator or approved test doubles.
- Configure browser end-to-end testing using Playwright or the team's approved alternative.
- Add test coverage reporting.
- Ensure frontend, backend, database and shared-contract tests run from the repository root.
- Ensure the testing commands can run in Gitea Actions.
- Document where each type of test belongs and how developers run it.
- Add at least one working example for each configured testing layer.

### ### Excluded

- Writing every feature test for the project.
- Full Basic workflow testing.
- Performance testing at Intermediate scale.
- Formal user testing.
- Final accessibility and security audits.
- Testing against the production database or production Firebase project.

## ## Required Testing Layers

The foundation must support:

- unit tests for statistic calculations and utility functions;
- frontend component tests using React Testing Library;
- backend API tests using Supertest;
- database integration tests using an isolated test database;

- browser end-to-end tests;
- automated accessibility checks where practical;
- shared contract and schema tests.

## ## Acceptance Criteria

- [ ] Unit, frontend, backend and contract tests run successfully from the repository root.
- [ ] An isolated PostgreSQL test database is configured.
- [ ] Test database migrations can run automatically.
- [ ] The test database can be reset and seeded safely.
- [ ] Tests never connect to the production database.
- [ ] Reusable test fixtures or builders exist for the main domain entities.
- [ ] Firebase authentication can be tested without using real user accounts.
- [ ] A browser end-to-end test runner is configured.
- [ ] At least one browser smoke test runs successfully.
- [ ] At least one frontend component test demonstrates the approved pattern.
- [ ] At least one backend API integration test demonstrates the approved pattern.
- [ ] At least one database integration test demonstrates commit and rollback behaviour.
- [ ] Test coverage can be generated using a documented command.
- [ ] Gitea Actions runs the required automated tests.
- [ ] Failed tests cause the CI workflow to fail.
- [ ] Test credentials and secrets are not committed.
- [ ] Testing documentation explains how and where to write each test type.
- [ ] `npm run check` passes.
- [ ] The work is reviewed and merged through a Pull Request.

## ## Suggested Commands

Add or confirm commands similar to:

```
```bash
npm test
npm run test:unit
npm run test:frontend
npm run test:api
npm run test:database
npm run test:e2e
```



```
npm run test:coverage  
npm run check
```

The final command names may follow the repository's existing naming conventions.

Dependencies

- Existing monorepo and workspace setup.
- Existing Vitest, React Testing Library and Supertest configuration.
- PostgreSQL development environment.
- Firebase authentication foundation.
- Gitea Actions CI workflow.
- Database migration and seed tooling.

Where a dependency is not complete, coordinate with the related issue rather than duplicating its implementation.

Testing and Verification

Verify the foundation by running:

1. A frontend component test.
2. A backend API test.
3. A database integration test.
4. A shared contract or schema test.
5. A browser smoke test.
6. The full root test command.
7. `npm run check`.
8. The same test workflow through Gitea Actions.

All tests must be repeatable on another developer's machine using the documented setup instructions.

Documentation and Evidence

Provide:

- testing setup documentation;
- required environment-variable examples;
- test database instructions;
- example tests for each configured layer;
- local test output;
- test coverage output;
- a passing Gitea Actions run;
- the linked Pull Request.

Security, Accessibility and Reliability

Security

- Never use production credentials in tests.
- Never run destructive test cleanup against development or production databases.
- Do not commit Firebase service credentials, database passwords or access tokens.
- Use clearly separated test environment variables.

Accessibility

The browser and component testing setup should support automated accessibility checks for relevant frontend pages and components.

Reliability

Tests must be isolated, deterministic and repeatable. A test should not depend on execution order or data left behind by another test.

Definition of Done

This issue is complete when the team has a documented and repeatable testing environment that supports frontend, backend, database, contract and browser tests locally and in Gitea Actions.

The separate Basic vertical-slice testing issue will use this foundation to test the complete administrator, submitter, event-submission, statistic-derivation and public-viewing workflow.

Project and Git Methodology

- Keep this issue in **Backlog** until it is assigned and its dependencies are understood.
- Move it to **Ready** once it satisfies the Definition of Ready.
- Create the branch from the latest **main** using:

```
test/<issue-number>-testing-foundation
```

- Use small, meaningful commits following the approved Conventional Commit format.
- Reference the issue in commit bodies using:

```
Refs #<issue-number>
```

- Run all relevant tests and **npm run check** before opening the Pull Request.
- Move the issue to **In Review** when the Pull Request is opened.
- The Pull Request must include testing performed, evidence, known limitations and:

```
Closes #<issue-number>
```

- Move the issue to **Done** only after review, passing CI and merge.
- Record relevant AI-assisted work in the repository AI usage register.
- Include the required **Assisted-by** trailer in commits where AI generated or substantially modified code.

AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

8/6/2026, 4:31:14 PM



give me the whole markdown as one thing, u got cut off midway

8/6/2026, 4:33:14 PM



Copy everything inside this single box into the Gitea issue description:

Description

Complete the shared automated testing foundation for the frontend, backend API, database and browser-level workflows.

The repository already uses Vitest, React Testing Library and Supertest. This issue must complete the supporting test environment so that feature teams can write reliable tests and Gitea Actions can run them consistently.

This issue sets up testing infrastructure. Tests for individual features must still be included in their own implementation issues.

Scope

Included

- Confirm the test structure for unit, component, API integration and end-to-end tests.
- Configure an isolated PostgreSQL test database.
- Add commands to migrate, reset and seed the test database safely.
- Create reusable test fixtures, builders or helper functions.
- Configure test environment variables without committing secrets.
- Configure Firebase authentication testing using an emulator or approved test doubles.
- Configure browser end-to-end testing using Playwright or the team's approved alternative.
- Add test coverage reporting.
- Ensure frontend, backend, database and shared-contract tests run from the repository root.
- Ensure the testing commands can run in Gitea Actions.
- Document where each type of test belongs and how developers run it.

- Add at least one working example for each configured testing layer.

Excluded

- Writing every feature test for the project.
- Full Basic workflow testing.
- Intermediate-scale performance testing.
- Formal user testing.
- Final accessibility and security audits.
- Testing against the production database or production Firebase project.

Required Testing Layers

The testing foundation must support:

- unit tests for statistic calculations and utility functions;
- frontend component tests using React Testing Library;
- backend API tests using Supertest;
- database integration tests using an isolated test database;
- browser end-to-end tests;
- automated accessibility checks where practical;
- shared contract and schema tests.

Acceptance Criteria

- [] Unit, frontend, backend and contract tests run successfully from the repository root.
- [] An isolated PostgreSQL test database is configured.
- [] Test database migrations can run automatically.
- [] The test database can be reset and seeded safely.
- [] Tests never connect to the production database.
- [] Reusable test fixtures or builders exist for the main domain entities.
- [] Firebase authentication can be tested without using real user accounts.
- [] A browser end-to-end test runner is configured.
- [] At least one browser smoke test runs successfully.
- [] At least one frontend component test demonstrates the approved testing pattern.
- [] At least one backend API integration test demonstrates the approved testing pattern.
- [] At least one database integration test demonstrates commit and rollback behaviour.
- [] At least one shared contract or schema test demonstrates the approved

testing pattern.

- [] Test coverage can be generated using a documented command.
- [] Gitea Actions runs the required automated tests.
- [] Failed tests cause the CI workflow to fail.
- [] Test credentials and secrets are not committed.
- [] Testing documentation explains how and where to write each test type.
- [] `npm run check` passes.
- [] The work is reviewed and merged through a Pull Request.

Suggested Commands

Add or confirm commands similar to:

```
```bash
npm test
npm run test:unit
npm run test:frontend
npm run test:api
npm run test:database
npm run test:e2e
npm run test:coverage
npm run check
```
```

The final command names may follow the repository's existing naming conventions.

Dependencies

- Existing monorepo and workspace setup.
- Existing Vitest, React Testing Library and Supertest configuration.
- PostgreSQL development environment.
- Firebase authentication foundation.
- Gitea Actions CI workflow.
- Database migration and seed tooling.

Where a dependency is not complete, coordinate with the related issue rather than duplicating its implementation.

Testing and Verification

Verify the testing foundation by running:

1. A frontend component test.
2. A backend API test.
3. A database integration test.
4. A shared contract or schema test.
5. A browser smoke test.
6. The full root test command.
7. `npm run check`.
8. The same test workflow through Gitea Actions.

All tests must be repeatable on another developer's machine using the documented setup instructions.

Documentation and Evidence

Provide:

- testing setup documentation;
- required environment-variable examples;
- test database instructions;
- example tests for each configured layer;
- local test output;
- test coverage output;
- a passing Gitea Actions run;
- the linked Pull Request.

Security, Accessibility and Reliability

Security

- Never use production credentials in tests.
- Never run destructive test cleanup against development or production databases.
- Do not commit Firebase service credentials, database passwords or access tokens.
- Use clearly separated test environment variables.
- Ensure test logs do not expose secrets or authentication tokens.

Accessibility

The browser and component testing setup should support automated accessibility checks for relevant frontend pages and components.

Reliability

Tests must be:

- isolated;
- deterministic;
- repeatable;
- independent of execution order;
- responsible for cleaning up their own test data.

A test must not depend on data left behind by another test.

Definition of Done

This issue is complete when the team has a documented and repeatable testing environment that supports frontend, backend, database, contract and browser tests locally and in Gitea Actions.

The separate Basic vertical-slice testing issue will use this foundation to test the complete workflow:

```text

Administrator approves submitter

↓

Approved submitter signs in

↓

Submitter uploads fixture events

↓

Invalid events receive useful rejection messages

↓

Valid events are stored with provenance

↓

Statistics are derived from the events

↓

Public users view the statistics without signing in

```

Project and Git Methodology

- Keep this issue in `Backlog` until it is assigned and its dependencies are understood.

- Move it to `Ready` once it satisfies the team's Definition of Ready.
- Create the branch from the latest `main` using:

```
```text
test/<issue-number>-testing-foundation
```
```

- Use small, meaningful commits following the approved Conventional Commit format.
- Reference the issue in commit bodies using:

```
```text
Refs #<issue-number>
```
```

- Run all relevant tests and `npm run check` before opening the Pull Request.
- Move the issue to `In Review` when the Pull Request is opened.
- The Pull Request must include:
 - a summary of the testing infrastructure added;
 - testing performed;
 - evidence of passing local tests;
 - evidence of passing Gitea Actions;
 - known limitations;
 - documentation changes;
 - `Closes #<issue-number>`.
- Move the issue to `Done` only after:
 - review approval;
 - passing CI;
 - required changes are addressed;
 - the Pull Request is merged.
- Record relevant AI-assisted work in the repository AI usage register.
- Include the required `Assisted-by` trailer in commits where AI generated or substantially modified code.

AI Declaration

> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

8/6/2026, 4:33:20 PM



e90173a7-f0ba-4627-9ab3-d3...



bcead505-231c-40f9-9fbc-b1e...

ok, should we maybe pause here and document all weve done for sprint 1 b4 doing next? and roadmap whatever that is? will i need to branch and do docs etc? here issue numbers

Add Cricsheet T20 dataset downloader #28

Design and implement the delivery event schema #27

Merge confict for DB #21

Restore green CI and reproducible installation #10

Validate developer onboarding and motivate the technology stack #12

Establish the Azure backend deployment foundation #15

Establish Azure frontend deployment foundation #17

Deploy MkDocs documentation to Cloudflare Pages using Wrangler #29

Create per-team-member folders for AI transcripts #30

Investigate and establish the OAuth/OIDC authentication foundation #14

Establish the Supabase PostgreSQL foundation #13

Deploy the public documentation site with Cloudflare Pages #11

Protect main and require Pull Request approval #8

Align repository governance and document initial setup deviation #7

Add Git and project methodology documentation #2

Complete the automated testing foundation and test environment setup #72

Backlog planning = #36

Sport and domain definition = #37

Produce the complete system architecture, data flow and development roadmap = #38

Build frontend authentication pages and navigation controls #39



Thinking



Inspecting repository structure and documentation needs

- **Documenting sprint planning**

I'm recommending a brief checkpoint now: refine Sprint 1 evidence, assign and select work, and avoid creating more issues. I'll keep backlog traceability and architecture roadmap documentation separate, with each change tied to its existing issue.

- **Documented sprint planning**

- **Organizing the next checkpoint**

I'm keeping this repository-based, so I don't need external research. I'm suggesting a documentation checkpoint: refine Sprint 1, record decisions, and leave #36 open until the complete backlog is actually finished.

- **Organized the next checkpoint**



Reviewing Architecture Documentation Structure

- **Inspected repository documentation, methodology, and architecture structure**

✓ Worked for 2m 2s

Yes — **pause creating more issues now** and do a short Sprint 1 planning/documentation checkpoint.

You have created the Sprint 1 backlog, but you have **not completed Sprint 1 work yet**. What you should document now is:

- what issues already existed;
- what new Sprint 1 issues were created;
- which issues overlap;
- the Sprint 1 goal;

- the order and dependencies of the work;
- who will start which issue;
- what the system should demonstrate by the end of Sprint 1.

This is exactly what your approved Scrumban methodology expects before active development.

First: resolve obvious overlaps

Before assigning everything, check these:

#27 and #49

- #27 Design and implement the delivery event schema
- #49 Define the executable sport-specific event schema and example submissions

These appear to cover nearly the same work.

Because #27 existed first, I recommend:

1. Keep #27.
2. Copy any useful acceptance criteria from #49 into #27.
3. Comment on #49 :

This work is already covered by #27.

The additional requirements from this issue will be incorporated into #27 to avoid maintaining duplicate event-schema issues.

Duplicate of #27.

4. Close #49 as a duplicate.

#11 and #29

- #11 Deploy the public documentation site with Cloudflare Pages
- #29 Deploy MkDocs documentation to Cloudflare Pages using Wrangler

These may be duplicates, but they could also be separated as:

- **#11** : initial public documentation deployment;
- **#29** : automate the Wrangler/Gitea deployment workflow.

Check their descriptions. Keep both only when they have clearly different outputs.

The rest of your issues mostly look like foundations followed by actual feature implementation rather than duplicates.

What “the roadmap” means

The roadmap is not another enormous issue list.

It is a simple explanation of the intended outcome for each milestone:

| Milestone | Intended outcome |
|------------------|--|
| Sprint 1 | A working event-to-statistic vertical slice |
| Sprint 2 | Complete Basic and Intermediate capabilities |
| Sprint 3 | Complete Advanced feature implementation |
| Final Submission | Fixes, hardening, documentation and presentation |

For Sprint 1, the roadmap should show this path:

```
Confirm T20 cricket domain
    ↓
Define delivery-event schema
    ↓
Create database structures
    ↓
Set up authentication and roles
    ↓
Approve submitters
    ↓
Submit and validate delivery events
    ↓
Store events with provenance
```

```
↓  
Calculate cricket statistics  
↓  
Expose events and statistics publicly  
↓  
Display them on the frontend  
↓  
Test the complete workflow
```

It tells the group what must happen first and what can happen in parallel.

Yes, use branches for the repository documents

Creating issues in Gitea does **not** require a branch.

Adding planning, architecture or evidence documents to the repository **does require a branch and Pull Request**.

Use separate branches for `#36` and `#38`. Do not combine everything into one giant documentation branch.

Branch 1: Sprint 1 backlog and planning

This work relates to:

```
#36 Create and prioritise the complete project backlog
```

Create:

```
git switch main  
git pull origin main  
git switch -c docs/36-sprint1-backlog-plan
```

Add these files:

```
docs/planning/project-backlog.md  
evidence/sprints/sprint-1/2026-08-06-planning.md
```

You will also need to add the planning page to `mkdocs.yml`.

`project-backlog.md` **should contain**

Do not copy every full issue description into it. Gitea remains the source of truth.

Include a compact table such as:

```
Issue	Work	Area	Current state	Key dependency
#37	Confirm sport and domain	Data	Backlog	None
#38	System architecture and roadmap	Documentation	Backlog	#37 partly
#39	Authentication pages	Frontend	Backlog	#14
#43	Account and role schema	Database	Backlog	#13, #37
```

Also include sections for:

- existing foundation issues;
- newly created Sprint 1 issues;
- duplicate or merged issues;
- Sprint 1 dependency order;
- future Sprint 2 and Sprint 3 backlog still to be added.

Because you have not created the entire later backlog yet, **do not close #36** after this first document is merged. Keep it `In Progress`.

The Pull Request should say:

```
Refs #36
```

not:

```
Closes #36
```

You will close `#36` only after the remaining project backlog has been created later.

Branch 2: Architecture and roadmap

This work relates to:

```
#38 Produce the complete system architecture, data flow and development roadmap
```

Create a separate branch:


```
git switch main
git pull origin main
git switch -c docs/38-system-architecture-roadmap
```

Your repo already has:

```
docs/architecture/overview.md
```

Do not create a second architecture document that contradicts it. Expand the existing file.

Also create:

```
docs/planning/roadmap.md
```

Update `mkdocs.yml` so the roadmap appears on the public documentation site.

The architecture document should show:

```
Public visitor
  ↓
React frontend
  ↓
Handwritten backend API
  ↓
PostgreSQL
```

and:

```
Firebase identity
  ↓
Application account and role
  ↓
Approved submitter scope
```

and:

```
Cricket delivery submission
  ↓
Validation
  ↓
```

Ordered event storage
↓
Derived statistics
↓
Public API and frontend

When the architecture and roadmap satisfy [#38](#), the Pull Request can contain:

Closes [#38](#)

Create the Sprint 1 planning record

Your repository already expects one directory per sprint under:

evidence/sprints/

Create:

evidence/sprints/sprint-1/2026-08-06-planning.md

It can be concise:

Sprint 1 Planning

****Sprint:**** Sprint 1

****Date:**** 6 August 2026

****Attendees:**** [names]

Sprint Goal

Deliver a deployed working vertical slice in which an approved submitter can submit validated T20 cricket delivery events, the platform derives fixture statistics, and public users can view fixtures, events and statistics without signing in.

Selected Work

- Domain and event model: [#37](#), [#27](#), [#28](#)
- Architecture and contracts: [#38](#), [#55](#), [#58](#), [#59](#)
- Database foundation: [#13](#), [#43](#), [#46](#), [#60](#), [#61](#), [#62](#)
- Authentication and access: [#14](#), [#39](#), [#44](#), [#45](#), [#64](#), [#65](#), [#66](#)

- Submission and derivation: #50, #51, #52, #53
- Public access: #47, #48, #54, #67
- Testing: #63, #68, #72
- Deployment and documentation: #10, #15, #17, #29, #69, #70

Key Risks and Dependencies

- The T20 cricket domain and event vocabulary must be confirmed before finalising database and derivation work.
- #27 and #49 overlap and must be consolidated.
- Authentication UI depends on the Firebase foundation in #14.
- Database feature work depends on migration tooling in #60.
- Full workflow testing depends on #72.

Decisions

- Published fixtures, events and statistics will be publicly viewable without sign-in.
- Authentication is required for account management.
- Only approved submitters may upload delivery-event data.
- Submitters upload events, not final statistic totals.

Unresolved Questions

- Exact administrator bootstrap process.
- Final submitter-scope model.
- Whether #11 and #29 overlap.
- Which Cricsheet T20 competition will be used for the reference dataset.

Only include attendee names and decisions that are genuine.

Update issue #36 now

Add a comment to #36 before starting the branch:

Sprint 1 Backlog Checkpoint

The initial Sprint 1 backlog has been created.

Existing foundation issues

- #2 Git and project methodology

- #7 Repository governance
- #8 Main branch protection
- #10 CI and reproducible installation
- #11 Public documentation deployment
- #12 Developer onboarding and technology stack
- #13 PostgreSQL foundation
- #14 Authentication foundation
- #15 Azure backend foundation
- #17 Azure frontend foundation
- #27 Delivery event schema
- #28 Cricsheet T20 dataset downloader
- #29 MkDocs deployment using Wrangler
- #30 AI transcript folders

Planning and implementation issues

- #37 Sport and domain definition
- #38 System architecture and roadmap
- #39 Frontend authentication pages
- #43-#72 Sprint 1 implementation, testing, deployment and close-out work

Backlog actions

- [] Resolve overlap between #27 and #49.
- [] Check whether #11 and #29 are separate or duplicate.
- [] Create the Sprint 1 planning record.
- [] Create the repository backlog index.
- [] Assign initial owners.
- [] Move dependency-ready work to Ready.
- [] Add later-sprint backlog items before closing this issue.

The Gitea issues and Project board remain the authoritative live work tracker.

Development does not need to stop

One person can work on documentation for #36 .

Another can work on architecture and roadmap for #38 .

Other members can already begin dependency-safe work such as:

- #60 database migrations and seed tooling;

- #72 testing foundation;
- #56 user journeys and wireframes;
- #39 frontend authentication, provided #14 is sufficiently ready;
- #58 shared API contracts.

So yes: **pause issue creation, not development.** Do a short planning and duplicate-cleanup checkpoint, merge the planning evidence, then begin implementation with a clear dependency order.

8/6/2026, 4:35:55 PM



Pasted text.txt

49:

Description

Define an executable, version-controlled schema for the selected sport's fixture events.

The schema must describe the event envelope, allowed event types, required and optional fields, event order and sport-specific values. Include valid and invalid example submissions.

Motivation

Every accepted submission must be checked against one shared event contract before events are stored or used to derive statistics.

Scope

Included

Submission and event envelope structure.

Stable fixture, competitor and participant references.

Sequence or ordering fields.

Allowed event types.

Required and optional fields per event type.

Schema version.

Valid and invalid JSON examples.

Documentation mapping events to affected statistics.

Excluded

Persisting events.

Upload user interface.

Full sport-rule validation that cannot be expressed in the schema.

Live-feed transport.

Acceptance Criteria

The schema is executable by the backend validation library.

Every initial Basic event type is defined.

Required and optional fields are unambiguous.

Event ordering is represented.

Schema versioning is included.
Valid examples pass schema validation.
Invalid examples fail for the expected reasons.
Event-to-statistic mappings are documented.
Schema tests are included.
The implementation is reviewed through a Pull Request.

Dependencies
Confirmed sport, event vocabulary and required statistics.
Shared stable identifier conventions.

Testing and Verification
Run automated validation against each valid and invalid example and confirm the expected result.

Documentation and Evidence
Provide the schema, examples, automated tests, event documentation and the linked Pull Request.

Security, Accessibility and Responsiveness

Security
Set reasonable field lengths and reject unexpected structures to reduce malformed input risk.

Accessibility
Not directly applicable to the schema.

Responsiveness
Not directly applicable to the schema.

AI Declaration
The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

27:
(attached)

27 is done, so should i rather edit 49? or is all that work done already in 27?

8/6/2026, 4:40:24 PM



Thinking

- **Narrowed Issue 49 and retained it alongside Issue 27**

✓ Worked for 29s

Do **not** close #49 as a duplicate.

#27 and #49 are related, but they are meant to produce **different types of schema**:

- **#27**: the PostgreSQL relational database schema: tables, keys, corrections, submissions, wickets, fielders, extras, migrations and indexes.
- **#49**: the executable submission contract used to validate incoming JSON before it reaches the database.

Issue #27 explicitly focuses on the relational event model and excludes ingestion tooling and the submission API. `Pasted text.txt`

So the correct flow is:

```
Incoming JSON
  ↓
#49 Executable submission schema
  ↓
#50 Validation pipeline
  ↓
#27 Database event schema
```

What #27 has already completed

#27 appears to cover:

- delivery identity;
- ordering;
- correction and superseding model;
- fixture, innings, team and person entities;
- submissions and provenance;
- extras, wickets and fielders;

- migrations and indexes;
- real Cricsheet data validation;
- database documentation and ADRs.

You should not repeat any of that in #49.

What #49 still needs to do

#49 should create something such as a:

Zod schema

or:

JSON Schema

that validates a request like:

```
{
  "schemaVersion": "1.0",
  "fixtureId": "fixture-123",
  "deliveries": [
    {
      "innings": 1,
      "over": 5,
      "position": 3,
      "sequence": 34,
      "batterId": "person-1",
      "bowlerId": "person-2",
      "runs": {
        "batter": 4,
        "extras": 0,
        "total": 4
      }
    }
  ]
}
```

It should answer:

- What JSON fields may a submitter send?

- Which fields are required?
- Which values are valid?
- How is the schema version represented?
- How are events ordered?
- How do submitted IDs map to the database entities from #27?
- What do valid and invalid example files look like?

Edit #49

Rename it to:

```
Define the executable delivery submission schema and validation examples
```

Keep these labels:

```
type: feature
area: data
area: api
area: documentation
tier: basic
priority: high
```

Use this replacement description:

Description

Define and implement the executable submission schema used to validate incoming T20 cricket delivery-event payloads before they are stored in the relational event model created by #27.

This issue covers the external JSON contract accepted by the backend. It does not redesign or recreate the PostgreSQL schema.

The schema may be implemented using Zod, JSON Schema or the team's approved validation library.

Scope

Included

- Submission envelope structure.
- Schema-version field.
- Fixture reference.
- Ordered delivery-event array.
- Stable competitor and participant references.
- Innings, over, position and sequence fields.
- Runs, extras, wickets, fielders and other supported delivery information.
- Required and optional fields.
- Allowed values and reasonable field limits.
- Valid JSON examples.
- Invalid JSON examples.
- Mapping between submission fields and the relational entities created by #27.
- Mapping between event fields and the statistics they affect.
- Automated schema-validation tests.

Excluded

- Recreating the relational database schema from #27.
- Database migrations.
- Persisting submitted events.
- Submitter authentication and authorisation.
- The submission API endpoint.
- Full sport-rule validation that requires database queries.
- Statistic derivation.
- Live-feed transport.

Acceptance Criteria

- [] An executable, version-controlled delivery-submission schema exists.
- [] The schema uses the stable identifiers and ordering model approved in #27.
- [] The submission envelope includes a schema version.
- [] Every initial Basic delivery-event field is defined.
- [] Required and optional fields are unambiguous.
- [] Runs, extras, wickets and fielders supported by #27 can be represented.
- [] Delivery ordering is represented without relying only on printed ball numbers.
- [] Valid example submissions pass automated validation.
- [] Invalid examples fail for the expected fields and reasons.
- [] Unexpected fields and unreasonable input lengths are rejected where appropriate.

- [] Submission fields are mapped to the relevant database entities from #27.
- [] Event fields are mapped to the statistics they affect.
- [] Schema tests run through the repository test commands.
- [] API and event-schema documentation is updated.
- [] `npm run check` passes.
- [] The implementation is reviewed and merged through a Pull Request.

Dependencies

- #27 Design and implement the delivery event schema
- #37 Confirm the sport, professional competition scope, event vocabulary and required statistics
- #58 Define stable identifiers and shared API response, error, filtering, sorting and pagination contracts

This issue blocks:

- #50 Implement event validation with detailed rejection messages
- #51 Implement approved-submitter event submission with provenance and scope enforcement

Testing and Verification

Run the executable schema against:

1. A valid standard delivery.
2. A valid wide or no-ball.
3. A delivery containing multiple extras where supported.
4. A wicket with one or more fielders.
5. A delivery from a super over.
6. Missing required fields.
7. Invalid identifiers.
8. Invalid field types.
9. Duplicate or invalid ordering information.
10. Unexpected fields and oversized values.

Each invalid example must fail for the expected reason.

Documentation and Evidence

Provide:

- the executable schema;
- valid and invalid JSON examples;
- automated schema tests;
- submission-to-database field mapping;
- event-to-statistic mapping;
- updated documentation;
- the linked Pull Request.

Project and Git Methodology

- Keep the issue in `Backlog` until its dependencies are ready and an assignee is selected.
- Create a branch from the latest `main` using:

```
```text
feat/49-delivery-submission-schema
```
```

- Use small Conventional Commits.
- Reference this issue using `Refs #49`.
- Run the relevant tests and `npm run check`.
- Move the issue to `In Review` when its Pull Request opens.
- Close it through the merged Pull Request using `Closes #49`.
- Record any AI-assisted implementation in the AI usage register and commit trailers where required.

AI Declaration

> *The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].*

Add this comment to #49 after editing:

This issue has been refined to avoid duplicating #27.

Issue #27 owns the relational PostgreSQL event model, migrations, correction model and database-level provenance. This issue owns only the executable incoming delivery-submission contract and its validation examples.

So: #27 remains done, and #49 remains open with a narrower API/data-contract scope.

8/6/2026, 4:40:26 PM



Pasted markdown(2).md

ok, now dodoc with me slowly 1 step at a time.. #38 says its been completed by a team member, so may need to add/edit to his stuff, lemme send it

8/6/2026, 4:44:33 PM



Thinking

- **Reviewing document status**

I'm assessing #38 as an architecture document, and it's strong overall. I'm spotting two important fixes: align the roadmap with completing Basic, Intermediate, and Advanced by Sprint 3, and reconcile Firebase references with the Supabase Auth decision before calling it complete.

- **Reviewed document status**

✓ Worked for 26s

The architecture document is **very strong and mostly completes #38**, but it should **not be approved or closed yet**.

Two important things must be corrected first:

1. **The roadmap does not match your agreed target.**

It currently plans:

- Sprint 1: foundations
- Sprint 2: Basic
- Sprint 3: Intermediate
- Final Submission: optional Advanced stretch

Your team's target is:

- Sprint 1: working Basic vertical slice

- Sprint 2: complete Basic and Intermediate
- Sprint 3: Advanced feature completion
- Final Submission: hardening, evidence and release

2. The authentication provider must be reconciled.

The document says Firebase was superseded and the repository now uses **Supabase Auth**. Many of the new issues we created still mention Firebase, so those issue descriptions must eventually be updated to Supabase Auth or provider-neutral wording. The architecture document correctly identifies this as an unresolved decision. `Pasted markdown.md`

Also, the final AI declaration says the document still needs group review and a Pull Request, so the issue is not fully complete until those happen. `Pasted markdown.md`

Step 1: comment on #38 or its Pull Request

Do not create your own branch yet because another team member owns this work. Paste this review comment:

```
## Architecture review feedback
```

```
This is a strong and comprehensive architecture document. Before #38 is approved and closed, please address the following:
```

```
### 1. Align the roadmap with the team's delivery target
```

```
The current roadmap schedules Basic for Sprint 2, Intermediate for Sprint 3 and Advanced as an optional Final Submission stretch.
```

```
Our agreed target is:
```

- **Sprint 1:** Deliver a working Basic vertical slice in which an approved submitter can submit validated T20 delivery events, statistics are derived from those events, and public users can view fixtures, events and statistics without signing in.
- **Sprint 2:** Complete all remaining Basic requirements and the Intermediate batch, review, aggregate, performance, API-consumer and dataset-release requirements.
- **Sprint 3:** Complete the Advanced custom-statistic, live-feed, temporal-query, release-comparison, asynchronous-job, change-feed, compatibility,

anomaly-detection and submitter-reconciliation requirements.

- ****Final Submission:**** Reserve for defect resolution, hardening, production verification, documentation, evidence, reports, presentation and release packaging.

Please update the roadmap and exit evidence accordingly.

2. Reconcile the authentication provider

The document states that ADR-004 superseded Firebase and that the current implementation uses Supabase Auth.

Please confirm whether Supabase Auth is the final approved provider. If it is, the dependent authentication issues and documentation must be updated from Firebase-specific wording to Supabase Auth or provider-neutral wording.

If stakeholder or lecturer confirmation is still required, keep this clearly recorded as a blocking decision before production authentication and role implementation.

3. Complete the issue review requirements

Before closing #38:

- [] All six group members must review or approve the architecture.
- [] Any rejected feedback must be addressed or rejected with motivation.
- [] The roadmap changes must be incorporated.
- [] Authentication terminology must be made consistent.
- [] The documentation site must build successfully.
- [] The Pull Request must pass CI and be merged.

Once these changes are complete, #38 can be closed through the Pull Request.

Do that first. Do not close #38 yet.

8/6/2026, 4:44:36 PM



its already been closed, must i ope it with a comment, assign myself to it too, and make edits or what?

8/6/2026, 4:46:35 PM



Thinking

- **Choosing next steps**

I'd leave #38 closed and avoid assigning yourself to it. First, add a comment explaining the roadmap mismatch and link a new follow-up issue. Then assign yourself to that new issue before editing.

- **Chose next steps**

- **Planning the correction**

I'd keep #38 closed, avoid assigning yourself there, and create a follow-up issue for the roadmap and authentication corrections. Link it from #38, assign yourself to the new issue, then edit in a branch tied to that issue.

- **Planned the correction**

- **Created follow-up issues for roadmap and documentation corrections without reopening #38**

✓ Worked for 40s

Do **not** reopen #38 or assign yourself to it.

Because #38 is already closed, treat it as the historical record of the original architecture work. Create a **new follow-up issue** for the corrections. That keeps the issue history honest and lets your changes go through their own branch and Pull Request.

The document currently places the Basic vertical slice in Sprint 2, Intermediate work in Sprint 3 and Advanced work as optional at Final Submission. It also records an unresolved Firebase-versus-Supabase Auth decision. `Pasted markdown.md`

Step 1 only

Create this issue:

Title

Align the architecture roadmap and authentication terminology

Template: Task

Labels

type: documentation
area: documentation
area: infrastructure
area: api
tier: common
priority: high

Milestone: Sprint 1

Project column: Backlog

Assignee: Assign yourself only when you intend to make the edits.

Do not edit the document yet. Create the issue and send me its number. Then I'll give you the short description to paste and the exact next step.

8/6/2026, 4:47:01 PM



#73

8/6/2026, 4:48:55 PM



Great - record:

Architecture follow-up = #73

Paste this into issue **#73**:

Description

Update the existing architecture and roadmap documentation created under #38 so that it matches the team's agreed delivery target and uses consistent authentication terminology.

The architecture document currently schedules the Basic vertical slice for Sprint 2, Intermediate work for Sprint 3 and Advanced work as optional Final Submission scope. It also records a conflict between Firebase Authentication and the repository's current Supabase Auth implementation. `Pasted markdown.md`

Update:

```
```text
docs/architecture/system-architecture.md
```
```

Required changes

- Change the roadmap to target:
 - Sprint 1: working Basic event-to-statistic vertical slice;
 - Sprint 2: complete remaining Basic and Intermediate requirements;
 - Sprint 3: Advanced feature completion;
 - Final Submission: hardening, evidence, reports, presentation and release packaging.
- Update the milestone exit evidence to match these targets.
- Confirm Supabase Auth as the current implementation where supported by the repository.

- Use provider-neutral wording where the final provider still requires stakeholder or lecturer approval.
- Clearly record any remaining authentication-provider decision as a risk or blocker.
- Ensure related authentication issues and documentation can be updated consistently afterward.
- Keep planned functionality clearly distinguished from implemented functionality.

Acceptance Criteria

- [] The Sprint 1 roadmap targets a working Basic vertical slice.
- [] The Sprint 2 roadmap targets complete Basic and Intermediate functionality.
- [] The Sprint 3 roadmap targets Advanced feature completion.
- [] Final Submission is reserved mainly for hardening and submission preparation.
- [] Authentication terminology is consistent throughout the document.
- [] Supabase Auth is not presented as formally approved unless supporting evidence exists.
- [] Remaining authentication-provider uncertainty is clearly recorded.
- [] Implemented and planned components remain clearly distinguished.
- [] The documentation site builds successfully.
- [] `npm run check` passes.
- [] The changes are reviewed and merged through a Pull Request.

Dependencies

- #38 Produce the complete system architecture, data flow and development roadmap
- ADR-002 and ADR-004
- Current authentication implementation and documentation

Project and Git Methodology

- Keep this issue in `Backlog` until assigned.
- Move it to `Ready` before beginning work.
- Create a branch from the latest `main` using:

```
```text
docs/73-align-architecture-roadmap
```
```

- Use a commit such as:

```
```text
docs(architecture): align roadmap and authentication terminology
```
```

- Reference this issue in the commit body:

```
```text
Refs #73
```
```

- Open a Pull Request containing:

```
```text
Closes #73
```
```

- Move the issue to `In Review` when the Pull Request opens and to `Done` only after approval, passing CI and merge.

AI Declaration

> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

For now, only paste this description into **#73**.

8/6/2026, 4:49:07 PM



ok, done, now, should we do doc on 73 or 36 first?

8/6/2026, 4:50:30 PM



Do **#73 first**, then #36.

The reason is that #36's backlog document should point to the **correct roadmap**. Right now, the architecture document still places Basic in Sprint 2 and Intermediate in Sprint 3, so documenting the backlog first could preserve that contradiction. `Pasted markdown.md`

Order

1. Complete and merge **#73**

- Edit `docs/architecture/system-architecture.md`
- Correct the Sprint 1-3 roadmap
- Reconcile the Supabase Auth terminology
- Run the documentation build and `npm run check`
- Merge through a Pull Request

2. Return to the latest `main`

3. Start **#36**

- Create `docs/planning/project-backlog.md`
- Add the Sprint 1 planning record
- Reference the now-correct architecture roadmap

Start **#73** now

Move #73:

Backlog → Ready → In Progress

Assign yourself, then create:

```
git switch main  
git pull origin main  
git switch -c docs/73-align-architecture-roadmap
```

Do not start the #36 branch until #73 is merged.

8/6/2026, 4:50:42 PM



ok, apparently we using supabase oauth now btw, whats next

8/6/2026, 4:51:47 PM



Since the team has decided on **Supabase Auth with Google OAuth**, the next step is to begin #73.

Step 1: prepare the issue and branch

In Gitea:

- Assign **#73** to yourself.
- Move it to **In Progress**.

Then, in the repository terminal:

```
git switch main
git pull origin main
git switch -c docs/73-align-architecture-roadmap
```

Do not edit anything else yet.

The documentation changes will confirm:

```
Authentication provider: Supabase Auth
Initial sign-in provider: Google OAuth
```

and remove the Firebase-versus-Supabase uncertainty from the architecture document.

Once the branch is created, send me what your terminal shows.

8/6/2026, 4:52:00 PM



ok, created branch, next

8/6/2026, 4:52:20 PM



Next, update the authentication section first.

Open:

```
docs/architecture/system-architecture.md
```

Find the section beginning:

```
### Authentication provider clarification
```

Replace everything from that heading up to, but not including:

```
## 3. High-level architecture
```

with:

```
### Authentication provider
```

The project uses **Supabase Auth** as its managed authentication provider, with **Google OAuth** as the initial sign-in method.

Supabase Auth is responsible for:

- managing the Google OAuth sign-in flow;
- issuing authenticated user sessions and access tokens;
- providing a stable provider subject for each authenticated user;
- supporting the required account lifecycle features.

The frontend sends the Supabase access token to the handwritten backend API. The backend verifies the token and then loads the user's application account, role, approved-submitter status and competition scope from PostgreSQL.

Authentication and authorisation remain separate:

- **Supabase Auth** proves the user's identity.
- **The Sport Analytics backend** decides what the user is allowed to do.

Application users are identified using the provider-neutral combination:

```
```text
(auth_provider, provider_subject)
```

This prevents authentication-provider details from becoming tightly coupled to sport, event or statistic data.

Firebase Authentication is not part of the current architecture and should not be implemented alongside Supabase Auth.

Supabase Auth is the team's selected implementation. Any outstanding stakeholder, lecturer or institutional approval required for production use must remain recorded as a project risk until confirmed.

Save the file, but **do** not commit yet.

8/6/2026, 4:52:40 PM



Sport-Analytics-Tool (4).zip

should i maybe give u the updated repo b4 we change docs?? here it is, check and then regive what im changing for 73

8/6/2026, 4:53:54 PM



## Thinking



### Inspecting repository contents from ZIP

- **Reviewing the repository**

I'm confirming the current implementation before changing documentation. For issue #73, the repository already uses Supabase Auth, so I'll adjust only the roadmap wording and clarify that authentication is already implemented—not replace the whole section.

- **Reviewed the repository**

- **Defining issue #73 changes**

I'm narrowing this to documentation alignment, not an architecture rewrite: keep Supabase Auth, clarify production approval, update sprint scope, revise risk gates, and add the related issue reference.

- **Defined changes for issue #73**

- **Inspected repository structure, architecture, authentication, ADRs, and roadmap**

✓ Worked for 1m 25s

Good call uploading the updated repo first. It changes the edit slightly.

The repository already confirms that:

- ADR-004 is accepted and supersedes Firebase.
- The backend uses `@supabase/supabase-js`.



- Tokens are verified using `supabase.auth.getUser(...)`.
- Google OAuth is the initial provider.
- The current branch is correctly named `docs/73-align-architecture-roadmap`.
- The lecturer's written Supabase approval only explicitly covered **hosted PostgreSQL**, not Supabase Auth.

So for **#73**, do **not** rewrite the diagrams or the whole authentication flow. Those are already accurate.

## What #73 needs to change

Only four areas:

1. Add `#73` to the document header.
2. Clarify that Supabase Auth is the team's selected and implemented authentication foundation, while formal course approval for using Supabase Auth specifically is not yet evidenced.
3. Rewrite the Sprint 1, 2, 3 and Final Submission roadmap.
4. Update the risks table so it no longer describes Firebase as a competing current option.

## Step 1: update the header and authentication section

Open:

```
docs/architecture/system-architecture.md
```

At the top, replace:

```
Related issue: #38
```

with:

```
Related issues: #38, #73
```

Then replace everything from:

```
Authentication provider clarification
```

up to, but not including:

### ## 3. High-level architecture

with this:

#### ### Authentication provider

The project uses **Supabase Auth** as its managed authentication foundation, with

**Google OAuth** as the initial sign-in provider.

ADR-004 supersedes the earlier Firebase decision in ADR-002. The repository now implements

Supabase token verification in the Express backend using

``supabase.auth.getUser(accessToken)``. Firebase Authentication is therefore not part of the

current runtime architecture and must not be implemented alongside Supabase Auth.

The authentication flow is:

1. The React frontend initiates the managed Google OAuth flow through Supabase Auth.
2. Supabase Auth issues a user session and access token.
3. The frontend sends the access token to the handwritten Express API using a bearer token.
4. The backend verifies the token with Supabase Auth.
5. The backend maps the verified provider subject to the application's own account, role, approval state and scoped permissions.

Authentication and authorisation remain separate:

- **Supabase Auth** proves the user's identity.
- **The Sport Analytics backend** determines what the user is allowed to do.

Application users are identified using the provider-neutral combination:

```
```text
(auth_provider, provider_subject)
```

This prevents authentication-provider details from becoming coupled to competition, event, submission or statistic data.

The written lecturer ruling currently stored in the repository explicitly approves using Supabase as hosted PostgreSQL while avoiding its generated data API. It does not explicitly

mention Supabase Auth. Supabase Auth is therefore the team's selected and implemented

authentication foundation, but any required stakeholder or lecturer confirmation for its production use must remain recorded as an open compliance decision until confirmed.

Save the file. Do not commit yet; the roadmap is the next edit.

8/6/2026, 4:53:56 PM



your markdown again cut off!!

8/6/2026, 4:59:57 PM



Copy this entire block into `docs/architecture/system-architecture.md`.

Replace the current `**Related issue:**` line and the full section from `### Authentication provider clarification` up to, but not including, `## 3. High-level architecture`.

****Related issues:**** #38, #73

Authentication provider

The project uses ****Supabase Auth**** as its managed authentication foundation, with

****Google OAuth**** as the initial sign-in provider.

ADR-004 supersedes the earlier Firebase decision in ADR-002. The repository now implements

Supabase token verification in the Express backend using ``supabase.auth.getUser(accessToken)``. Firebase Authentication is therefore not part of the current runtime architecture and must not be implemented alongside Supabase Auth.

The authentication flow is:

1. The React frontend initiates the managed Google OAuth flow through Supabase Auth.
2. Supabase Auth issues a user session and access token.
3. The frontend sends the access token to the handwritten Express API using a bearer token.
4. The backend verifies the token with Supabase Auth.
5. The backend maps the verified provider subject to the application's own account, role, approval state and scoped permissions.

Authentication and authorisation remain separate:

- **Supabase Auth** proves the user's identity.
- **The Sport Analytics backend** determines what the user is allowed to do.

Application users are identified using the provider-neutral combination:

```
```text
(auth_provider, provider_subject)
```
```

This prevents authentication-provider details from becoming coupled to competition, event, submission or statistic data.

The written lecturer ruling currently stored in the repository explicitly approves using Supabase as hosted PostgreSQL while avoiding its generated data API. It does not explicitly mention Supabase Auth.

Supabase Auth is therefore the team's selected and implemented authentication foundation.

Any required stakeholder or lecturer confirmation for its production use must remain recorded as an open compliance decision until confirmed.

8/6/2026, 5:00:05 PM



ok, next

8/6/2026, 5:00:51 PM



Thinking

- **Planning the roadmap edit**

I'm replacing only the roadmap section, keeping the existing architecture decisions intact. I'm structuring it around a prioritized vertical slice, explicit pivots, and later-stage work, without adding implementation details prematurely.

- **Planned the roadmap edit**

- **Replaced the roadmap section and updated the sprint execution plan**

✓ Worked for 30s

Next, replace only the roadmap section.

In `docs/architecture/system-architecture.md`, replace everything from:

```
## 11. Development roadmap
```

up to, but **not including**:

```
## 12. Risks and unresolved decisions
```

with this complete block. The current roadmap puts the Basic vertical slice in Sprint 2 and Advanced work after Sprint 3; this replacement aligns it with your agreed target.

Pasted markdown.md

```
## 11. Development roadmap
```

The roadmap is outcome-based and follows the project's lightweight Scrumban methodology.

Issues move into active development only when their dependencies are understood, an assignee is available, and they satisfy the Definition of Ready.

The team is targeting:

- a working Basic vertical slice during Sprint 1;
- complete Basic and Intermediate functionality by the end of Sprint 2;
- Advanced feature completion by the end of Sprint 3; and
- Final Submission work focused on hardening, evidence and release preparation.

This is an ambitious target. Basic and Intermediate requirements remain non-negotiable.

High-risk Advanced design work must begin early enough that it is not all deferred until Sprint 3.

Sprint 1 - deliver the working Basic vertical slice

****Goal:**** establish the project foundations and demonstrate the complete path from approved event submission to publicly viewable event-derived statistics.

Planned outcomes include:

- complete and prioritise the project backlog under #36;
- confirm the T20 competition scope, event vocabulary and required statistic catalogue under #37;
- maintain the agreed architecture and roadmap through #38 and #73;
- preserve the separate React frontend and handwritten Express backend;
- use Supabase Auth with Google OAuth for managed identity;
- implement application accounts, roles, approved-submitter status and scoped permissions;
- implement repeatable PostgreSQL migrations and development seed data;
- implement the approved cricket fixture and delivery-event model;
- define the executable delivery-submission contract;
- validate incoming event submissions and return actionable rejection messages;
- allow only approved, in-scope submitters to upload delivery events;
- retain submission, submitter, fixture and event provenance;
- derive the first required fixture statistics from accepted delivery events;
- expose fixtures, ordered events and derived statistics through the public API;
- allow public users to browse fixtures, events and statistics without signing in;

- establish frontend, backend, database, contract and browser testing foundations;
- repair and verify frontend and backend deployment workflows;
- maintain green CI, public documentation and AI-use evidence.

****Exit evidence:****

- a deployed public user can browse a fixture and view its ordered events and derived statistics without signing in;
- an administrator can approve a submitter and assign an appropriate scope;
- an approved submitter can sign in and submit in-scope delivery events;
- invalid submissions receive useful structured rejection messages;
- valid submissions are stored with provenance;
- required fixture statistics are derived from accepted events;
- automated tests cover the main vertical-slice behaviour;
- frontend, backend and documentation deployments are verifiable;
- architecture, database, API, testing and setup documentation reflect the implemented state;
- Sprint 1 review and retrospective evidence is recorded.

Sprint 2 - complete Basic and Intermediate functionality

****Goal:**** complete all remaining Basic requirements and extend the platform for reliable whole-season processing, aggregate statistics, API consumers and reproducible dataset releases.

Planned outcomes include:

- complete any unfinished Basic account, administration, correction, provenance and export workflows;
- complete public competition, season, fixture, competitor, participant, event and statistic interfaces;
- implement JSON and CSV file submissions through the shared validation pipeline;
- implement authorised event corrections while retaining correction history;
- automatically refresh statistics affected by accepted corrections;
- provide clear statistic provenance and "how calculated" information;

- integrate one relevant external API through an isolated adapter;
- stage and validate whole-season and back-catalogue batches before publication;
- report accepted and rejected batch records;
- make repeated batch submissions idempotent;
- resume failed batch processing from durable checkpoints;
- introduce submission review and publication workflows;
- detect impossible and conflicting data through documented validation rules;
- derive season, career and competition-wide aggregates;
- recompute only results affected by changed events;
- compare derived figures against independently verified reference results;
- create representative-scale performance data;
- define and meet documented API response-time targets;
- optimise database indexes, storage layout and query plans;
- implement explicit API versioning;
- issue, rotate and revoke API keys;
- enforce consumer rate limits and quotas;
- cache repeated reads where measurements justify caching;
- publish versioned dataset releases with schemas, field descriptions and checksums;
- complete Intermediate integration, correctness, performance and recovery tests;
- publish complete Basic and Intermediate API, database, testing and third-party code documentation.

****Exit evidence:****

- all mandatory Basic requirements are demonstrably complete;
- representative whole-season batches can be staged, validated, resumed and safely resubmitted;
- corrections retain history and update only dependent results;
- season and career aggregates match approved reference results;
- API consumers use documented versioned endpoints and managed credentials;
- representative performance targets pass;
- reproducible versioned dataset releases can be downloaded and verified;
- stakeholder and user feedback has been recorded and incorporated where appropriate.

Sprint 3 - complete Advanced functionality

****Goal:**** complete the Advanced event-processing, analyst-definition, temporal-query, compatibility and data-defence capabilities and bring the product to a near-release state.

Planned outcomes include:

- define a restricted and safe custom-statistic definition language;
- validate and version analyst-defined statistic definitions;
- execute custom definitions under resource and safety limits;
- evaluate approved custom statistics across the full event history;
- trace each result to its source events and definition version;
- propagate accepted event corrections into affected custom-statistic results;
- accept an authenticated live fixture-event feed;
- handle duplicate, late and out-of-order live events deterministically;
- replay the event pipeline and prove convergence with an equivalent ordered feed;
- support as-of-date statistic queries;
- compare two dataset releases and show what changed;
- provide aggregate API queries beyond record retrieval;
- hand large API requests to asynchronous jobs;
- allow consumers to submit jobs, inspect status and collect completed results;
- publish an event and dataset change feed for delta synchronisation;
- implement and document API deprecation and safe version retirement;
- add automated contract and backwards-compatibility tests;
- provide per-consumer API usage information;
- detect events that appear anomalous against historical data;
- reconcile conflicting submitters through an auditable resolution workflow;
- carry accepted corrections through aggregates, caches, custom statistics and dataset releases;
- complete representative-scale Advanced correctness and performance testing;
- implement production observability, structured logs, metrics and health checks;
- complete accessibility, responsiveness, security and failure-recovery reviews;
- publish complete Advanced architecture, API, analyst and operations documentation;
- run the Advanced stakeholder acceptance demonstration.

****Exit evidence:****

- approved custom statistic definitions run reproducibly without arbitrary code execution;
- a disordered live feed converges on the same result as the equivalent ordered events;
- consumers can query historical statistic state and compare dataset releases;
- large requests complete through observable asynchronous jobs;
- API compatibility and deprecation behaviour is tested;
- anomalies and conflicting submissions can be reviewed and resolved with audit evidence;
- accepted corrections reach every affected downstream result and release;
- the deployed product is feature-complete for the Advanced tier and ready for final hardening.

Final Submission - harden, evidence and release

****Goal:**** release a stable and reproducible product without introducing major new feature scope.

Planned outcomes include:

- resolve all release-blocking defects;
- complete production database migrations, backups and restore verification;
- verify production secrets, configuration and all deployed components;
- run final performance, security, accessibility, recovery and cross-browser audits;
- complete the requirements traceability matrix and limitations register;
- complete database, third-party code, licence, testing and AI-use documentation;
- prepare the reproducible demonstration dataset;
- rehearse the complete product demonstration;
- prepare the group report, individual reports, peer review and group presentation;
- reconcile the final implementation with architecture diagrams, ADRs and API documentation;
- publish release notes;
- create the final annotated version tag;
- assemble and verify the final submission package.

****Exit evidence:****

- all required automated and manual checks pass;
- the frontend, API, database, documentation and supporting services are deployed and verifiable;
- every claimed requirement links to implementation, testing and documentation evidence;
- known limitations are recorded honestly;
- backup, restore and rollback procedures have been exercised;
- the final demonstration is reproducible;
- the exact submitted commit is tagged and documented.

Save the file, but do **not** commit yet. The next edit will update the authentication row in the risks table.

8/6/2026, 5:01:07 PM



next

8/6/2026, 5:02:22 PM



Next, update the authentication risk in the table.

In `docs/architecture/system-architecture.md`, find the row in **Section 12: Risks and unresolved decisions** that begins:

```
| Authentication source conflict
```

Replace that entire row with:

```
| Supabase Auth compliance confirmation | The repository and ADR-004 use  
Supabase Auth with Google OAuth, but the written lecturer approval currently  
recorded in the repository explicitly covers Supabase-hosted PostgreSQL and  
does not separately confirm Supabase Auth. | Obtain written stakeholder or  
lecturer confirmation that Supabase Auth is acceptable for managed  
authentication. Continue using the provider-neutral application identity model  
and do not introduce Firebase alongside Supabase Auth. | Before production  
authentication is enabled or account and role functionality is considered  
complete. |
```

This removes the outdated idea that Firebase and Supabase are competing current implementations, while still honestly recording the remaining approval question.

Pasted markdown.md

Save the file, but do not commit yet.

8/6/2026, 5:02:39 PM



next

8/6/2026, 5:03:46 PM



Thinking

- **Verifying terminology consistency**

Next, I'm checking the architecture document for any remaining outdated Firebase references and confirming that the roadmap and authentication wording now match the updated direction.

- **Verified terminology consistency**

✓ Worked for 14s

Next, verify that the old wording has been removed before we run the checks.

In the terminal, run:

```
git grep -n -i "firebase" -- docs/architecture/system-architecture.md
```

The only acceptable Firebase reference should explain that **ADR-004 superseded Firebase and it is no longer part of the current architecture**. There should be no wording suggesting that Firebase is still an active option.

Then run:

```
git diff -- docs/architecture/system-architecture.md
```

Review that the diff contains only:

- **#73** added to the related issues;
- the updated Supabase Auth section;
- the revised Sprint 1-Final Submission roadmap;
- the updated Supabase Auth compliance-risk row.

Do not commit yet. Send me the output of the Firebase search, or tell me how many matches it shows.

8/6/2026, 5:03:57 PM



15013dbc-35f5-46a7-b528-f60...

8/6/2026, 5:04:36 PM



Those three matches are correct. They only state that Firebase was superseded and must not be introduced alongside Supabase Auth.

Next, check the edited Markdown for whitespace or formatting problems:

```
git diff --check
```

A successful result shows no output and returns to the prompt. Send me what it shows.

8/6/2026, 5:04:47 PM



next

8/6/2026, 5:05:04 PM



Thinking



Inspecting repository build and documentation scripts

- Inspected repository scripts and searched for MkDocs build commands

✓ Worked for 35s

Next, build the documentation strictly:

```
mkdocs build --strict
```

This checks that:

- the edited Markdown is valid;
- the Mermaid blocks are accepted;
- the navigation links still work;
- there are no broken documentation references.

A successful run should end without errors.

8/6/2026, 5:05:06 PM



7a0ba8f0-8d9a-4188-9ff9-7a2...

8/6/2026, 5:06:36 PM



Thinking



Checking Repository Documentation Configuration

- **Checked repository and requirements documentation configuration**

✓ Worked for 29s

That only means MkDocs is not installed on your computer yet.

Run this from the repository root:

```
py -m pip install -r requirements-docs.txt
```

The repository already includes `requirements-docs.txt`, which installs the required MkDocs Material version.

Once it finishes, send me the final output.

8/6/2026, 5:06:38 PM



```
+ mkdocs.cmd build --strict
+ ~~~~~~
+ CategoryInfo          : ObjectNotFound:
(mkdocs.cmd:String) [], CommandNotFoundException
+ FullyQualifiedErrorId :
CommandNotFoundException
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-
Tool> py -m pip install -r requirements-docs.txt
Collecting mkdocs-material<10,>=9 (from -r
requirements-docs.txt (line 1))
  Downloading mkdocs_material-9.7.7-py3-none-
any.whl.metadata (19 kB)
Requirement already satisfied: babel>=2.10 in
C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-
64\Lib\site-packages (from mkdocs-material<10,>=9-
>-r requirements-docs.txt (line 1)) (2.18.0)
Collecting backrefs>=5.7.post1 (from mkdocs-
material<10,>=9->-r requirements-docs.txt (line 1))
  Downloading backrefs-8.0-py314-none-
any.whl.metadata (3.0 kB)
Requirement already satisfied: colorama>=0.4 in
C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-
64\Lib\site-packages (from mkdocs-material<10,>=9-
>-r requirements-docs.txt (line 1)) (0.4.6)
Requirement already satisfied: jinja2>=3.1 in
C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-
64\Lib\site-packages (from mkdocs-material<10,>=9-
>-r requirements-docs.txt (line 1)) (3.1.6)
Collecting markdown>=3.2 (from mkdocs-
material<10,>=9->-r requirements-docs.txt (line 1))
  Downloading markdown-3.10.3-py3-none-
any.whl.metadata (5.1 kB)
Collecting mkdocs-material-extensions>=1.3 (from
mkdocs-material<10,>=9->-r requirements-docs.txt
(line 1))
  Downloading mkdocs_material_extensions-1.3.1-py3-
none-any.whl.metadata (6.9 kB)
Collecting mkdocs<2,>=1.6 (from mkdocs-
```

material<10,>=9->-r requirements-docs.txt (line 1))
 Downloading mkdocs-1.6.1-py3-none-any.whl.metadata (6.0 kB)
Collecting paginate>=0.5 (from mkdocs-material<10,>=9->-r requirements-docs.txt (line 1))
 Downloading paginate-0.5.7-py2.py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: pygments>=2.16 in C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from mkdocs-material<10,>=9->-r requirements-docs.txt (line 1)) (2.19.2)
Collecting pymdown-extensions>=10.2 (from mkdocs-material<10,>=9->-r requirements-docs.txt (line 1))
 Downloading pymdown_extensions-11.0.1-py3-none-any.whl.metadata (3.1 kB)
Requirement already satisfied: requests>=2.30 in C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from mkdocs-material<10,>=9->-r requirements-docs.txt (line 1)) (2.32.5)
Collecting click>=7.0 (from mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1))
 Downloading click-8.4.2-py3-none-any.whl.metadata (2.6 kB)
Collecting ghp-import>=1.0 (from mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1))
 Downloading ghp_import-2.1.0-py3-none-any.whl.metadata (7.2 kB)
Requirement already satisfied: markupsafe>=2.0.1 in C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1)) (3.0.3)
Collecting mergedeep>=1.3.4 (from mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1))
 Downloading mergedeep-1.3.4-py3-none-any.whl.metadata (4.3 kB)
Collecting mkdocs-get-deps>=0.2.0 (from mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1))

Downloading mkdocs_get_deps-0.2.2-py3-none-any.whl.metadata (4.0 kB)
 Requirement already satisfied: packaging>=20.5 in C:\Users\shayn\AppData\Roaming\Python\Python314\site-packages (from mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1)) (26.0)
 Collecting pathspec>=0.11.1 (from mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1))
 Downloading pathspec-1.1.1-py3-none-any.whl.metadata (14 kB)
 Collecting pyyaml-env-tag>=0.1 (from mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1))
 Downloading pyyaml_env_tag-1.1-py3-none-any.whl.metadata (5.5 kB)
 Requirement already satisfied: pyyaml>=5.1 in C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1)) (6.0.3)
 Collecting watchdog>=2.0 (from mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1))
 Downloading watchdog-6.0.0-py3-none-win_amd64.whl.metadata (44 kB)
 Requirement already satisfied: python-dateutil>=2.8.1 in C:\Users\shayn\AppData\Roaming\Python\Python314\site-packages (from ghp-import>=1.0->mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1)) (2.9.0.post0)
 Requirement already satisfied: platformdirs>=2.2.0 in C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from mkdocs-get-deps>=0.2.0->mkdocs<2,>=1.6->mkdocs-material<10,>=9->-r requirements-docs.txt (line 1)) (4.9.4)
 Requirement already satisfied: six>=1.5 in C:\Users\shayn\AppData\Roaming\Python\Python314\site-packages (from python-dateutil>=2.8.1->ghp-import>=1.0->mkdocs<2,>=1.6->mkdocs-

```

material<10,>=9->-r requirements-docs.txt (line 1))
(1.17.0)
Requirement already satisfied:
charset_normalizer<4,>=2 in
C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-
64\Lib\site-packages (from requests>=2.30->mkdocs-
material<10,>=9->-r requirements-docs.txt (line 1))
(3.4.5)
Requirement already satisfied: idna<4,>=2.5 in
C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-
64\Lib\site-packages (from requests>=2.30->mkdocs-
material<10,>=9->-r requirements-docs.txt (line 1))
(3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-
64\Lib\site-packages (from requests>=2.30->mkdocs-
material<10,>=9->-r requirements-docs.txt (line 1))
(2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in
C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-
64\Lib\site-packages (from requests>=2.30->mkdocs-
material<10,>=9->-r requirements-docs.txt (line 1))
(2026.2.25)
Downloading mkdocs_material-9.7.7-py3-none-any.whl
(9.3 MB)

9.3/9.3 MB 6.2 MB/s 0:00:01
Downloading mkdocs-1.6.1-py3-none-any.whl (3.9 MB)

3.9/3.9 MB 5.4 MB/s 0:00:00
Downloading backrefs-8.0-py314-none-any.whl (411 kB)
Downloading click-8.4.2-py3-none-any.whl (119 kB)
Downloading ghp_import-2.1.0-py3-none-any.whl (11
kB)
Downloading markdown-3.10.3-py3-none-any.whl (110
kB)
Downloading mergedeeep-1.3.4-py3-none-any.whl (6.4
kB)
Downloading mkdocs_get_deps-0.2.2-py3-none-any.whl
(9.6 kB)
Downloading mkdocs_material_extensions-1.3.1-py3-
none-any.whl (8.7 kB)

```

Downloading paginate-0.5.7-py2.py3-none-any.whl (13 kB)
Downloading pathspec-1.1.1-py3-none-any.whl (57 kB)
Downloading pymdown_extensions-11.0.1-py3-none-any.whl (269 kB)
Downloading pyyaml_env_tag-1.1-py3-none-any.whl (4.7 kB)
Downloading watchdog-6.0.0-py3-none-win_amd64.whl (79 kB)
Installing collected packages: paginate, watchdog, pyyaml-env-tag, pathspec, mkdocs-material-extensions, mergedeep, markdown, click, backrefs, pymdown-extensions, mkdocs-get-deps, ghp-import, mkdocs, mkdocs-material
Successfully installed backrefs-8.0 click-8.4.2 ghp-import-2.1.0 markdown-3.10.3 mergedeep-1.3.4 mkdocs-1.6.1 mkdocs-get-deps-0.2.2 mkdocs-material-9.7.7 mkdocs-material-extensions-1.3.1 paginate-0.5.7 pathspec-1.1.1 pymdown-extensions-11.0.1 pyyaml-env-tag-1.1 watchdog-6.0.0

[notice] A new release of pip is available: 26.0.1 -> 26.2.1

[notice] To update, run:

C:\Users\shayn\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> mkdocs build --strict

| ⚠ Warning from the Material for MkDocs team
|
| MkDocs 2.0, the underlying framework of Material for MkDocs,
| will introduce backward-incompatible changes, including:
|
| × All plugins will stop working – the plugin system has been removed
| × All theme overrides will break – the theming system has been rewritten
| × No migration path exists – existing projects cannot be upgraded

```
| x Closed contribution model – community members  
can't report bugs  
| x Currently unlicensed – unsuitable for production  
use  
|  
| Our full analysis:  
|  
| https://squidfunk.github.io/mkdocs-  
material/blog/2026/02/18/mkdocs-2.0/
```

```
INFO - Cleaning site directory  
INFO - Building documentation to directory:  
C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-  
Tool\site  
INFO - The following pages exist in the docs  
directory, but are not included in the "nav"  
configuration:  
    - environment.md  
    - adr\0003-azure-hosting.md  
    - data\cricsheet.md  
    - deployment\azure-backend.md  
    - deployment\azure-fronted.md  
    - deployment\cloudflare_pages.md  
INFO - Documentation built in 1.67 seconds  
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-  
Tool> py -m pip install -r requirements-docs.txt
```

8/6/2026, 5:08:37 PM



That worked. The documentation build passed successfully.

The MkDocs 2.0 notice and the pages-not-in-navigation list are informational, not failures. You also do not need to run the `pip install` command again.

Next step

Run the complete repository check:

```
npm run check
```

This will run the structure check, formatting, linting, type checking, tests and builds.

8/6/2026, 5:08:45 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check
```

```
> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check
&& npm run lint && npm run typecheck && npm run
test && npm run build
```

```
> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs
```

Repository structure check passed (20 required files).

```
> sport-analytics-tool@0.1.0 format:check
> prettier --check .
```

Checking formatting...

[warn] docs/architecture/system-architecture.md

[warn] docs/database/schema.md

[warn] Code style issues found in 2 files. Run Prettier with --write to fix.

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/6/2026, 5:09:17 PM



Thinking

- **Checking repository state**

I'm starting by confirming which files are currently modified, especially `docs/database/schema.md`. That tells me whether the format issue is part of the user's work or an existing repository problem before changing anything.

- **Checked repository state**

✓ Worked for 12s

The check stopped only because of formatting.

Do **not** format the whole repository yet. First check whether `docs/database/schema.md` has changes on your branch:

```
git status --short
```

Send me that output. We need to avoid accidentally including someone else's database-document changes in issue #73.

8/6/2026, 5:09:30 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short
M docs/architecture/system-architecture.md
M site/404.html
M site/ai/usage/index.html
M site/api/overview/index.html
M site/architecture/overview/index.html
M site/architecture/repository-structure/index.html
M site/database/overview/index.html
M site/database/schema/index.html
M site/deployment/azure/index.html
M site/deployment/overview/index.html
M site/development/dependencies/index.html
M site/development/setup/index.html
M site/development/testing/index.html
M site/git-methodology/index.html
M site/index.html
M site/project_methodology/index.html
M site/search/search_index.json
M site/security/overview/index.html
M site/sitemap.xml.gz
?? site/adr/
?? site/ai_policy.pdf
?? site/architecture/system-architecture/
?? site/data/
?? site/deployment/azure-backend/
?? site/deployment/azure-fronted/
?? site/deployment/cloudflare_pages/
?? site/environment/
?? site/security/auth-provider-comparison/
?? site/security/authentication/
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/6/2026, 5:10:01 PM



Thinking

- **Checking repository state**

I'm identifying the relevant architecture document and separating generated-site changes from the actual edit. I'll format only the requested file, verify its diff, and avoid touching unrelated documentation or build artifacts.

- **Checked repository state**

✓ Worked for 17s

Good — `docs/database/schema.md` is **not changed on your branch**. Its Prettier warning is unrelated to #73.

The `site/` changes were generated by `mkdocs build`; they should not be included in this documentation Pull Request.

Run these commands:

```
git restore --worktree -- site
git clean -fd -- site
```

Then format only the file you edited:

```
npx.cmd prettier --write docs/architecture/system-architecture.md
```

Finally, check the status again:

```
git status --short
```

It should ideally show only:

```
M docs/architecture/system-architecture.md
```

8/6/2026, 5:10:11 PM



yes, nect? can we commit, i have to leave in 1 minute and want to finish this

8/6/2026, 5:11:11 PM



Thinking

- **Finalizing documentation changes**

I'm checking the edited file directly, since the full formatter still reports an unrelated pre-existing issue. I also need to update the AI-use declaration, verify the diff, then commit and push once everything is clean.

- **Finalized documentation changes**

✓ Worked for 21s

Yes. First add this sentence to the **AI Declaration at the bottom** of

`docs/architecture/system-architecture.md`:

The roadmap and authentication terminology were later reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Then run:

```
npx.cmd prettier --write docs/architecture/system-architecture.md
git diff --check
git add docs/architecture/system-architecture.md
git commit -m "docs(architecture): align roadmap and authentication terminology" -m "Refs #73"
git push -u origin docs/73-align-architecture-roadmap
```

Then open the Gitea link shown after the push and create the Pull Request with:

Summary

- aligns the roadmap with the Sprint 1 Basic vertical slice target;
- targets complete Basic and Intermediate functionality by Sprint 2;

- targets Advanced feature completion by Sprint 3;
- reserves Final Submission for hardening and release preparation;
- confirms Supabase Auth with Google OAuth as the current implementation;
- records the outstanding Supabase Auth approval question accurately.

Verification

- `mkdocs build --strict` passed.
- `git diff --check` passed.
- The full `npm run check` currently stops on a pre-existing Prettier warning in `docs/database/schema.md`, which is outside this issue.

Closes #73

AI Declaration

The Pull Request was prepared with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Move #73 to **In Review** after opening the Pull Request.

8/6/2026, 5:11:27 PM



write proper, full pull request acc to methodology pls

8/6/2026, 5:13:28 PM



Pull Request title

docs(architecture): align roadmap and authentication terminology

Copy everything below into the Pull Request description:

Related Issue

Closes #73

Purpose

Update the system architecture documentation so that it reflects the team's agreed project delivery targets and the authentication approach currently implemented in the repository.

The original architecture document was completed under #38. This follow-up corrects the milestone roadmap and reconciles the authentication terminology with ADR-004 and the existing Supabase Auth implementation.

Summary of Changes

This Pull Request updates:

```
```text
docs/architecture/system-architecture.md
```
```

The following changes were made:

- added #73 as a related issue;
- confirmed Supabase Auth as the current authentication foundation;
- confirmed Google OAuth as the initial sign-in provider;
- clarified the separation between authentication and application

authorisation;

- clarified that Firebase Authentication is no longer part of the current runtime architecture;
- retained provider-neutral application identities using:

```
```text
(auth_provider, provider_subject)
```
```

- recorded that the current written lecturer approval explicitly covers Supabase-hosted PostgreSQL but does not separately confirm Supabase Auth;
- replaced the previous milestone roadmap with the team's agreed delivery targets;
- updated the authentication-related risk entry.

Updated Milestone Targets

Sprint 1

Deliver a working Basic vertical slice in which:

- an administrator can approve a submitter;
- an approved submitter can sign in;
- an approved submitter can submit in-scope T20 cricket delivery events;
- submitted events are validated;
- invalid submissions receive useful rejection messages;
- valid events are stored with provenance;
- fixture statistics are derived from accepted events;
- public users can browse fixtures, events and statistics without signing in.

Sprint 2

Complete:

- all remaining Basic requirements;
- batch staging and processing;
- idempotent and resumable imports;
- review and publication workflows;
- correction history;
- season, career and competition aggregates;
- selective recomputation;
- API versioning;

- API keys, quotas and rate limits;
- caching where justified;
- versioned dataset releases;
- Intermediate testing and documentation.

Sprint 3

Complete the Advanced requirements, including:

- analyst-defined custom statistics;
- safe and versioned statistic definitions;
- live event ingestion;
- duplicate, late and out-of-order event handling;
- replayable event processing;
- as-of-date statistic queries;
- dataset release comparison;
- asynchronous large-query jobs;
- change feeds;
- API compatibility and deprecation;
- usage reporting;
- anomaly detection;
- conflicting-submitter reconciliation;
- downstream correction propagation.

Final Submission

Reserve the final period mainly for:

- defect resolution;
- release hardening;
- production verification;
- final testing;
- documentation;
- traceability evidence;
- reports;
- presentation preparation;
- release tagging;
- submission packaging.

Authentication Decision

The architecture now records the following implementation:

```
```text
```

Frontend

↓

Supabase Auth with Google OAuth

↓

Supabase access token

↓

Handwritten Express API

↓

Application account, role and scoped permissions

```
```
```

Supabase Auth proves identity.

The Sport Analytics backend remains responsible for:

- application account status;
- user roles;
- approved-submitter status;
- competition, season or fixture scope;
- administrator permissions;
- route-specific authorisation.

The frontend must not be treated as the security boundary.

Scope

Included

- Architecture documentation changes.
- Roadmap alignment.
- Authentication terminology alignment.
- Risk-register update.
- AI attribution update.

Excluded

- Authentication implementation changes.
- Supabase configuration changes.
- Database migrations.
- Frontend authentication changes.

- Role or scope implementation.
- Stakeholder or lecturer approval of Supabase Auth.
- Changes to unrelated documentation files.

Verification Performed

Documentation Build

The documentation was built using:

```
```powershell
mkdocs build --strict
```
```

Result:

```
```text
Documentation built successfully.
```
```

MkDocs reported informational notices about pages that currently exist outside the configured navigation. These notices did not cause the strict build to fail and were not introduced by this Pull Request.

Formatting and Diff Validation

The modified architecture document was formatted using:

```
```powershell
npx prettier --write docs/architecture/system-architecture.md
```
```

The diff was checked using:

```
```powershell
git diff --check
```
```

Result:

```
```text
Passed with no whitespace errors.
```
```



```

### ### Firebase Terminology Check

The architecture document was searched using:

```
```powershell
git grep -n -i "firebase" -- docs/architecture/system-architecture.md
```
```

The remaining Firebase references only explain that:

- ADR-004 superseded the earlier Firebase decision;
- Firebase is not part of the current runtime architecture;
- Firebase must not be implemented alongside Supabase Auth.

### ### Repository Check

The full repository check was started using:

```
```powershell
npm run check
```
```

The following stages passed:

- repository structure check.

The check then stopped during Prettier validation because of formatting warnings in:

```
```text
docs/architecture/system-architecture.md
docs/database/schema.md
```
```

The architecture file changed by this Pull Request was formatted afterward.

The remaining warning in:

```
```text
docs/database/schema.md
```

```

is outside the scope of this Pull Request and was not modified on this branch.

The unrelated file has therefore not been reformatted or included in this change.

## ## Files Changed

```text

docs/architecture/system-architecture.md

```

No generated `site/` files are included in the Pull Request.

## ## Risks and Limitations

- Supabase Auth is selected and implemented by the team, but the written lecturer approval currently stored in the repository explicitly mentions Supabase-hosted PostgreSQL rather than Supabase Auth.
- Confirmation of Supabase Auth for production use remains recorded as an open compliance decision.
- This Pull Request updates architecture documentation only. It does not prove that every documented Sprint 1, Sprint 2 or Sprint 3 feature is already implemented.
- The revised roadmap is intentionally ambitious and must remain subject to backlog refinement, dependencies, team capacity and stakeholder decisions.
- Planned components remain clearly distinguished from implemented components.

## ## Reviewer Guidance

Please verify that:

- [ ] the roadmap matches the team's agreed milestone targets;
- [ ] Sprint 1 includes the working Basic vertical slice;
- [ ] Sprint 2 targets complete Basic and Intermediate functionality;
- [ ] Sprint 3 targets Advanced feature completion;
- [ ] Final Submission is primarily reserved for hardening and release preparation;
- [ ] Supabase Auth and Google OAuth are described consistently;
- [ ] Firebase is no longer presented as a current implementation option;
- [ ] authentication and authorisation remain clearly separated;

- [ ] the Supabase Auth approval limitation is recorded honestly;
- [ ] planned functionality is not presented as already implemented;
- [ ] no unrelated files are included;
- [ ] the documentation build evidence is sufficient.

## ## Definition of Done

This Pull Request is ready to merge when:

- [ ] at least one team member has reviewed it;
- [ ] requested changes have been addressed or rejected with motivation;
- [ ] required CI checks pass, except where a clearly documented unrelated pre-existing issue prevents the full check;
- [ ] the architecture document accurately reflects the agreed roadmap;
- [ ] authentication terminology is consistent;
- [ ] no generated documentation-site files are included;
- [ ] the Pull Request is approved;
- [ ] the Pull Request is merged into `main`.

## ## Git Methodology

Branch:

```
```text
docs/73-align-architecture-roadmap
```
```

Commit:

```
```text
docs(architecture): align roadmap and authentication terminology
```
```

Issue reference:

```
```text
Refs #73
```
```

This Pull Request closes the issue using:

```
```text
```

Closes #73

...

The issue should remain in `In Review` until the Pull Request is approved, all required checks have passed and the change is merged.

AI Declaration

The architecture roadmap and authentication terminology were reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

The Pull Request description was prepared with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

The team remains responsible for reviewing and verifying all documentation, technical claims and project decisions included in this Pull Request.

8/6/2026, 5:13:42 PM



k, now do for #36 docs etc, and woill do later

8/6/2026, 5:15:10 PM



Thinking



Inspecting repository docs and planning files

- Inspected repository documentation, planning files, and evidence directories

✓ Worked for 48s

Yes. For **#36**, create a Sprint 1 backlog checkpoint now, but **do not close #36 yet** because Sprint 2, Sprint 3 and Final Submission issues still need to be added later.

This Pull Request will use:

Refs #36

not:

Closes #36

Step 1: wait for #73 to merge, then create the branch

After the #73 Pull Request is merged:

```
git switch main
git pull origin main
git switch -c docs/36-project-backlog
```

This branch will add:

```
docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md
```

and update:

```
mkdocs.yml
```

Do only the branch creation first. Then I'll give you the complete `project-backlog.md` content using your real issue numbers.

8/6/2026, 5:15:12 PM



ok, go for it

8/7/2026, 1:51:35 PM



Thinking

- Created project backlog and searched catalogue for issue #36 details



Remembering

- Recovered Sport analytics issues and matched Gitea milestones and statuses

- Planning the backlog document

I'm preparing the project-planning file around issue #36, using the Gitea board as the source of truth. I'll document milestone scope, prioritization rules, ownership tracking, and only verified issue mappings.

- Planned the backlog document

✓ Worked for 1m 10s

Yes. Since #36 is specifically the **backlog/prioritisation** issue, the clean way to do it is to add a repository planning document that records the roadmap without pretending that future Sprint 2/3 issues have already been fully refined. That matches your Scrumban methodology: Gitea remains the source of truth, Sprint 1 is expanded first, and future work stays in Backlog until refinement. `Project_Methodology_v1.1.md`
`COMS3011A_Complete_Gitea_Issue_Catalogue.md`

Create:

```
docs/planning/project-backlog.md
```

Paste this entire file:

```
# Project Backlog and Milestone Plan
```

```
| Document Information | Details |
```

```
|---|---|
| Project | Sport Analytics Tool |
| Related Issue | #36 |
| Methodology | Lightweight Scrumban |
| Planning Target | Advanced-tier feature completeness by the end of Sprint 3 |
| Final Submission Target | Hardening, evidence, documentation and release
packaging |
| Last Updated | 7 August 2026 |
| Status | Active planning document |
```

1. Purpose

This document records the high-level project backlog, milestone allocation and prioritisation approach for the Sport Analytics Tool.

It supports issue #36 by providing a persistent repository record of:

- the agreed milestone outcomes;
- the current Sprint 1 backlog;
- existing Gitea work already mapped to project requirements;
- work that still requires a dedicated Gitea issue;
- the order in which work should be prioritised;
- the planned Sprint 2, Sprint 3 and Final Submission scope; and
- the rules used when refining future work.

This document does not replace Gitea.

Gitea Issues, Milestones and the Project board remain the authoritative source for:

- current issue status;
- assignees;
- acceptance criteria;
- dependencies;
- board position;
- Pull Requests; and
- completion evidence.

This document is a planning index and roadmap snapshot.

2. Planning Approach

The project uses a lightweight Scrumban methodology.

The backlog is therefore planned across the four formal milestones:

```
```text
Sprint 1
Sprint 2
Sprint 3
Final Submission
```
```

Sprint 1 work is refined first because it represents the current delivery period.

Sprint 2 and Sprint 3 work may remain at roadmap level until refinement one sprint ahead. Future work must not be moved to `Ready` merely because it appears in this document.

An issue may move to `Ready` only once it satisfies the project Definition of Ready, including:

- a clear purpose;
- an understood expected outcome;
- acceptance criteria;
- identified dependencies;
- required stakeholder clarification;
- an appropriate assignee;
- a scope small enough to complete within the sprint; and
- an understood verification approach.

The Project board remains:

```
```text
Backlog
Ready
In Progress
In Review
Blocked
```

Done

...

---

### ## 3. Prioritisation Rules

Work is prioritised in the following order:

1. mandatory project requirements;
2. work required for the current sprint;
3. issues blocking other work;
4. stakeholder priorities;
5. high-risk technical work;
6. defects affecting core functionality;
7. testing and documentation supporting completed features; and
8. optional enhancements.

For the Sport Analytics Tool, the highest early priorities are:

- event-data modelling;
- submission validation;
- provenance and traceability;
- authentication and authorisation;
- approved-submitter scope;
- API design and implementation;
- statistic derivation;
- database foundations;
- automated testing;
- deployment; and
- project documentation.

Basic functionality must remain reliable before significant effort is moved away from it toward Intermediate or Advanced functionality.

---

### ## 4. Milestone Outcomes

#### ### Sprint 1

Sprint 1 targets a deployed and documented Basic vertical slice.

The intended end-to-end workflow is:

```
```text
administrator approves submitter
    ↓
approved submitter signs in
    ↓
submitter sends delivery events
    ↓
backend verifies identity and scope
    ↓
events are validated
    ↓
valid submission is stored with provenance
    ↓
statistics are derived from accepted events
    ↓
public user retrieves fixtures, events and statistics
```
```

Sprint 1 must also establish the technical, testing, documentation and deployment foundations required by later work.

### ### Sprint 2

Sprint 2 targets:

- completion of any remaining Basic requirements; and
- completion of the Intermediate feature tier.

This includes:

- file-based and batch submissions;
- resumable and idempotent processing;
- review and publication workflows;
- event corrections and audit history;
- competition, season and career aggregates;
- selective recomputation;
- performance testing and optimisation;
- API versioning;
- API keys, quotas and rate limits;

- caching;
- external API integration;
- dataset releases;
- formal user testing; and
- Intermediate documentation and testing.

### ### Sprint 3

Sprint 3 targets the Advanced feature tier.

This includes:

- analyst-defined custom statistics;
- safe and versioned statistic definitions;
- full-history statistic evaluation;
- live event ingestion;
- duplicate, late and out-of-order event handling;
- replayable processing;
- temporal and as-of-date statistics;
- dataset release comparisons;
- aggregate query APIs;
- asynchronous large-query jobs;
- change feeds;
- API deprecation and backwards-compatibility testing;
- usage metering;
- anomaly detection;
- conflicting-submitter reconciliation;
- downstream correction propagation;
- production observability; and
- Advanced testing and documentation.

### ### Final Submission

No major new feature scope should normally be introduced during Final Submission.

The period is reserved primarily for:

- defect resolution;
- release hardening;
- production verification;
- database backup and restore verification;

- security testing;
- accessibility testing;
- performance testing;
- browser compatibility;
- final documentation;
- requirements traceability;
- known limitation records;
- demonstration preparation;
- reports;
- presentation material;
- release notes;
- version tagging; and
- submission packaging.

---

## ## 5. Existing Foundation Work

The following Gitea issues already represent important project-foundation work.

| Issue | Work                                                   |
|-------|--------------------------------------------------------|
| ---   | ---                                                    |
| #2    | Git and project methodology documentation              |
| #7    | Repository governance and initial setup alignment      |
| #8    | Main-branch protection and Pull Request approval rules |
| #10   | CI quality gate and reproducible installation          |
| #11   | Public documentation-site deployment foundation        |
| #12   | Developer onboarding and technology-stack motivation   |
| #13   | Supabase-hosted PostgreSQL foundation                  |
| #14   | Managed authentication and OAuth/OIDC foundation       |
| #15   | Azure backend foundation                               |
| #17   | Azure frontend foundation                              |
| #27   | Relational delivery-event schema                       |
| #28   | Cricsheet T20 dataset downloader                       |
| #29   | MkDocs deployment through Cloudflare and Wrangler      |
| #30   | Per-team AI transcript folders and usage process       |
| #36   | Complete and prioritise the project backlog            |

These issues should not be duplicated merely because a catalogue item describes similar work.

Where a later issue extends an existing foundation, it should reference the

existing issue as a dependency or predecessor.

---

## ## 6. Sprint 1 Requirement Mapping

The full planning catalogue defines 36 Sprint 1 work items.

The table below records the currently verified Gitea mappings.

A single Gitea issue may satisfy more than one catalogue item where the work was intentionally combined and remains small enough to review effectively.

`Pending mapping` means that the required work exists in the roadmap but a verified dedicated Gitea issue number has not yet been recorded in this planning document.

| Catalogue ID | Sprint 1 Work                                                                                               | Current Gitea Mapping   |
|--------------|-------------------------------------------------------------------------------------------------------------|-------------------------|
| ---          | ---                                                                                                         | ---                     |
| S1-01        | Confirm sport, professional competition scope, fixture definition, event vocabulary and required statistics | #37                     |
| S1-02        | Clarify Basic, Intermediate and Advanced acceptance decisions with stakeholder                              | Pending mapping         |
| S1-03        | Create requirements traceability matrix and requirement change log                                          | Pending mapping         |
| S1-04        | Produce full system architecture and deployment design                                                      | #38, with follow-up #73 |
| S1-05        | Record decisions for caching, background jobs, file storage and live ingestion                              | Pending mapping         |
| S1-06        | Produce information architecture, user journeys and responsive wireframes                                   | Pending mapping         |
| S1-07        | Define accessible design system and reusable frontend baseline                                              | Pending mapping         |
| S1-08        | Define stable identifiers and shared API response, error, filtering, sorting and pagination contracts       | Pending mapping         |
| S1-09        | Create OpenAPI baseline and versioning/deprecation conventions                                              | Pending mapping         |
| S1-10        | Implement repeatable database migrations and development seed tooling                                       | Pending mapping         |
| S1-11        | Implement competition, season, competitor, participant and fixture schema                                   | Pending mapping         |
| S1-12        | Implement account, role, submitter approval and scope schema                                                | #43                     |

| S1-13 | Implement submission, ordered-event, provenance and correction schema  
| #27 |

| S1-14 | Implement backend database repository and transaction foundation |  
Pending mapping |

| S1-15 | Create representative development seed and reference fixture dataset  
| Related work: #28; dedicated mapping pending |

| S1-16 | Configure managed authentication providers and account flows |  
Related foundation: #14; dedicated mapping pending |

| S1-17 | Implement application account synchronisation and authenticated  
profile API | Pending mapping |

| S1-18 | Implement reusable authentication, role, approval and scope  
middleware | Pending mapping |

| S1-19 | Implement frontend authentication state, protected routes and  
authenticated API client | Pending mapping |

| S1-20 | Build Create Account, Sign In, Account and Sign Out navigation | #39  
|

| S1-21 | Build accessible sign-up and sign-in forms | #39 |

| S1-22 | Build forgotten-password and password-reset flow | #39 where  
included; otherwise follow-up required |

| S1-23 | Implement secure account deletion | Pending mapping |

| S1-24 | Implement administrator APIs for submitter approval, revocation and  
scope assignment | Pending mapping |

| S1-25 | Build administrator submitter approval and scope-management interface  
| Pending mapping |

| S1-26 | Define executable delivery submission schema and valid/invalid  
examples | #49 |

| S1-27 | Implement delivery-schema and sport-rule validation with structured  
rejection errors | Pending mapping |

| S1-28 | Implement direct JSON delivery submission with authorisation and  
provenance | Pending mapping |

| S1-29 | Implement public fixture and ordered-event APIs with filtering and  
pagination | Pending mapping |

| S1-30 | Implement Basic event-derived statistic engine and fixture-statistics  
API | Pending mapping |

| S1-31 | Build first public fixture, event and statistics browsing experience  
| Pending mapping |

| S1-32 | Add backend and frontend tests for the Basic vertical slice | Related  
testing foundation: #72; feature-specific tests remain required |

| S1-33 | Repair Azure deployment workflows and add deployed smoke checks |  
Related foundations: #15 and #17; dedicated repair issue required where still  
applicable |

| S1-34 | Automate MkDocs deployment from Gitea to Cloudflare Pages | #29 |

| S1-35 | Publish Sprint 1 setup, architecture, API, testing documentation and evidence | Pending mapping |  
| S1-36 | Conduct Sprint 1 stakeholder review, retrospective and milestone tag | Pending mapping |

---

## ## 7. Authentication Terminology

The original planning catalogue referenced Firebase because it reflected an earlier authentication investigation.

The current architecture uses:

```
```text
Supabase Auth
  +
Google OAuth
```
```

ADR-004 supersedes the earlier Firebase direction.

Future backlog issues must therefore use current provider terminology and must not introduce Firebase as a parallel authentication implementation unless a later approved ADR explicitly changes the architecture.

Authentication and application authorisation remain separate.

Supabase Auth proves identity.

The Sport Analytics backend determines:

- application account status;
- roles;
- approved-submitter status;
- assigned scope; and
- administrator permissions.

Application identity remains provider-neutral through:

```
```text
(auth_provider, provider_subject)
```



```

---

## ## 8. Sprint 1 Priority Groups

Sprint 1 work should be pulled according to dependencies rather than simply by issue number.

### ### Priority A - foundations and blockers

Complete or stabilise:

- CI;
- database connectivity and migrations;
- authentication;
- deployment;
- architecture;
- domain decisions;
- event schema; and
- testing infrastructure.

Relevant current issues include:

```text

#10

#13

#14

#15

#17

#27

#29

#37

#38

#49

#72

#73

```

### ### Priority B - identity and authorisation

Complete:

- application accounts;
- roles;
- approved-submitter state;
- competition or fixture scope;
- authentication middleware;
- administrator approval APIs; and
- administrator approval UI.

Relevant current issues include:

```
```text
#39
#43
```
```

Additional implementation issues must be created where no existing issue covers the work.

### ### Priority C - Basic event vertical slice

Complete the core product flow:

1. validate delivery submissions;
2. reject invalid deliveries with actionable errors;
3. verify submitter approval and scope;
4. store submissions and events with provenance;
5. expose public fixture and event reads;
6. derive statistics from events;
7. expose statistics through the handwritten API; and
8. display the first usable public browsing interface.

These items are the central Sprint 1 product outcome.

### ### Priority D - Sprint evidence and close-out

Before Sprint 1 closes:

- automated tests must cover the vertical slice;
- deployment must be verified;
- documentation must reflect the implemented system;
- stakeholder feedback must be recorded;

- the sprint review must be completed;
- known limitations must be recorded;
- the exact milestone state must pass the required checks; and
- the `sprint-1` tag must be created from the approved `main` commit.

---

## ## 9. Sprint 2 Planned Backlog

The complete planning catalogue contains 40 Sprint 2 items.

These cover:

### ### Remaining Basic

- public competition and participant APIs;
- administration CRUD;
- public browsing pages;
- administrator management pages;
- JSON and CSV uploads;
- submission history;
- submitter UI;
- corrections;
- statistic provenance;
- exports;
- external API integration;
- full Basic end-to-end acceptance;
- accessibility;
- security; and
- user testing.

### ### Intermediate

- batch design;
- staging;
- asynchronous processing;
- accepted/rejected reports;
- idempotence;
- resumability;
- review and publication;
- conflicting-event checks;
- correction history;

- season/career/competition aggregates;
- selective recomputation;
- golden reference results;
- representative-scale performance testing;
- database optimisation;
- explicit API versioning;
- consumer API keys;
- rate limits and quotas;
- caching;
- versioned dataset releases;
- release downloads;
- Intermediate integration testing; and
- Intermediate documentation.

These items remain part of the committed roadmap.

They should be converted to, or mapped against, real Gitea issues during Sprint 2 refinement.

They remain in `Backlog` until they satisfy the Definition of Ready.

---

## ## 10. Sprint 3 Planned Backlog

The complete planning catalogue contains 28 Sprint 3 items.

The Advanced backlog includes:

- custom-statistic definition language;
- statistic-definition versioning;
- safe execution;
- full-history evaluation;
- custom-statistic APIs and UI;
- provenance;
- correction propagation;
- live event ingestion;
- late and out-of-order event handling;
- deterministic replay;
- temporal history;
- as-of statistics;
- release comparison;

- aggregate queries;
- asynchronous query jobs;
- change feeds;
- API deprecation;
- contract compatibility tests;
- consumer usage reporting;
- anomaly detection;
- conflicting-submitter reconciliation;
- downstream correction propagation;
- advanced correctness and performance testing;
- production observability;
- application-wide accessibility and UX polish;
- security and recovery review;
- advanced technical documentation;
- stakeholder acceptance testing; and
- Sprint 3 close-out.

These items remain in the project roadmap but should not be marked `Ready` until refined.

---

## ## 11. Final Submission Planned Backlog

The Final Submission catalogue contains 10 planned work areas:

1. triage and resolve release-blocking defects;
2. production migrations, backups and restore verification;
3. production secrets and deployment verification;
4. performance, security, accessibility and browser audits;
5. final requirements traceability and limitations register;
6. database, third-party, licence, testing and AI documentation;
7. reproducible demonstration dataset and scripted demo;
8. group presentation preparation;
9. group report, individual report and peer-review evidence; and
10. release notes, final tag and submission package.

The Final Submission period is not intended to become a fourth major feature-development sprint.

---

## ## 12. Backlog Maintenance Rules

The backlog is a living planning artifact.

When stakeholder feedback or implementation discoveries change the required work:

1. update the relevant Gitea issue where possible;
2. create a new issue where the work is genuinely separate;
3. update dependencies;
4. reprioritise the Project board where necessary;
5. record any important requirement change;
6. update this document where the milestone roadmap changes materially.

Requirements must not be changed silently.

Duplicate issues should not be created merely to make the catalogue appear complete.

Where one existing issue legitimately satisfies multiple catalogue items, the relationship should be documented.

Where an issue grows beyond a manageable unit of work, it should be split.

---

## ## 13. Remaining Work for Issue #36

Issue #36 remains open while the complete roadmap is progressively converted into the working Gitea backlog.

The current checkpoint is complete when:

- [x] the four milestone outcomes are defined;
- [x] the prioritisation rules are documented;
- [x] existing foundation work is mapped;
- [x] the Sprint 1 catalogue is recorded;
- [x] known real Gitea issue mappings are recorded;
- [x] unmapped Sprint 1 work is made visible;
- [x] Sprint 2 scope is recorded;
- [x] Sprint 3 scope is recorded;
- [x] Final Submission scope is recorded;

- [x] current Supabase Auth terminology is reflected;
- [ ] every remaining Sprint 1 catalogue item has either a verified existing issue or a dedicated Gitea issue;
- [ ] Sprint 2 issues are fully mapped/refined before Sprint 2 begins;
- [ ] Sprint 3 issues are fully mapped/refined before Sprint 3 begins;
- [ ] Final Submission tasks are mapped before final release planning;
- [ ] issue ownership and milestone allocation are reviewed by the team.

Issue #36 should therefore use:

```
```text
Refs #36
```
```

for this documentation checkpoint.

It should not be automatically closed by this Pull Request.

---

## ## 14. Review Cadence

The backlog should be reviewed:

- during Sprint planning;
- after significant stakeholder feedback;
- during the Thursday standup where priorities or blockers change;
- before moving work from `Backlog` to `Ready`; and
- during sprint close-out.

The backlog does not need to be rewritten after every small status change because those changes belong on the Gitea Project board.

---

## ## AI Declaration

The preceding document was planned, generated and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

The project team remains responsible for verifying the issue mappings,

milestone allocation, requirements and prioritisation decisions recorded in this document.

The important part here is that we **do not falsely claim the whole future Gitea backlog already exists**. The original catalogue says Sprint 1 should be expanded first and future Sprint 2/3 issues remain in Backlog until refinement, while your methodology says only sufficiently defined issues move to `Ready`.

`COMS3011A_Complete_Gitea_Issue_Catalogue.md` `Project_Methodology_v1.1.md`

Also, the AI declaration is necessary: your course policy requires written documents to identify the AI tool and model used. `ai_policy_20_jul.pdf`

Once you save that file, **don't commit yet**. Next we should make the short Sprint 1 planning-evidence file for `evidence/sprints/sprint-1/2026-08-06-planning.md`, because your methodology explicitly requires a repository sprint-planning record.

`Project_Methodology.md`

*8/7/2026, 1:51:59 PM*





6eb9215e-6a2c-455a-8266-b2...



da2a598c-7d10-4162-8817-f7...

redo with relevant mappings

Add Cricsheet T20 dataset downloader #28

Design and implement the delivery event schema #27

Merge conflict for DB #21

Restore green CI and reproducible installation #10

Validate developer onboarding and motivate the technology stack #12

Establish the Azure backend deployment foundation #15

Establish Azure frontend deployment foundation #17

Deploy MkDocs documentation to Cloudflare Pages using Wrangler #29

Create per-team-member folders for AI transcripts #30

Investigate and establish the OAuth/OIDC authentication foundation #14

Establish the Supabase PostgreSQL foundation #13

Deploy the public documentation site with Cloudflare Pages #11

Protect main and require Pull Request approval #8

Align repository governance and document initial setup deviation #7

Add Git and project methodology documentation #2

Complete the automated testing foundation and test environment setup #72

Backlog planning = #36

Sport and domain definition = #37

Produce the complete system architecture, data flow and development roadmap = #38

Build frontend authentication pages and navigation controls #39

Implement the application account, role, approved-submitter and scope database schema #43

49 was renamed to Define the executable delivery

submission schema and validation examples

Architecture follow-up = #73

8/7/2026, 1:55:41 PM



## Thinking

- **Mapping implementation work**

I'm mapping numbered work items to existing issues, noting renames where titles are stale. I'm also separating overlapping fixture, event, documentation, deployment, architecture, and testing tasks so each dependency remains clear.

- **Mapped implementation work**

✓ Worked for 21s

Yes - with the new issue numbers, the backlog document should be updated properly rather than leaving most Sprint 1 work as "Pending mapping".

One important correction before the file: **#66 still says Firebase** in its title. Since your architecture now uses Supabase Auth, keep #66 as the mapping but rename that Gitea issue to something like **"Implement secure account deletion across Supabase Auth and application data"** before anyone starts it.

Replace the previous `docs/planning/project-backlog.md` with this version:

### # Project Backlog and Milestone Plan

```
Document Information	Details
Project	Sport Analytics Tool
Related Issue	#36
Methodology	Lightweight Scrumban
Planning Target	Advanced-tier feature completeness by the end of Sprint 3
Final Submission Target	Hardening, evidence, documentation and release packaging
Last Updated	7 August 2026
Status	Active planning document
```

---

## ## 1. Purpose

This document records the high-level project backlog, milestone allocation and prioritisation approach for the Sport Analytics Tool.

It supports issue #36 by providing a persistent repository record of:

- the agreed milestone outcomes;
- the current Sprint 1 backlog;
- existing Gitea issues mapped to project requirements;
- work that still requires refinement or a dedicated Gitea issue;
- dependencies between foundation and implementation work;
- the planned Sprint 2, Sprint 3 and Final Submission scope; and
- the rules used when refining future work.

This document does not replace Gitea.

Gitea Issues, Milestones and the Project board remain the authoritative source for:

- issue status;
- assignees;
- acceptance criteria;
- dependencies;
- board position;
- Pull Requests; and
- completion evidence.

This document acts as a planning index and milestone snapshot.

---

## ## 2. Planning Approach

The project uses a lightweight Scrumban methodology.

The backlog is planned across four formal delivery periods:

```text

Sprint 1
Sprint 2
Sprint 3
Final Submission
...

Sprint 1 work is refined first because it represents the current delivery period.

Sprint 2 and Sprint 3 work may remain at roadmap level until refinement one sprint ahead.

Future work must not be moved to `Ready` merely because it appears in this document.

An issue may move to `Ready` only once it satisfies the project Definition of Ready, including:

- a clear purpose;
- an understood expected outcome;
- acceptance criteria;
- identified dependencies;
- required stakeholder clarification;
- an appropriate assignee;
- a scope small enough to complete within the sprint; and
- an understood verification approach.

The Project board remains:

```text  
Backlog  
Ready  
In Progress  
In Review  
Blocked  
Done  
...  
  
---

## 3. Prioritisation Rules

Work is prioritised in the following order:

1. mandatory project requirements;
2. work required for the current sprint;
3. work blocking other issues;
4. stakeholder priorities;
5. high-risk technical work;
6. defects affecting core functionality;
7. testing and documentation supporting completed features; and
8. optional enhancements.

For the Sport Analytics Tool, the highest early priorities are:

- domain and sport definitions;
- event-data modelling;
- submission validation;
- provenance and traceability;
- authentication and authorisation;
- approved-submitter scope;
- API contracts;
- database foundations;
- statistic derivation;
- automated testing;
- deployment; and
- project documentation.

Basic functionality must remain reliable before significant effort is redirected toward Intermediate or Advanced functionality.

---

## ## 4. Milestone Outcomes

### ### Sprint 1 - Working Basic Vertical Slice

Sprint 1 targets a deployed and documented Basic vertical slice.

The intended end-to-end workflow is:

```
```text
```

```
administrator approves submitter
```

↓

```
approved submitter signs in
    ↓
submitter sends delivery events
    ↓
backend verifies identity, role and scope
    ↓
events are validated
    ↓
invalid events receive useful rejection messages
    ↓
valid events are stored with provenance
    ↓
statistics are derived from accepted events
    ↓
public users retrieve fixtures, events and statistics
...

```

Sprint 1 must also establish the technical, testing, documentation and deployment foundations required by later work.

Sprint 2 - Complete Basic and Intermediate

Sprint 2 targets:

- completion of any remaining Basic requirements; and
- completion of the Intermediate feature tier.

This includes:

- file-based and batch submissions;
- resumable and idempotent processing;
- review and publication workflows;
- event corrections and audit history;
- competition, season and career aggregates;
- selective recomputation;
- performance testing and optimisation;
- API versioning;
- API keys, quotas and rate limits;
- caching;
- external API integration;
- dataset releases;
- formal user testing; and

- Intermediate documentation and testing.

Sprint 3 - Advanced Feature Completion

Sprint 3 targets the Advanced feature tier.

This includes:

- analyst-defined custom statistics;
- safe and versioned statistic definitions;
- full-history statistic evaluation;
- live event ingestion;
- duplicate, late and out-of-order event handling;
- deterministic replay;
- temporal and as-of-date statistics;
- dataset release comparisons;
- aggregate query APIs;
- asynchronous large-query jobs;
- change feeds;
- API deprecation and backwards-compatibility testing;
- usage metering;
- anomaly detection;
- conflicting-submitter reconciliation;
- downstream correction propagation;
- production observability; and
- Advanced testing and documentation.

Final Submission - Hardening and Release

No major new feature scope should normally be introduced during Final Submission.

The period is reserved primarily for:

- defect resolution;
- release hardening;
- production verification;
- database backup and restore verification;
- security testing;
- accessibility testing;
- performance testing;
- browser compatibility;

- final documentation;
- requirements traceability;
- known limitation records;
- demonstration preparation;
- reports;
- presentation material;
- release notes;
- version tagging; and
- submission packaging.

5. Existing Project and Foundation Issues

The following issues already establish the repository, infrastructure, governance and technical foundations of the project.

Issue	Work
#2	Add Git and project methodology documentation
#7	Align repository governance and document initial setup deviation
#8	Protect `main` and require Pull Request approval
#10	Restore green CI and reproducible installation
#11	Deploy the public documentation site with Cloudflare Pages
#12	Validate developer onboarding and motivate the technology stack
#13	Establish the Supabase PostgreSQL foundation
#14	Investigate and establish the OAuth/OIDC authentication foundation
#15	Establish the Azure backend deployment foundation
#17	Establish the Azure frontend deployment foundation
#21	Resolve database merge conflict
#27	Design and implement the delivery event schema
#28	Create the Cricsheet T20 dataset downloader
#29	Deploy MkDocs documentation to Cloudflare Pages using Wrangler
#30	Create per-team-member folders for AI transcripts
#36	Complete and prioritise the project backlog
#37	Confirm sport and domain definitions
#38	Produce the complete system architecture, data flow and development roadmap
#39	Build frontend authentication pages and navigation controls
#43	Implement the application account, role, approved-submitter and scope database schema
#49	Define the executable delivery submission schema and validation


```
examples |
| #72 | Complete the automated testing foundation and test environment setup |
| #73 | Align architecture roadmap and authentication terminology |
```

These issues must not be duplicated merely because a later planning item describes related work.

Where later implementation issues extend an existing foundation issue, the relationship should be recorded as a dependency or predecessor.

6. Sprint 1 Requirement-to-Issue Mapping

The Sprint 1 planning catalogue contains 36 work areas.

The following table records the current verified mapping between those work areas and Gitea issues.

Where multiple issues are listed, they collectively satisfy or support the catalogue item.

Catalogue ID	Sprint 1 Work	Gitea Mapping
S1-01	Confirm sport, professional competition scope, fixture definition, event vocabulary and required statistics	#37
S1-02	Clarify Basic, Intermediate and Advanced acceptance decisions with the stakeholder	Pending dedicated mapping
S1-03	Create requirements traceability matrix and requirement change log	Pending dedicated mapping
S1-04	Produce complete system architecture, data flow, deployment design and development roadmap	#38, follow-up #73
S1-05	Record architecture decisions for caching, background jobs, file storage and live ingestion	#55
S1-06	Produce information architecture, user journeys and responsive wireframes	#56
S1-07	Define accessible design system and reusable frontend component baseline	#57
S1-08	Define stable identifiers and shared API response, error, filtering, sorting and pagination contracts	#58
S1-09	Create OpenAPI baseline and API versioning and deprecation conventions	#59

| S1-10 | Implement repeatable database migrations and development seed tooling | #60 |

| S1-11 | Implement competition, season, competitor, participant and fixture database schema | #46 |

| S1-12 | Implement application account, role, approved-submitter and scope database schema | #43 |

| S1-13 | Implement submission, ordered-event, provenance and correction database schema | #61, building on #27 |

| S1-14 | Implement backend database repository and transaction foundation | #62 |

| S1-15 | Create representative development seed and reference fixture dataset | #63, supported by #28 |

| S1-16 | Configure managed authentication provider and account authentication foundation | #14 |

| S1-17 | Implement account synchronisation, profile API and role-based authorisation | #44 |

| S1-18 | Implement reusable authentication, role, approval and scope authorisation | #44, with submission scope enforcement in #51 |

| S1-19 | Implement frontend authentication state, protected routes and authenticated API client | #64 |

| S1-20 | Build Create Account, Sign In, Account, Sign Out and authentication navigation | #39 |

| S1-21 | Build accessible sign-up and sign-in forms | #39, supported by #57 |

| S1-22 | Build forgotten-password and password-reset flow | #65 |

| S1-23 | Implement secure account deletion across authentication and application data | #66 |

| S1-24 | Implement administrator APIs for submitter approval, revocation and scope assignment | #45 |

| S1-25 | Build administrator submitter approval and scope-management interface | #45 |

| S1-26 | Define executable delivery submission schema and valid/invalid validation examples | #49 |

| S1-27 | Implement event validation with detailed and structured rejection messages | #50 |

| S1-28 | Implement approved-submitter event submission with provenance and scope enforcement | #51 |

| S1-29 | Implement public fixture and ordered-event APIs with filtering and pagination | #47 and #67 |

| S1-30 | Implement Basic event-derived fixture statistics and public statistics API | #52 |

| S1-31 | Build first public competition, fixture, event and statistics browsing experience | #48 and #54 |

S1-32	Add backend and frontend tests for the Basic vertical slice	#68, supported by testing foundation #72
S1-33	Repair Azure deployment workflows for the current monorepo and add deployed smoke checks	#69, building on #15 and #17
S1-34	Automate MkDocs deployment from Gitea to Cloudflare Pages	#29, building on #11
S1-35	Publish Sprint 1 setup, architecture, API, testing documentation and evidence	#70
S1-36	Conduct Sprint 1 stakeholder review, retrospective and milestone tag	#71

7. Additional Sprint 1 Implementation Issues

The following implementation issues form the main Sprint 1 product work.

Account, Authentication and Authorisation

Issue	Work
---	---
#14	OAuth/OIDC authentication foundation
#39	Frontend authentication pages and navigation controls
#43	Account, role, approved-submitter and scope database schema
#44	Account synchronisation, profile API and role-based authorisation
#45	Administrator submitter approval and scope management
#64	Frontend authentication state, protected routes and authenticated API client
#65	Forgotten-password and password-reset flow
#66	Secure account deletion across authentication and application data

Issue #66 currently contains legacy Firebase terminology in its title.

Because ADR-004 and the current architecture use Supabase Auth, #66 should be renamed before implementation to:

```text

Implement secure account deletion across Supabase Auth and application data

```

Firebase must not be implemented alongside Supabase Auth.

Database and Domain

Issue	Work
#13	Supabase PostgreSQL foundation
#21	Database merge-conflict resolution
#27	Delivery event schema
#28	Cricsheet T20 dataset downloader
#43	Application account, role, approved-submitter and scope schema
#46	Competition, season, competitor, participant and fixture schema
#60	Repeatable database migrations and development seed tooling
#61	Submission, ordered-event, provenance and correction schema
#62	Backend database repository and transaction foundation
#63	Representative development seed and reference fixture dataset

Event Submission and Validation

Issue	Work
#49	Define executable delivery submission schema and validation examples
#50	Implement event validation with detailed rejection messages
#51	Implement approved-submitter event submission with provenance and scope enforcement
#53	Build approved-submitter event upload and validation-results interface

Public API, Statistics and Frontend

Issue	Work
#47	Public competition, fixture and participant read APIs
#48	Public competition, fixture and participant browsing pages
#52	Event-derived fixture statistics and public statistics API
#54	Public fixture and competitor statistics pages
#67	Public ordered-event read APIs with filtering and pagination

Architecture, API Design and UX

Issue	Work
#38	Complete system architecture, data flow and roadmap
#55	Architecture decisions for caching, background jobs, file storage and

```
live ingestion |
| #56 | Information architecture, user journeys and responsive wireframes |
| #57 | Accessible design system and reusable frontend component baseline |
| #58 | Stable identifiers and shared API response, error, filtering, sorting
and pagination contracts |
| #59 | OpenAPI baseline and API versioning/deprecation conventions |
| #73 | Architecture roadmap and authentication terminology alignment |
```

Testing, Deployment and Evidence

```
| Issue | Work |
| --- | --- |
| #10 | Restore green CI and reproducible installation |
| #11 | Deploy public documentation site |
| #15 | Azure backend deployment foundation |
| #17 | Azure frontend deployment foundation |
| #29 | MkDocs deployment through Cloudflare Pages and Wrangler |
| #68 | Basic vertical-slice backend and frontend tests |
| #69 | Repair Azure workflows and add deployed smoke checks |
| #70 | Sprint 1 setup, architecture, API, testing documentation and evidence |
| #71 | Sprint 1 stakeholder review, retrospective and milestone tag |
| #72 | Automated testing foundation and test environment setup |
```

8. Authentication Terminology

The project originally investigated Firebase Authentication.

The current architecture uses:

```
```text
Supabase Auth
 +
Google OAuth
```
```

ADR-004 supersedes the earlier Firebase direction.

Future backlog issues must therefore use the current Supabase Auth terminology and must not introduce Firebase as a parallel authentication implementation unless a later approved ADR explicitly supersedes ADR-004.

The authentication flow is:

```
```text
```

React frontend

↓

Supabase Auth

↓

Google OAuth

↓

Supabase access token

↓

Handwritten Express API

↓

Application account, role, approval state and scope

```
```
```

Authentication and application authorisation remain separate.

Supabase Auth proves identity.

The Sport Analytics backend determines:

- application account state;
- role;
- approved-submitter status;
- competition, season or fixture scope; and
- administrator permissions.

Application identity remains provider-neutral through:

```
```text
```

```
(auth_provider, provider_subject)
```

```
```
```

The written lecturer approval currently stored in the repository explicitly covers Supabase-hosted PostgreSQL but does not separately confirm Supabase Auth.

Supabase Auth is therefore the team's selected and implemented authentication foundation, while any required formal confirmation remains an open compliance decision.

9. Sprint 1 Dependency Order

Sprint 1 work should be pulled according to dependencies rather than simply by issue number.

Priority A - Governance and Technical Foundations

Relevant issues include:

```text

#2  
#7  
#8  
#10  
#12  
#13  
#14  
#15  
#17  
#21  
#27  
#28  
#29  
#30  
#36  
#37  
#38  
#55  
#56  
#57  
#58  
#59  
#60  
#72  
#73  
```

These establish:

- repository governance;
- CI;
- development setup;
- architecture;
- domain definitions;
- database connectivity;
- event modelling;
- authentication;
- API conventions;
- testing;
- deployment; and
- documentation infrastructure.

Priority B - Persistent Domain and Identity Model

Relevant issues include:

```text

#43

#46

#61

#62

#63

```

These establish the database structures and transaction foundations required by the product vertical slice.

Priority C - Authentication and Approval Flow

Relevant issues include:

```text

#39

#44

#45

#64

#65

#66

```

The required flow is:


```
```text
```

user authenticates

↓

application account is resolved

↓

role is determined

↓

administrator approves submitter

↓

submitter receives allowed scope

```
```
```

Priority D - Event Submission Vertical Slice

Relevant issues include:

```
```text
```

#49

#50

#51

#53

```
```
```

The required flow is:

```
```text
```

approved submitter

↓

delivery submission

↓

schema validation

↓

sport-rule validation

↓

scope authorisation

↓

structured rejection or acceptance

↓

provenance-backed persistence

```
```
```

Priority E - Public Read and Statistics Vertical Slice

Relevant issues include:

```
```text
```

```
#47
```

```
#48
```

```
#52
```

```
#54
```

```
#67
```

```
```
```

The required public flow is:

```
```text
```

```
public user
```

```
↓
```

```
competition / fixture discovery
```

```
↓
```

```
ordered event viewing
```

```
↓
```

```
derived statistics
```

```
```
```

Public read functionality must not require authentication.

Priority F - Verification and Sprint Close-out

Relevant issues include:

```
```text
```

```
#68
```

```
#69
```

```
#70
```

```
#71
```

```
```
```

Before Sprint 1 closes:

- the Basic vertical slice must be covered by automated tests;
- deployment must be verified;
- documentation must reflect the implemented system;

- stakeholder feedback must be recorded;
- the sprint review must be completed;
- known limitations must be recorded;
- required checks must pass for the approved milestone state; and
- the Sprint 1 milestone tag must identify the reviewed `main` commit.

10. Sprint 1 End-to-End Trace

The main Sprint 1 product flow can now be traced directly to Gitea issues.

1. Define the Sport

```
```text
#37
```
```

Confirm T20 cricket terminology, competition scope, fixtures, events and required statistics.

2. Establish Architecture

```
```text
#38
#55
#58
#59
#73
```
```

Define the system boundaries, service interactions, API conventions and technical decisions.

3. Establish Data Storage

```
```text
#13
#27
#43
#46
#60
```

#61  
#62  
#63  
```

Establish PostgreSQL, migrations, domain data, authentication-linked application accounts, events, submissions, provenance and corrections.

4. Establish Authentication

```text  
#14  
#39  
#44  
#64  
#65  
#66  
```

Support account authentication, frontend auth state, account resolution and role-based access.

5. Approve Submitters

```text  
#43  
#45  
```

Store approval and scope and provide administrator management functionality.

6. Define and Validate Deliveries

```text  
#49  
#50  
```

Define the executable delivery contract and validate submitted events.

7. Submit Events

```
```text
```

```
#51
```

```
#53
```

```
```
```

Allow approved submitters to submit in-scope delivery events and receive validation outcomes.

8. Persist Events with Provenance

```
```text
```

```
#27
```

```
#61
```

```
#62
```

```
```
```

Store accepted events and their submission provenance through the backend repository layer.

9. Read Public Event Data

```
```text
```

```
#47
```

```
#48
```

```
#67
```

```
```
```

Expose public competition, fixture, participant and ordered-event data.

10. Derive Statistics

```
```text
```

```
#52
```

```
```
```

Compute fixture statistics from accepted event data rather than treating statistics as independent source data.

11. Display Statistics

```
```text
```

```
#54
```

```

Provide public fixture and competitor statistics pages.

12. Verify the Vertical Slice

```text

#68

#72

```

Provide automated backend and frontend verification.

13. Deploy

```text

#15

#17

#69

```

Verify the frontend and backend through the project's Azure deployment path.

14. Publish Documentation and Evidence

```text

#11

#29

#70

```

Publish documentation and retain Sprint 1 evidence.

15. Review and Close Sprint

```text

#71

```

Conduct the stakeholder review and retrospective before creating the milestone tag.

11. Sprint 2 Planned Backlog

Sprint 2 targets completion of all remaining Basic functionality and the complete Intermediate tier.

Planned work includes:

Remaining Basic Functionality

- remaining public competition and participant APIs;
- remaining administration CRUD operations;
- public browsing improvements;
- administrator management pages;
- JSON and CSV uploads;
- submission history;
- submitter workflow improvements;
- event corrections;
- statistic provenance;
- exports;
- external API integration;
- complete Basic end-to-end acceptance testing;
- accessibility testing;
- security testing; and
- formal user testing.

Intermediate Functionality

- batch submission design;
- staging;
- asynchronous processing;
- accepted/rejected batch reports;
- idempotent processing;
- resumable processing;
- review and publication workflow;
- conflicting-event checks;
- correction history;
- season aggregates;
- career aggregates;
- competition aggregates;
- selective recomputation;

- golden reference results;
- representative-scale performance testing;
- database optimisation;
- explicit API versioning;
- consumer API keys;
- rate limits;
- quotas;
- caching;
- versioned dataset releases;
- release downloads;
- Intermediate integration testing; and
- Intermediate documentation.

These items remain part of the committed roadmap.

They should be converted into or mapped against Gitea issues during Sprint 2 refinement.

They remain in `Backlog` until they satisfy the Definition of Ready.

12. Sprint 3 Planned Backlog

Sprint 3 targets completion of the Advanced feature tier.

The Advanced backlog includes:

- custom-statistic definition language;
- statistic-definition versioning;
- safe statistic execution;
- full-history custom-statistic evaluation;
- custom-statistic APIs;
- custom-statistic frontend;
- statistic provenance;
- correction propagation;
- live event ingestion;
- duplicate live-event handling;
- late and out-of-order event handling;
- deterministic replay;
- temporal history;
- as-of-date statistics;

- dataset release comparison;
- aggregate query APIs;
- asynchronous large-query jobs;
- change feeds;
- API deprecation mechanisms;
- contract compatibility testing;
- consumer usage reporting;
- anomaly detection;
- conflicting-submitter reconciliation;
- downstream correction propagation;
- Advanced correctness and performance testing;
- production observability;
- application-wide accessibility and UX polish;
- security and recovery review;
- Advanced technical documentation; and
- stakeholder acceptance testing.

These items remain in the project roadmap but should not be marked `Ready` until they have been refined into actionable issues.

13. Final Submission Planned Backlog

The Final Submission phase includes:

1. triage and resolve release-blocking defects;
2. verify production migrations, backups and restore procedures;
3. verify production secrets and deployment configuration;
4. complete performance, security, accessibility and browser audits;
5. finalise requirements traceability and the known-limitations register;
6. finalise database, third-party, licence, testing and AI documentation;
7. prepare a reproducible demonstration dataset and scripted demonstration;
8. prepare the group presentation;
9. complete group report, individual reports and peer-review evidence; and
10. produce release notes, the final version tag and submission package.

The Final Submission period is not intended to become a fourth major feature-development sprint.

14. Backlog Maintenance Rules

The backlog is a living planning artifact.

When stakeholder feedback or implementation discoveries change the required work:

1. update the relevant Gitea issue where possible;
2. create a new issue where the work is genuinely separate;
3. update issue dependencies;
4. reprioritise the Project board where necessary;
5. record important requirement changes;
6. update architecture or ADR documentation where technical decisions change; and
7. update this document where milestone scope changes materially.

Requirements must not be changed silently.

Duplicate issues must not be created merely to make the catalogue appear complete.

Where one existing issue legitimately satisfies multiple catalogue items, that relationship should be documented.

Where an issue becomes too large to review or complete comfortably within a sprint, it should be split into smaller issues.

15. Remaining Work for Issue #36

Issue #36 represents the creation and prioritisation of the overall project backlog.

The current checkpoint has established:

- [x] the four delivery-period outcomes;
- [x] the project prioritisation rules;
- [x] the existing project-foundation issue mapping;
- [x] the Sprint 1 catalogue;
- [x] the current Sprint 1 Gitea issue mappings;
- [x] the dependency order for Sprint 1;

- [x] the Basic vertical-slice trace;
- [x] the Sprint 2 roadmap;
- [x] the Sprint 3 roadmap;
- [x] the Final Submission roadmap;
- [x] current Supabase Auth terminology;
- [x] testing, deployment and documentation close-out work;
- [] S1-02 has a verified dedicated Gitea issue;
- [] S1-03 has a verified dedicated Gitea issue;
- [] issue #66 has been renamed to remove legacy Firebase terminology;
- [] Sprint 2 issues are fully refined and mapped before Sprint 2 begins;
- [] Sprint 3 issues are fully refined and mapped before Sprint 3 begins;
- [] Final Submission tasks are mapped before final release planning;
- [] milestone allocation and ownership are reviewed by the team.

Until the remaining roadmap is refined, documentation Pull Requests updating this planning artifact should reference:

```
```text
```

```
Refs #36
```

```
```
```

rather than automatically closing the backlog issue.

```
---
```

16. Review Cadence

The backlog should be reviewed:

- during Sprint planning;
- after significant stakeholder feedback;
- during standups when priorities or blockers change;
- before moving work from `Backlog` to `Ready`;
- when an architecture decision materially changes planned work; and
- during sprint close-out.

The planning document does not need to be rewritten after every small issue-status change.

Routine workflow status remains on the Gitea Project board.

```
---
```

AI Declaration

The preceding document was planned, generated and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

The project team remains responsible for verifying all issue mappings, milestone allocations, requirements, dependencies and prioritisation decisions recorded in this document.

There are now only **two Sprint 1 catalogue mappings I would leave unresolved**: `s1-02` and `s1-03`, because you haven't given me verified issue numbers for those and I don't want to invent #40, #41 or #42.

Also fix **#66** when you get a chance:

OLD:

Implement secure account deletion across Firebase and application data

NEW:

Implement secure account deletion across Supabase Auth and application data

After saving this, the next file we should make is the **Sprint 1 planning evidence file**, and then we can do the commit + full #36 PR properly.

8/7/2026, 1:56:04 PM



ok lets do it

8/7/2026, 1:58:33 PM



Thinking

- **Drafting sprint evidence**

I'm preparing a grounded sprint-planning record using the documented backlog and methodology. I'll avoid inventing attendees or decisions, leaving only clearly marked placeholders where the project record needs confirmation.

- **Created lightweight Scrumban sprint planning evidence and methodology**

✓ Worked for 19s

Yep. Next create the **Sprint 1 planning evidence record**. Your current methodology requires a short repository record containing the sprint, date, attendees, goal, selected work, risks/dependencies, decisions and unresolved questions.

Project_Methodology_v1.1.md

Create:

evidence/sprints/sprint-1/2026-08-07-planning.md

Use this. I have deliberately left the attendees as a placeholder rather than inventing who was present.

Sprint 1 Planning

****Sprint:**** Sprint 1

****Date:**** 7 August 2026

****Attendees:**** [Add names of team members who participated in Sprint 1 planning]

Sprint Goal

Deliver and demonstrate a working ****Basic vertical slice**** of the Sport

Analytics Tool while establishing the technical, governance, deployment, testing and documentation foundations required for later sprints.

The target end-to-end workflow is:

```
```text
administrator approves submitter
 ↓
approved submitter signs in
 ↓
submitter sends T20 cricket delivery events
 ↓
backend verifies identity, role and permitted scope
 ↓
events are validated
 ↓
invalid events receive useful rejection messages
 ↓
valid events are stored with provenance
 ↓
statistics are derived from accepted events
 ↓
public users view fixtures, events and statistics
```
```

Public browsing must remain available without authentication. Authentication is required only for account-specific and authorised submission or administration functionality.

Selected Work

Sprint 1 includes the following work areas and Gitea issues.

1. Repository Governance and Development Process

- #2 - Add Git and project methodology documentation.
- #7 - Align repository governance and document initial setup deviation.
- #8 - Protect `main` and require Pull Request approval.
- #10 - Restore green CI and reproducible installation.
- #12 - Validate developer onboarding and motivate the technology stack.

- #30 - Create per-team-member folders for AI transcripts.
- #36 - Complete and prioritise the project backlog.

2. Domain and Architecture

- #37 - Confirm the sport and domain definition.
- #38 - Produce the complete system architecture, data flow and development roadmap.
- #55 - Record architecture decisions for caching, background jobs, file storage and live ingestion.
- #56 - Produce information architecture, user journeys and responsive wireframes.
- #57 - Define the accessible design system and reusable frontend component baseline.
- #58 - Define stable identifiers and shared API response, error, filtering, sorting and pagination contracts.
- #59 - Create the OpenAPI baseline and API versioning and deprecation conventions.
- #73 - Align architecture roadmap and authentication terminology.

3. Database and Data Foundation

- #13 - Establish the Supabase PostgreSQL foundation.
- #21 - Resolve the database merge conflict.
- #27 - Design and implement the delivery event schema.
- #28 - Create the Cricsheet T20 dataset downloader.
- #43 - Implement the application account, role, approved-submitter and scope database schema.
- #46 - Implement the competition, season, competitor, participant and fixture database schema.
- #60 - Implement repeatable database migrations and development seed tooling.
- #61 - Implement the submission, ordered-event, provenance and correction database schema.
- #62 - Implement the backend database repository and transaction foundation.
- #63 - Create a representative development seed and reference fixture dataset.

4. Authentication, Accounts and Authorisation

- #14 - Investigate and establish the OAuth/OIDC authentication foundation.
- #39 - Build frontend authentication pages and navigation controls.
- #44 - Implement account synchronisation, profile API and role-based authorisation.

- #45 - Build administrator submitter approval and scope management.
- #64 - Implement frontend authentication state, protected routes and the authenticated API client.
- #65 - Build the forgotten-password and password-reset flow.
- #66 - Implement secure account deletion across authentication and application data.

The current architecture uses ****Supabase Auth with Google OAuth****.

Any remaining Firebase terminology in Sprint 1 issues must be updated before implementation. Firebase is not to be implemented as a parallel authentication provider.

5. Event Contract, Validation and Submission

- #49 - Define the executable delivery submission schema and validation examples.
- #50 - Implement event validation with detailed rejection messages.
- #51 - Implement approved-submitter event submission with provenance and scope enforcement.
- #53 - Build the approved-submitter event upload and validation-results interface.

6. Public API and Frontend

- #47 - Implement public competition, fixture and participant read APIs.
- #48 - Build public competition, fixture and participant browsing pages.
- #52 - Implement event-derived fixture statistics and the public statistics API.
- #54 - Build public fixture and competitor statistics pages.
- #67 - Implement public ordered-event read APIs with filtering and pagination.

7. Testing

- #68 - Add backend and frontend tests for the Basic vertical slice.
- #72 - Complete the automated testing foundation and test environment setup.

Testing must cover both successful and rejected workflows, including authentication, authorisation, validation and public access.

8. Deployment and Documentation

- #11 - Deploy the public documentation site with Cloudflare Pages.
- #15 - Establish the Azure backend deployment foundation.
- #17 - Establish the Azure frontend deployment foundation.
- #29 - Deploy MkDocs documentation to Cloudflare Pages using Wrangler.
- #69 - Repair Azure deployment workflows for the current monorepo and add deployed smoke checks.
- #70 - Publish Sprint 1 setup, architecture, API and testing documentation and evidence.

9. Sprint Close-Out

- #71 - Conduct the Sprint 1 stakeholder review, retrospective and milestone tag.

Only work that satisfies the project's Definition of Done may be reported as completed at Sprint 1 close-out.

Priority and Dependency Order

Sprint 1 work will not be completed simply in numerical issue order.

The agreed dependency sequence is:

```

```text
governance + CI + domain decisions + architecture
 ↓
database + authentication foundations
 ↓
persistent domain + account model
 ↓
submitter approval and authorisation
 ↓
delivery contract and validation
 ↓
approved event submission
 ↓
event persistence with provenance
 ↓
public event APIs
 ↓

```

event-derived statistics

↓

public frontend

↓

automated tests + deployed smoke checks

↓

documentation + sprint close-out

...

Work that blocks the Basic vertical slice receives priority over non-blocking enhancements.

---

## ## Key Risks and Dependencies

### ### Authentication Approval

Supabase Auth with Google OAuth is the team's selected and implemented authentication foundation.

The written lecturer approval currently recorded in the repository explicitly covers Supabase-hosted PostgreSQL but does not separately confirm Supabase Auth.

Any formal confirmation required for production use must therefore remain visible as an open compliance item.

### ### Authentication Terminology

Some earlier planning material and issue text may still refer to Firebase.

ADR-004 and the current architecture supersede the earlier Firebase direction.

Remaining Firebase terminology must be corrected rather than implementing two authentication systems.

### ### Database Dependencies

Event submission, provenance and statistic derivation depend on stable database entities, migrations and backend repository behaviour.

Database work must therefore be reviewed before dependent ingestion and statistic features are treated as complete.

### ### Event Contract Dependencies

The executable submission contract in #49 must remain consistent with the relational event model originating from #27 and the submission/provenance schema in #61.

The external submission contract and internal relational database schema serve different purposes and should not be treated as duplicate work.

### ### Deployment

The Azure frontend and backend foundations exist, but deployment workflows must match the current monorepo paths and be verified using deployed smoke checks.

### ### Testing

Feature work depends on #72 establishing a reliable automated testing environment.

New feature issues must still contain feature-specific tests rather than relying only on the general testing-foundation issue.

### ### CI

The repository quality gate must remain usable throughout Sprint 1.

Unrelated existing formatting or repository problems should be recorded and assigned rather than silently mixed into unrelated feature Pull Requests.

### ### Scope

Sprint 1 contains a large amount of work.

If an issue cannot reasonably be completed within the sprint, it must be split, reprioritised or returned to the backlog rather than being reported as partially complete.

---

## ## Decisions

### ### Sport

The project will focus on **\*\*T20 cricket\*\***.

Detailed professional competition scope, cricket terminology and required statistics are owned by #37.

### ### Architecture

The system will maintain separate frontend and backend applications.

The frontend will communicate with application data through the handwritten backend API rather than directly using a generated database API.

### ### Database

The project uses **\*\*Supabase-hosted PostgreSQL\*\*** as the managed PostgreSQL platform.

### ### Authentication

The current authentication architecture is:

```
```text
React frontend
  ↓
Supabase Auth
  ↓
Google OAuth
  ↓
Supabase access token
  ↓
handwritten Express API
  ↓
application account, role, approval and scope
```
```

Authentication and authorisation are separate responsibilities.

Supabase Auth establishes identity.

The Sport Analytics backend determines application permissions.

### ### Application Identity

Application users will be identified using the provider-neutral combination:

```
```text
(auth_provider, provider_subject)
```
```

This prevents competition, event and statistic data from becoming coupled to a particular authentication provider.

### ### Public Access

Public users will be able to browse supported competition, fixture, event and statistic information without signing in.

Authentication will be required for functions such as:

- account management;
- authorised event submission;
- submitter-specific functionality; and
- administration.

### ### Event Data

Delivery events are the authoritative source from which supported cricket statistics are derived.

Statistics must not become an independent conflicting source of truth.

### ### Provenance

Accepted event data must retain sufficient provenance to trace:

```
```text
statistic
  ↓
event
  ↓
```

```
submission
  ↓
submitter/source
...
```

Project Management

The project will continue using the approved lightweight Scrumban methodology.

Gitea remains the authoritative source for issue state, assignment, milestones and Pull Request progress.

The repository will retain sprint-planning and other methodology evidence.

Sprint Targets

The agreed delivery targets are:

```
```text
```

Sprint 1

Working Basic vertical slice

Sprint 2

Complete remaining Basic + Intermediate

Sprint 3

Complete Advanced functionality

Final Submission

Hardening, evidence, documentation and release

```
```
```

```
---
```

Expected Sprint 1 Outcome

Sprint 1 will be considered successful when the team can demonstrate a coherent Basic path in which:

1. an administrator can manage an approved submitter and permitted scope;
2. the submitter can authenticate;
3. the backend can resolve the authenticated application account;

4. an approved submitter can submit valid T20 delivery data;
5. unauthorised or out-of-scope submissions are rejected;
6. invalid events receive useful validation feedback;
7. accepted events are persisted with provenance;
8. fixture statistics are derived from accepted events;
9. public event and statistic information can be retrieved without authentication;
10. the principal workflow is covered by automated tests;
11. frontend, backend and documentation deployment paths are verified; and
12. the implemented state is documented and reviewed.

Unresolved Questions

The following questions remain open at this planning checkpoint:

1. Does the stakeholder or lecturer require separate written approval specifically for Supabase Auth, in addition to the existing approval for Supabase-hosted PostgreSQL?
2. What exact professional T20 competition scope will be supported? This is owned by #37.
3. What exact Basic statistics must be demonstrated during Sprint 1? This is owned by #37.
4. What decisions from #55 will require separate ADRs before implementation?
5. What final file-storage approach will be required once upload and dataset-release requirements are implemented?
6. Which external cricket API will ultimately satisfy the mandatory external API integration requirement?
7. Which issues, if any, need to be split further after team ownership and capacity are reviewed?
8. Are any Sprint 1 issues blocked by stakeholder decisions that should remain in `Backlog` rather than `Ready`?
9. What dedicated issue will own the formal requirements traceability matrix and requirement change log?
10. What dedicated issue will own clarification and recording of the remaining Basic, Intermediate and Advanced acceptance decisions?

Unresolved questions must be converted into Gitea issues, issue updates or documented decisions when they affect implementation or project scope.

Planning Follow-Up

Following this planning checkpoint, the team must:

- verify issue owners;
- verify the Sprint 1 milestone assignment for selected work;
- check that issues moved to `Ready` satisfy the Definition of Ready;
- leave insufficiently refined work in `Backlog`;
- make blockers visible;
- update issue dependencies where required;
- continue moving work through the Gitea Project board;
- review progress during the Thursday standup; and
- record stakeholder decisions following the Tuesday stakeholder meeting.

Related Gitea Issues

Primary planning issue:

```
```text
#36
```
```

Architecture and roadmap:

```
```text
#38
#73
```
```

Domain definition:

```
```text
#37
```
```

Testing foundation:

```
```text
#72
```



```

The full Sprint 1 issue mapping is maintained in:

```text

docs/planning/project-backlog.md

```

AI Declaration

The preceding Sprint 1 planning record was structured and drafted with the assistance of ChatGPT-Web[GPT-5.6 Sol].

The project team remains responsible for verifying that this record accurately reflects the planning meeting, attendees, issue selection, decisions, risks and unresolved questions before it is committed.

Before committing, **fill in the real attendees and remove/change anything that was not actually agreed by the team**. That matters because this is methodology evidence, not just planning documentation; the repository activity and records are supposed to demonstrate the process actually followed. `Project_Methodology_v1.1.md`

Once saved, we can do the `mkdocs.yml` **entry + formatting/checks + commit + proper #36 PR**.

8/7/2026, 1:58:55 PM



next

8/7/2026, 2:04:51 PM



Thinking

- **Updated MkDocs navigation and finalized PR #36**

✓ Worked for 20s

Next, we need to add the new backlog document to the MkDocs navigation, but I want to match your existing `mkdocs.yml` exactly rather than guessing its structure.

Run:

```
Get-Content mkdocs.yml
```

Send me the output. Then I'll give you the exact lines to add for:

```
docs/planning/project-backlog.md
```

We'll keep the Sprint planning evidence file under `evidence/` rather than publish it automatically unless your existing MkDocs structure indicates otherwise.

8/7/2026, 2:05:14 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content mkdocs.yml
site_name: Sport Analytics Tool
site_description: Public documentation for the COMS3011A Sport Analytics Tool
repo_name: Git Push Pray / Sport Analytics Tool
repo_url: https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool
```

theme:

name: material

features:

- navigation.tabs
- navigation.sections
- content.code.copy

nav:

- Home: index.md
- Getting Started:
 - Local Setup: development/setup.md
 - Repository Structure: architecture/repository-structure.md
- Testing: development/testing.md
- Dependencies: development/dependencies.md
- Design:
 - Architecture: architecture/overview.md
 - System Architecture and Roadmap: architecture/system-architecture.md
- API: api/overview.md
- Database: database/overview.md
- Event Model: database/schema.md
- Security:
 - Overview: security/overview.md
 - Authentication Foundation: security/authentication.md
- Provider Comparison: security/auth-provider-comparison.md
- Deployment:
 - Overview: deployment/overview.md

```

- Azure Investigation: deployment/azure.md
- Methodology:
  - Git Methodology: git-methodology.md
  - Project Methodology: project_methodology.md
- AI Use: ai/usage.md

markdown_extensions:
  - admonition
  - attr_list
  - tables
  - toc:
      permalink: true
  - pymdownx.details
  - pymdownx.highlight
  - pymdownx.superfences:
      custom_fences:
        - name: mermaid
          class: mermaid
          format:
            !!python/name:pymdownx.superfences.fence_code_format

plugins:
  - search
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>

```

8/7/2026, 2:05:46 PM



Add the backlog page under **Methodology**.

In `mkdocs.yml`, replace this:

```

- Methodology:
  - Git Methodology: git-methodology.md
  - Project Methodology: project_methodology.md

```

with:

```

- Methodology:
  - Git Methodology: git-methodology.md

```

- Project Methodology: `project_methodology.md`
- Project Backlog and Milestone Plan: `planning/project-backlog.md`

Leave the Sprint 1 planning evidence file out of MkDocs navigation for now.

Then run:

```
mkdocs build --strict
```

Send me the result, and then we'll do the final checks and commit.

8/7/2026, 2:05:51 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> mkdocs build --strict
```

```
| ⚠ Warning from the Material for MkDocs team
|
| MkDocs 2.0, the underlying framework of Material
for MkDocs,
| will introduce backward-incompatible changes,
including:
|
| × All plugins will stop working – the plugin system
has been removed
| × All theme overrides will break – the theming
system has been rewritten
| × No migration path exists – existing projects cannot
be upgraded
| × Closed contribution model – community members
can't report bugs
| × Currently unlicensed – unsuitable for production
use
|
| Our full analysis:
|
| https://squidfunk.github.io/mkdocs-
material/blog/2026/02/18/mkdocs-2.0/
```

```
INFO - Cleaning site directory
```

```
INFO - Building documentation to directory:
C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\site
```

```
INFO - The following pages exist in the docs
directory, but are not included in the "nav"
configuration:
```

- environment.md
- adr\0003-azure-hosting.md
- data\cricsheet.md
- deployment\azure-backend.md
- deployment\azure-fronted.md
- deployment\cloudflare_pages.md

INFO - Documentation built in 1.38 seconds
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>

8/7/2026, 2:07:39 PM



Great — the MkDocs build passed.

Now clean the generated `site/` output, format only the files from #36, and verify the diff:

```
git restore --worktree -- site
git clean -fd -- site

npx.cmd prettier --write docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-07-planning.md mkdocs.yml

git diff --check
git status --short
```

Your final status should show only these three files:

```
M  mkdocs.yml
?? docs/planning/project-backlog.md
?? evidence/sprints/sprint-1/2026-08-07-planning.md
```

If that looks right, we can commit immediately after.

8/7/2026, 2:07:50 PM



yuo (note its 2026-08-06 not 07 so i changed it

8/7/2026, 2:09:30 PM



Perfect — use the corrected evidence filename:

```
evidence/sprints/sprint-1/2026-08-06-planning.md
```

Now commit the #36 work:

```
git add mkdocs.yml docs/planning/project-backlog.md evidence/sprints/sprint-1/2026-08-06-planning.md
```

```
git commit -m "docs(planning): add project backlog and sprint 1 planning evidence" -m "Refs #36"
```

```
git push -u origin docs/36-project-backlog
```

Then create the PR with:

Title

```
docs(planning): add project backlog and sprint 1 planning evidence
```

Description

Related Issue

Refs #36

Purpose

Add a persistent project backlog and Sprint 1 planning record that aligns the repository with the team's lightweight Scrumban methodology and current milestone targets.

This Pull Request records the current issue mapping, dependency order, Sprint 1 Basic vertical slice, and the planned Sprint 2, Sprint 3 and Final Submission

scope without falsely marking future work as already refined.

Summary of Changes

This Pull Request adds:

```
```text
docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md
```
```

and updates:

```
```text
mkdocs.yml
```
```

Project Backlog

The new backlog document:

- records the four delivery periods;
- documents project prioritisation rules;
- maps existing Gitea issues to Sprint 1 work;
- records current technical and governance foundation issues;
- maps the Basic vertical slice from authentication through event submission, statistics and public access;
- records dependency ordering;
- captures the Sprint 2 Intermediate roadmap;
- captures the Sprint 3 Advanced roadmap;
- reserves Final Submission for hardening and release work;
- reflects the current Supabase Auth architecture;
- identifies remaining refinement work under #36.

Sprint 1 Planning Evidence

The Sprint 1 planning record captures:

- sprint goal;
- selected work;
- relevant Gitea issues;
- dependency order;

- major risks;
- project decisions;
- expected Sprint 1 outcome;
- unresolved questions;
- planning follow-up actions.

The Sprint 1 goal is to deliver a working Basic vertical slice in which:

```
```text
administrator approves submitter
 ↓
approved submitter signs in
 ↓
submitter sends T20 delivery events
 ↓
backend verifies identity, role and scope
 ↓
events are validated
 ↓
valid events are stored with provenance
 ↓
statistics are derived
 ↓
public users retrieve fixtures, events and statistics
```
```

Key Issue Mappings

The planning documentation records relevant existing issues including:

```
```text
#2 Git and project methodology
#7 Repository governance
#8 Main branch protection
#10 CI and reproducible installation
#11 Public documentation site
#12 Developer onboarding and technology stack
#13 Supabase PostgreSQL foundation
#14 OAuth/OIDC authentication foundation
#15 Azure backend foundation
#17 Azure frontend foundation
#21 Database merge conflict
```
```

#27 Delivery event schema
#28 Cricsheet T20 dataset downloader
#29 MkDocs deployment through Cloudflare Pages and Wrangler
#30 AI transcript folders
#36 Backlog planning
#37 Sport and domain definition
#38 System architecture and roadmap
#39 Frontend authentication pages
#43 Account, role, approved-submitter and scope schema
#49 Executable delivery submission schema and validation examples
#72 Automated testing foundation
#73 Architecture roadmap and authentication terminology follow-up
...

The document also maps the newer Sprint 1 implementation issues covering database entities, authorisation, event validation, public APIs, statistics, frontend work, deployment and Sprint close-out.

Scope

Included

- project backlog documentation;
- Sprint 1 planning evidence;
- issue-to-work mapping;
- dependency and priority documentation;
- milestone roadmap documentation;
- MkDocs navigation entry;
- current Supabase Auth terminology;
- AI-use declarations.

Excluded

- implementation of Sprint 1 features;
- creation of all Sprint 2 and Sprint 3 issues;
- changes to authentication implementation;
- database migrations;
- unrelated formatting changes;
- generated MkDocs `site/` output.

Verification

Documentation Build

Run:

```
```powershell
mkdocs build --strict
```
```

Result:

```
```text
Documentation built successfully.
```
```

The build reported informational messages for existing documentation pages that are not currently included in MkDocs navigation. These did not cause the strict build to fail.

Formatting

The files changed by this Pull Request were formatted using:

```
```powershell
npx prettier --write docs/planning/project-backlog.md evidence/sprints/sprint-1/2026-08-06-planning.md mkdocs.yml
```
```

Diff Check

Run:

```
```powershell
git diff --check
```
```

Result:

```
```text
Passed with no whitespace errors.
```
```

Files Changed

```
```text
docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md
mkdocs.yml
```
```

Generated `site/` files are not included.

Risks and Limitations

- #36 is not closed by this Pull Request because Sprint 2, Sprint 3 and Final Submission work still require progressive refinement.
- The backlog document is a planning index; Gitea remains the authoritative source for current status, ownership, acceptance criteria and Pull Request state.
- Future work remains in Backlog until it satisfies the Definition of Ready.
- The Sprint 1 planning evidence must reflect the real meeting and must not contain invented attendees or decisions.
- Any remaining legacy Firebase wording in issues must be corrected because the current architecture uses Supabase Auth.

Reviewer Guidance

Please verify that:

- [] issue mappings are accurate;
- [] Sprint 1 reflects the Basic vertical slice target;
- [] public browsing remains unauthenticated;
- [] authentication and authorisation are separated correctly;
- [] Sprint 2 targets remaining Basic and Intermediate functionality;
- [] Sprint 3 targets Advanced functionality;
- [] Final Submission is reserved mainly for hardening and release work;
- [] future work is not incorrectly marked as refined or complete;
- [] the planning evidence accurately reflects the team's planning process;
- [] no generated `site/` files are included;
- [] no unrelated files are included.

Definition of Done

This Pull Request is ready to merge when:

- [] the backlog mapping has been reviewed by at least one team member;
- [] the Sprint 1 planning record has been checked against the actual planning meeting;
- [] requested review changes have been addressed;
- [] documentation checks pass;
- [] no unrelated files are included;
- [] the Pull Request is approved;
- [] the Pull Request is merged into `main`.

Git Methodology

Branch:

```
```text
docs/36-project-backlog
```
```

Commit:

```
```text
docs(planning): add project backlog and sprint 1 planning evidence
```
```

Issue reference:

```
```text
Refs #36
```
```

This Pull Request intentionally does ****not**** use:

```
```text
Closes #36
```
```

because backlog refinement continues into future sprints.

AI Declaration

The project backlog, Sprint 1 planning record and Pull Request description were prepared with the assistance of ChatGPT-Web[GPT-5.6 Sol].

The project team remains responsible for verifying the issue mappings, sprint-planning evidence, milestone allocation and project decisions recorded in this Pull Request.

After opening it, move **#36 to In Review**, not Done.

8/7/2026, 2:09:37 PM



what was issue 36? does it close or ref? i dont think we done, doesnt 36 include sprint 2 and 3?

8/7/2026, 2:11:41 PM



Thinking

- **Clarifying backlog closure**

I'm distinguishing backlog setup from feature implementation. Issue #36 should remain open until every catalogue item is created, prioritized, assigned to milestones, and mapped; full refinement or implementation of later sprints isn't required. The current PR covers documentation and Sprint 1 only.

- **Clarified backlog closure**

- **Verified Issue #36 scope, priority, and open status**

✓ Worked for 30s

Yes - you are right. **#36 should NOT close yet.**

#36 is the project-wide backlog setup issue: **“Complete and prioritise the project backlog.”** The catalogue says to create/map the complete roadmap, including Sprint 1, Sprint 2, Sprint 3 and Final Submission. Sprint 2 and Sprint 3 issues should exist in **Backlog**, but only be fully refined one sprint ahead.

COMS3011A_Complete_Gitea_Issue_Catalogue.md

So the distinction is important:

- We **do not need to implement** Sprint 2 and Sprint 3 to close #36.
- But we **do need to create/map their backlog issues** before #36 is complete.
- Right now we have mainly mapped/refined Sprint 1 and documented the later roadmap, so #36 is only partially complete.

Your methodology also explicitly has four milestones - Sprint 1, Sprint 2, Sprint 3 and Final Submission - and allows future-sprint items to remain in **Backlog**.

Project_Methodology.md

Therefore, for this PR

Use:

Refs #36

Do NOT use:

Closes #36

And one correction to what I told you before: **keep #36 in** `In Progress`, **not** `In Review`, because the issue itself is not finished. The PR can be reviewed and merged as a partial deliverable while #36 stays open.

When should #36 actually close?

Close #36 once we have done all of this:

- ☒ Existing foundation issues mapped
- ☒ Sprint 1 issues created/mapped
- ☐ Sprint 2 issues created and assigned to Sprint 2 + Backlog
- ☐ Sprint 3 issues created and assigned to Sprint 3 + Backlog
- ☐ Final Submission issues created and assigned to Final Submission + Backlog
- ☐ Duplicates reconciled with existing issues
- ☐ Priorities/tiers/labels applied
- ☐ Overall roadmap checked against the requirements
- ☐ Team ownership/milestone scope reviewed

The catalogue explicitly says “**Create every item marked Create**”, while keeping Sprint 2 and Sprint 3 in Backlog until refinement.

COMS3011A_Complete_Gitea_Issue_Catalogue.md

So our next big job under **#36** is actually to create/map the **Sprint 2 → Sprint 3 → Final Submission issues**. Once that catalogue is represented properly in Gitea, **then we can close #36**, probably still during Sprint 1 because backlog setup is a Sprint 1 foundation task. `Pasted markdown.md`

For the PR you're doing now, everything is therefore correct **as long as it says** `Refs #36` **and does not close it**.

8/7/2026, 2:12:11 PM



should we update the docs with sprint 2 and 3, but not the gittea bc we dont want to clogg it? does that make sense? and then we can create the issue when sprint 1 ends?

8/7/2026, 2:15:56 PM



Thinking

- **Reconciling backlog planning**

Yes, keeping the full roadmap in documentation while creating only Sprint 1 issues is operationally cleaner. However, issue #36 currently implies a complete backlog, so I'd revise its scope or leave it open until later refinement.

- **Reconciled backlog planning**

- **Refined the backlog one sprint ahead and evaluated the strategy**

✓ Worked for 16s

Yes - that makes sense, and I think it is actually cleaner for your Scrumban approach.

Your **documentation should contain the complete roadmap now** - Sprint 1, Sprint 2, Sprint 3 and Final Submission - so you can demonstrate that the project is planned beyond the current sprint. The rubric values a comprehensive development plan and roadmap, and your methodology allows future-sprint work to remain less detailed until it becomes relevant. `project_briefs.pdf` `Project_Methodology.md`

I would **not create 70-100 individual Sprint 2/3 Gitea issues right now just for the sake of having them**. That would make the board noisy and create lots of stale issues before stakeholder feedback and Sprint 1 implementation have clarified what is actually needed. Your methodology explicitly says work can remain in the Backlog when it belongs to a future sprint or still needs investigation, and the catalogue itself says Sprint 2 and Sprint 3 should only be refined one sprint ahead.

`Project_Methodology.md` `COMS3011A_Complete_Gitea_Issue_Catalogue.md`

So I would run it like this:

NOW - Sprint 1

- └─ Full project roadmap documented
- └─ Sprint 1 detailed Gitea issues
- └─ Sprint 2 planned in docs
- └─ Sprint 3 planned in docs
- └─ Final Submission planned in docs

END OF SPRINT 1

- └─ Review what changed
- └─ Review stakeholder feedback
- └─ Create/refine Sprint 2 Gitea issues
- └─ Assign Sprint 2 milestone
- └─ Keep Sprint 3 roadmap in docs

END OF SPRINT 2

- └─ Review what changed
- └─ Create/refine Sprint 3 Gitea issues
- └─ Keep Final Submission roadmap ready

END OF SPRINT 3

- └─ Create/refine Final Submission close-out issues

There is just **one consequence for #36**: if its current wording/acceptance criteria literally require the *complete Gitea backlog* to be created, then we should **not close #36 yet**. We can merge the current documentation PR with:

Refs #36

and keep #36 open.

Alternatively - and I prefer this - we can slightly update #36 so that its definition of “complete backlog” means **complete project roadmap documented, current sprint fully represented in Gitea, future sprints progressively converted to Gitea one sprint ahead**. That matches the methodology better and avoids a cluttered tracker. Any such adjustment should be explicitly recorded rather than silently changing the process, which your methodology requires.

Project_Methodology.md

What I recommend for you

Keep the docs we just made. Do not create all Sprint 2 and 3 Gitea issues now.

Then change the bottom of `project-backlog.md` slightly. Instead of saying:

- [] Sprint 2 issues are fully refined and mapped before Sprint 2 begins;
- [] Sprint 3 issues are fully refined and mapped before Sprint 3 begins;

make the progressive approach explicit:

- [] Sprint 2 work is converted into detailed Gitea issues and refined during Sprint 1 close-out;
- [] Sprint 3 work is converted into detailed Gitea issues and refined during Sprint 2 close-out;
- [] Final Submission work is converted into detailed Gitea issues during Sprint 3 close-out;

And add this immediately underneath:

Future-sprint roadmap items are intentionally retained at planning level until one sprint ahead. This avoids creating stale or premature Gitea issues while preserving a complete project roadmap. Gitea issues will be created and refined when the relevant sprint approaches, incorporating implementation discoveries and stakeholder feedback from the preceding sprint.

That gives you the **best of both worlds**: strong planning evidence without a Gitea board full of premature work.

For this current PR, I would still use `Refs #36`, not `Closes #36`, until we inspect the actual #36 acceptance criteria and decide whether to update them to this progressive-refinement model.

8/7/2026, 2:16:12 PM



ok, so give me step by step on what to do so we can close 36 now, saying its done, just the issues arent created on gitea
i havent made any changes yet

8/7/2026, 2:17:56 PM



Thinking

- **Clarifying issue scope**

I'm treating this as a planning change, not a silent rewrite. I'll document the revised scope in issue 36, preserve the original intent, and then align the catalogue and acceptance criteria.

- **Clarified the issue scope**

- **Refining backlog timing**

I'm distinguishing methodology from catalogue guidance: keeping distant work in Backlog is compatible, but the catalogue needs clarification. I'll first document that decision in issue 36, then update related planning text.

- **Refined backlog timing**

- **Revised Issue #36 scope, documentation, and acceptance criteria**

✓ Worked for 38s

Yes. We can close **#36 now**, but we need to do it transparently.

At the moment, the generated catalogue says **"Create every item marked Create"**, including future Sprint 2/3 work. `COMS3011A_Complete_Gitea_Issue_Catalogue.md` Your actual Scrumban methodology, however, allows future-sprint work to remain unrefined until it becomes relevant, and requires changes in scope/process to be recorded rather than made silently. `Project_Methodology_v1.1.md` `Git_Methodology.md`

So we will define #36 as:

The complete project backlog is planned and prioritised now in repository documentation. Sprint 1 is represented as actionable Gitea issues. Sprint

2, Sprint 3 and Final Submission are fully documented at roadmap level and will be converted into detailed Gitea issues one sprint ahead.

That lets #36 genuinely finish without filling Gitea with ~80 premature issues.

Step 1 - Update Issue #36 in Gitea

Before changing the files, go to **#36** and add this comment:

Backlog Planning Scope Clarification

Following review of the complete project roadmap and the team's lightweight Scrumban methodology, the team has agreed to use ****progressive backlog refinement**** rather than creating every future-sprint Gitea issue at the beginning of Sprint 1.

The complete project scope will still be planned and retained in repository documentation across:

- Sprint 1;
- Sprint 2;
- Sprint 3; and
- Final Submission.

Sprint 1 work is represented by detailed Gitea issues because it is the current development sprint.

Future work will remain documented at roadmap level and will be converted into detailed Gitea issues one sprint ahead:

```text

Sprint 1 close-out

→ create/refine Sprint 2 Gitea issues

Sprint 2 close-out

→ create/refine Sprint 3 Gitea issues

Sprint 3 close-out

→ create/refine Final Submission Gitea issues

```

This approach avoids creating a large number of premature or stale issues

before implementation discoveries and stakeholder feedback are available, while preserving a complete and traceable project roadmap.

For the purposes of #36, the backlog is considered complete when:

- [x] the full project roadmap is documented;
- [x] Sprint 1 work is mapped to actionable Gitea issues;
- [x] Sprint 1 priorities and dependencies are documented;
- [x] Sprint 2 scope is fully documented at roadmap level;
- [x] Sprint 3 scope is fully documented at roadmap level;
- [x] Final Submission scope is fully documented at roadmap level;
- [x] milestone outcomes are defined;
- [x] the progressive refinement approach is documented;
- [x] future issue-creation timing is defined.

Creating the detailed Sprint 2, Sprint 3 and Final Submission Gitea issues is therefore ****future sprint-planning work****, not unfinished work under #36.

The repository planning documentation produced by #36 will be the persistent roadmap, while Gitea remains the source of truth for actionable work once that work is brought into the active planning horizon.

Do not close #36 manually yet. The PR will close it after review and merge, which matches your Git methodology. `Git_Methodology.md`

Step 2 - Update `project-backlog.md`

In:

```
docs/planning/project-backlog.md
```

find the whole section:

```
## 15. Remaining Work for Issue #36
```

and replace that entire section, up to `## 16. Review Cadence`, with:

```
## 15. Completion of Issue #36
```

Issue #36 establishes and prioritises the complete project backlog.

The project backlog is considered complete at two levels:

Current Sprint

Sprint 1 work is represented by detailed, actionable Gitea issues because Sprint 1 is the current implementation period.

These issues are mapped in this document and are progressively moved through:

```
```text
```

Backlog

Ready

In Progress

In Review

Done

```
```
```

according to the project's Definition of Ready and Definition of Done.

Future Sprints

Sprint 2, Sprint 3 and Final Submission work is intentionally retained at roadmap level until one sprint ahead.

The team will use progressive backlog refinement:

```
```text
```

Sprint 1 close-out

→ create and refine Sprint 2 Gitea issues

Sprint 2 close-out

→ create and refine Sprint 3 Gitea issues

Sprint 3 close-out

→ create and refine Final Submission Gitea issues

```
```
```

This avoids creating large numbers of premature Gitea issues that may become stale following implementation discoveries, stakeholder feedback or changed dependencies.

It does not remove future work from the project backlog.

The complete planned scope remains documented in this file and will be converted into actionable Gitea issues when it enters the team's active planning horizon.

Issue #36 Completion Checklist

- [x] four delivery-period outcomes are defined;
- [x] project prioritisation rules are documented;
- [x] existing foundation work is mapped;
- [x] Sprint 1 work is mapped to actionable Gitea issues;
- [x] Sprint 1 dependency and priority order is documented;
- [x] the Basic vertical slice is traceable through the Sprint 1 backlog;
- [x] Sprint 2 scope is documented;
- [x] Sprint 3 scope is documented;
- [x] Final Submission scope is documented;
- [x] current authentication terminology is reflected;
- [x] testing, deployment and documentation close-out work is represented;
- [x] progressive future-sprint refinement is documented;
- [x] the timing for creation of future Gitea issues is defined.

Issue #36 may therefore be closed once this planning document and its associated Sprint 1 planning evidence have been reviewed and merged.

Future Sprint 2, Sprint 3 and Final Submission issue creation forms part of the relevant sprint-planning and refinement process and does not require #36 to remain open.

Step 3 - Make one small change higher up in the same file

Under:

```
## 2. Planning Approach
```

after:

Sprint 2 and Sprint 3 work may remain at roadmap level until refinement one sprint ahead.

add:

For future sprints, roadmap entries in this document are considered identified backlog work even where a corresponding detailed Gitea issue has not yet been created. Detailed Gitea issues are created when the work enters the one-sprint-ahead planning horizon.

This is important because it makes your meaning of “**backlog**” unambiguous.

Step 4 - Update the PR from `Refs` to `Closes`

Because #36 will now actually satisfy its revised acceptance criteria, your commit can still say:

```
Refs #36
```

That is correct for commits.

But the **Pull Request** should now say:

```
Closes #36
```

Your Git methodology specifically uses `Refs` in commits and `Closes` in the PR that completes an issue. `Git_Methodology.md`

So change the PR's opening from:

```
## Related Issue
```

```
Refs #36
```

to:

```
## Related Issue
```

```
Closes #36
```

And remove any sentence saying:

#36 remains open because Sprint 2 and Sprint 3 issues have not yet been created.

Replace it with:

Issue #36 is completed by establishing the full documented project backlog, mapping the current Sprint 1 work to actionable Gitea issues, and defining the progressive one-sprint-ahead refinement process for future Gitea issues.

Step 5 - Verify, commit and push

After those edits:

```
npx.cmd prettier --write docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md mkdocs.yml

git diff --check

mkdocs build --strict
```

Then clean the generated site:

```
git restore --worktree -- site
git clean -fd -- site
```

Check:

```
git status --short
```

Then commit:

```
git add mkdocs.yml docs/planning/project-backlog.md evidence/sprints/sprint-
1/2026-08-06-planning.md

git commit -m "docs(planning): add complete project backlog and sprint 1
evidence" -m "Refs #36"

git push -u origin docs/36-project-backlog
```

Then open the PR using `Closes #36`.

Once it is reviewed, approved and merged, **#36 can legitimately move to Done and close**. Future Sprint 2/3 issues are not being claimed as created; the documentation explicitly says they are deliberately created one sprint ahead. That is

much more defensible than pretending those issues already exist.

Project_Methodology_v1.1.md

8/7/2026, 2:18:22 PM



noo, but look at the rubric? dont we need a complete backlog documented by end of sprint 1? can we n0t rather:

do sprint 2, 3 etc backlog in the docs (not make issues yet), add something about being reloked at, and then close it (ie, do sprint 2 etc stuff, and roadmap etc, but done physically make oissues yet, and record that)

8/7/2026, 2:20:43 PM



Thinking

- **Planning backlog documentation**

I'm confirming the interpretation: document the complete future backlog in `project-backlog.md`, while creating Gitea issues only when work becomes actionable. I'll outline the exact documentation updates and closure steps for #36.

- **Planned backlog documentation**

- **Clarifying the backlog approach**

I'm aligning the plan with the rubric: document the full S1-Final roadmap in `project-backlog.md`, keep only actionable near-term work as Gitea issues, and clarify that future entries become issues when refined.

- **Clarified the backlog approach**

- **Reviewed rubrics and catalogues, then planned comprehensive sprint backlogs**

✓ Worked for 34s

Yes - that is the stronger approach, and it fits the Sprint 1 rubric much better.

The rubric's **Advanced** level for Sprint 1 gives 20% to **Initial Design & Dev Plan** and specifically expects a **"Comprehensive dev plan, system design, UI design, roadmap."** Sprint 1 itself is meant to establish project setup, remove major blockers, and give the group a strong direction for the rest of the semester. It does **not** say that

every future Sprint 2/3 task must already exist as an individual Gitea issue.

`project_briefs.pdf` `project_briefs.pdf`

So I would do exactly what you are proposing:

Complete the whole backlog in repository documentation now.

Keep detailed Gitea issues for the active planning horizon.

At each sprint close-out, review the documented future backlog and convert the next sprint's items into Gitea issues, adjusting them based on what was learned.

That also gives you evidence that your roadmap existed from Sprint 1, rather than making it look like Sprint 2/3 were invented later.

There is one wording distinction that makes this defensible against your methodology: the Sprint 2/3 entries are **planned backlog items/capabilities**, not yet refined “meaningful units of work.” When they are refined into actual development tasks, your methodology requires them to become Gitea issues with acceptance criteria, assignment, milestone etc. `Project_Methodology_v1.1.md`

What we're going to do

We will make `#36` complete when the repository contains:

- ✓ Full Sprint 1 backlog + real Gitea mappings
- ✓ Full Sprint 2 backlog documented
- ✓ Full Sprint 3 backlog documented
- ✓ Full Final Submission backlog documented
- ✓ Priorities / tiers shown
- ✓ Development roadmap
- ✓ Milestone outcomes
- ✓ Dependency/order explanation
- ✓ Future backlog review/refinement process
- ✓ Record that future entries are not yet Gitea issues
- ✓ Record when those issues WILL be created

Then **#36 genuinely can close.**

The original complete catalogue already contains 40 Sprint 2 items, 28 Sprint 3 items and 10 Final Submission items, so we should put those actual planned items into the

documentation rather than just saying “batch processing, caching, etc.”

COMS3011A_Complete_Gitea_Issue_Catalogue.md

Step 1 - replace the generic future backlog

Open:

```
docs/planning/project-backlog.md
```

Find:

```
## 11. Sprint 2 Planned Backlog
```

We are going to replace **Sections 11, 12 and 13** with the detailed roadmap below.

Do **only this first**:

```
## 11. Sprint 2 Planned Backlog
```

```
### Sprint Goal
```

Complete all remaining Basic functionality and deliver the full Intermediate feature tier.

Sprint 2 work is already identified as part of the project backlog, but these entries are retained as ****planned backlog items**** rather than detailed Gitea issues during Sprint 1.

At Sprint 1 close-out, the team will review these items against:

- completed Sprint 1 work;
- stakeholder feedback;
- technical discoveries;
- unresolved defects;
- changed dependencies; and
- team capacity.

The resulting Sprint 2 work will then be converted into appropriately sized Gitea issues with acceptance criteria, assignees, labels and the Sprint 2 milestone.

Complete Sprint 2 Backlog

| ID | Planned Work | Tier | Priority |
|-------|---|--------------|----------|
| S2-01 | Implement public competition, season, competitor and participant read APIs | Basic | High |
| S2-02 | Implement administrator CRUD APIs for competitions, seasons, competitors, participants and fixtures | Basic | High |
| S2-03 | Build public competition, season, competitor and fixture browsing pages | Basic | Normal |
| S2-04 | Build administrator competition, season, competitor and fixture management pages | Basic | Normal |
| S2-05 | Implement JSON and CSV file-upload submissions through the shared validation pipeline | Basic | High |
| S2-06 | Implement submission list, detail and validation-result APIs | Basic | Normal |
| S2-07 | Build the approved-submitter upload, direct-entry and result interface | Basic | High |
| S2-08 | Implement authorised event correction with automatic statistic refresh | Basic | High |
| S2-09 | Build the approved event-correction interface | Basic | Normal |
| S2-10 | Implement statistic provenance and the how-calculated trace in the API and frontend | Basic | High |
| S2-11 | Implement filtered CSV and JSON dataset exports in the API and frontend | Basic | High |
| S2-12 | Select and document a relevant external API integration | Common | Normal |
| S2-13 | Implement the selected external API integration in the backend and frontend | Common | Normal |
| S2-14 | Complete the full Basic integration and browser-level end-to-end acceptance workflow | Basic | High |
| S2-15 | Complete the Basic accessibility and responsive-design audit and fixes | Common | High |
| S2-16 | Complete the Basic security, privacy and dependency hardening review | Common | High |
| S2-17 | Conduct formal Basic user testing and integrate prioritised feedback | Basic | High |
| S2-18 | Design the batch staging, file storage and processing pipeline | Intermediate | High |
| S2-19 | Implement batch, batch-item and processing-checkpoint database models | Intermediate | High |
| S2-20 | Implement whole-season and back-catalogue batch upload and staging | | |

Intermediate | High |

| S2-21 | Implement asynchronous batch validation and processing | Intermediate | High |

| S2-22 | Generate accepted and rejected batch processing reports | Intermediate | Normal |

| S2-23 | Make batch resubmission idempotent and prevent double counting | Intermediate | High |

| S2-24 | Implement resumable batch processing from durable checkpoints | Intermediate | High |

| S2-25 | Implement submission review and publication workflow in the API and frontend | Intermediate | High |

| S2-26 | Implement impossible and conflicting event validation rules | Intermediate | High |

| S2-27 | Implement immutable correction history and audit logs | Intermediate | High |

| S2-28 | Implement season, career and competition-wide aggregate derivation | Intermediate | High |

| S2-29 | Implement dependency-aware selective recomputation after event changes | Intermediate | High |

| S2-30 | Build golden reference results and automated correctness checks | Intermediate | High |

| S2-31 | Create representative large-scale data and define API response-time targets | Intermediate | High |

| S2-32 | Optimise database indexes, storage layout and query plans under load | Intermediate | High |

| S2-33 | Implement explicit API versioning and compatibility behaviour | Intermediate | High |

| S2-34 | Implement API key issuance, rotation, revocation, rate limits and quotas | Intermediate | High |

| S2-35 | Implement caching for repeated reads with correct invalidation | Intermediate | Normal |

| S2-36 | Implement versioned dataset releases, snapshots, field documentation and checksums | Intermediate | High |

| S2-37 | Build the dataset release catalogue and download experience | Intermediate | Normal |

| S2-38 | Complete Intermediate integration, correctness, performance and API tests | Intermediate | High |

| S2-39 | Publish Intermediate API, database, testing and third-party code documentation | Intermediate | Normal |

| S2-40 | Conduct the Sprint 2 stakeholder review, user feedback round, retrospective and milestone tag | Common | High |

Sprint 2 Expected Outcome

By the end of Sprint 2:

- all remaining Basic requirements should be complete;
- the complete Intermediate feature tier should be operational;
- the system should support bulk event ingestion;
- imports should be resumable and idempotent;
- corrections and review workflows should be traceable;
- season, career and competition aggregates should be available;
- the API should support external consumers using appropriate controls;
- dataset releases should be versioned and reproducible;
- representative-scale performance should have been tested;
- formal user testing should have been conducted; and
- Sprint 2 evidence should be retained.

12. Sprint 3 Planned Backlog

Sprint Goal

Complete the Advanced feature tier and bring the system to a near-complete production-quality state.

Sprint 3 items remain documented during Sprint 1 as planned backlog capabilities.

At Sprint 2 close-out, the team will review and refine this backlog using implementation evidence, stakeholder feedback, performance findings and remaining project risk before creating the detailed Sprint 3 Gitea issues.

Complete Sprint 3 Backlog

| ID | Planned Work | Tier | Priority |
|-------|---|----------|----------|
| S3-01 | Define the safe custom statistic definition language and execution contract | Advanced | High |
| S3-02 | Implement versioned custom statistic definitions and validation | Advanced | High |
| S3-03 | Implement sandboxed and resource-limited custom statistic execution | Advanced | High |

| S3-04 | Evaluate custom statistic definitions across the full event history at scale | Advanced | High |

| S3-05 | Build custom statistic definition, evaluation and result APIs and interfaces | Advanced | High |

| S3-06 | Trace custom statistic results to source events and definition versions | Advanced | High |

| S3-07 | Propagate event corrections into affected custom statistic results | Advanced | High |

| S3-08 | Define and implement the authenticated live fixture event feed | Advanced | High |

| S3-09 | Handle duplicate, late and out-of-order live events deterministically | Advanced | High |

| S3-10 | Implement replayable event processing and prove ordered-feed convergence | Advanced | High |

| S3-11 | Implement temporal history and as-of-date statistic queries and views | Advanced | High |

| S3-12 | Implement dataset release comparison and difference views | Advanced | Normal |

| S3-13 | Implement aggregate query endpoints beyond record retrieval | Advanced | High |

| S3-14 | Implement asynchronous large-query jobs with submission, status and result collection | Advanced | High |

| S3-15 | Implement the event and dataset change feed for delta synchronisation | Advanced | Normal |

| S3-16 | Publish the API deprecation path and implement safe version retirement behaviour | Advanced | High |

| S3-17 | Add automated API contract and backwards-compatibility tests | Advanced | High |

| S3-18 | Implement per-consumer API usage metering and an administrator usage view | Advanced | Normal |

| S3-19 | Implement historical anomaly detection and anomaly review | Advanced | High |

| S3-20 | Implement conflicting-submitter reconciliation and accepted-resolution workflow | Advanced | High |

| S3-21 | Carry accepted corrections through aggregates, caches, custom statistics and releases | Advanced | High |

| S3-22 | Complete advanced end-to-end correctness and representative-scale performance testing | Advanced | High |

| S3-23 | Implement production observability, structured logs, metrics and health checks | Common | High |

| S3-24 | Complete the application-wide accessibility, responsiveness, UX and aesthetic polish | Common | High |

| S3-25 | Complete the advanced security, failure-recovery and dependency review | Common | High |
| S3-26 | Publish the complete advanced architecture, API, analyst and operations documentation | Advanced | High |
| S3-27 | Run the advanced browser-level demonstration and stakeholder acceptance test | Advanced | High |
| S3-28 | Complete the Sprint 3 review, retrospective, evidence pack and milestone tag | Common | High |

Sprint 3 Expected Outcome

By the end of Sprint 3:

- Advanced custom statistics should be supported safely;
- live event ingestion should be supported;
- duplicate, late and out-of-order events should converge correctly;
- event processing should be replayable;
- historical as-of queries should be available;
- dataset releases should be comparable;
- large queries should be supported asynchronously where required;
- consumers should be able to follow change feeds;
- API compatibility and deprecation should be tested;
- usage reporting should be available;
- anomalies and conflicting submissions should be reviewable;
- corrections should propagate through downstream outputs;
- production observability should be available;
- accessibility, security and recovery should be reviewed; and
- stakeholder acceptance should be demonstrated.

13. Final Submission Planned Backlog

Final Submission Goal

Release a stable, demonstrable and well-evidenced final product.

Final Submission is not planned as another major implementation sprint.

Its backlog is primarily concerned with verification, defects, documentation, evidence and release preparation.

At Sprint 3 close-out, these items will be reviewed against the actual state of the system and converted into final Gitea issues.

Complete Final Submission Backlog

| ID | Planned Work | Tier | Priority |
|------|---|--------|----------|
| F-01 | Triage all open defects and resolve every release-blocking issue | Common | High |
| F-02 | Prepare production database migrations, backups and restore verification | Common | High |
| F-03 | Verify production secrets, configuration and frontend, API, worker, cache and storage deployments | Common | High |
| F-04 | Run the final performance, security, accessibility and cross-browser release audits | Common | High |
| F-05 | Complete the final requirements traceability matrix, limitations register and acceptance record | Common | High |
| F-06 | Complete database, third-party code, licence, testing and AI usage documentation | Common | High |
| F-07 | Prepare the reproducible demonstration dataset and scripted product demonstration | Common | High |
| F-08 | Prepare the group presentation and architecture and UX showcase | Common | Normal |
| F-09 | Prepare the group report, individual report and peer-review evidence packs | Common | Normal |
| F-10 | Publish release notes, create the final version tag and assemble the submission package | Common | High |

Final Submission Expected Outcome

The final submission should provide:

- a stable deployed application;
- no unresolved release-blocking defects;
- verified production configuration;
- verified database backup and recovery;
- complete automated and manual testing evidence;
- final security and accessibility evidence;
- final requirements traceability;
- complete public and repository documentation;
- reproducible demonstration data;
- group and individual reports;

- presentation material;
- release notes;
- the final milestone tag; and
- the complete submission package.

14. Future Backlog Refinement

The backlog documented in this file represents the project's complete planned development scope as understood during Sprint 1.

Future sprint entries are deliberately documented before their corresponding detailed Gitea issues are created.

This provides a complete Sprint 1 development plan and roadmap while avoiding premature refinement of implementation tasks whose exact scope may change.

The refinement schedule is:

```text

During Sprint 1

- Sprint 1 work exists as detailed Gitea issues.
- Sprint 2, Sprint 3 and Final Submission remain fully documented here.

Sprint 1 close-out

- Review Sprint 2 backlog.
- Incorporate stakeholder feedback and Sprint 1 discoveries.
- Split, combine, remove or clarify planned items where justified.
- Create detailed Sprint 2 Gitea issues.
- Assign owners, labels and Sprint 2 milestone.

Sprint 2 close-out

- Review Sprint 3 backlog.
- Incorporate Sprint 2 evidence and stakeholder feedback.
- Create detailed Sprint 3 Gitea issues.

Sprint 3 close-out

- Review Final Submission backlog.
- Create detailed Final Submission Gitea issues based on the actual remaining release work.

```

A future backlog item becomes an actionable Gitea issue when it enters the team's active planning horizon and can be defined according to the Definition of Ready.

Any material change to the documented roadmap must record:

- what changed;
- why it changed;
- the stakeholder or technical evidence supporting the change; and
- the effect on milestone scope or priorities.

This allows the project roadmap to evolve without silently changing requirements.

15. Completion of Issue #36

Issue #36 is responsible for establishing and prioritising the complete project backlog and roadmap.

The backlog established under #36 includes both:

1. ****actionable current-sprint work****, represented by detailed Gitea issues; and
2. ****identified future-sprint work****, represented by the documented Sprint 2, Sprint 3 and Final Submission backlog above.

The distinction is intentional.

Future items have been identified and prioritised, but they have not yet been prematurely decomposed into detailed implementation issues.

Issue #36 Completion Checklist

- [x] the complete Sprint 1 backlog is documented;
- [x] Sprint 1 work is mapped to current Gitea issues;
- [x] Sprint 1 priorities and dependencies are documented;
- [x] the complete Sprint 2 planned backlog is documented;
- [x] the complete Sprint 3 planned backlog is documented;
- [x] the complete Final Submission backlog is documented;

- [x] Basic, Intermediate, Advanced and Common work is distinguishable;
- [x] milestone goals and expected outcomes are documented;
- [x] project-wide prioritisation rules are documented;
- [x] the Basic vertical slice is traceable through Sprint 1 work;
- [x] future backlog refinement timing is documented;
- [x] the process for incorporating stakeholder and implementation feedback is documented;
- [x] it is explicitly recorded that detailed future Gitea issues are created one sprint ahead.

Issue #36 is therefore complete once this backlog document and its associated Sprint 1 planning evidence have been reviewed and merged.

Closing #36 does not mean that Sprint 2, Sprint 3 or Final Submission work has been implemented.

It means that the project's complete development backlog and roadmap have been identified, prioritised and documented during Sprint 1.

Future Gitea issue creation and refinement will occur through the normal sprint-planning process described above.

This is much stronger for the rubric because an assessor can literally open your documentation during Sprint 1 and see the **entire development path through final submission**, rather than just three vague paragraphs. That directly supports the “comprehensive dev plan ... roadmap” criterion. `project_briefs.pdf`

It also preserves the catalogue's intended overall targets - Sprint 2 = remaining Basic + Intermediate, Sprint 3 = Advanced, Final = hardening/evidence/release.

`COMS3011A_Complete_Gitea_Issue_Catalogue.md`

Do this replacement first. Don't change #36 in Gitea yet. Once you've pasted it, tell me **done**, and the next step will be a much smaller edit near the top of the document explaining the distinction between a **documented backlog item** and a **refined Gitea issue**.

8/7/2026, 2:21:03 PM



Pasted markdown(3).md

seems wrong? give me the whole doc with the changes needed pls

8/7/2026, 2:24:36 PM



Thinking

- **Planning document**

I'm preparing one clean document that removes duplicated sections and clarifies the roadmap. I'll explain why issue #36 can close while future capabilities remain documented as planned work rather than active tickets.

- **Planned document structure**

- **Clarifying the roadmap**

I'm consolidating the document into one consistent structure, removing duplicate sections, and distinguishing documented future work from actionable Gitea issues so closing #36 remains accurate and auditable.

- **Clarified the roadmap**

- **Updated the project backlog and sprint roadmap using advanced rubric criteria**

✓ Worked for 1m 9s

Yes - the pasted version has become inconsistent because the old Sections 14-16 were left underneath the new Sections 14-15. It currently contains **two Section 14s, two Section 15s, duplicate Final Submission material, and an old "Remaining Work for #36" section that contradicts the new plan to close #36.** Pasted markdown.md

I also checked this against the actual Sprint 1 rubric. For the Advanced band, **Initial Design & Dev Plan is worth 20% and explicitly asks for a "Comprehensive dev plan, system design, ui design, roadmap."** Sprint 1 is intended to establish

the project and give the team a strong direction for the semester. `project_briefs.pdf`

So, yes: I would document the **complete Sprint 1 → Sprint 2 → Sprint 3 → Final Submission backlog now**, but make it explicit that future-sprint entries are planned backlog items and only become detailed Gitea issues during progressive refinement. That keeps the roadmap comprehensive without cluttering Gitea prematurely.

Replace the **entire contents** of `docs/planning/project-backlog.md` with this:

Project Backlog and Milestone Plan

Document Information Details
--- ---
Project Sport Analytics Tool
Related Issue #36
Methodology Lightweight Scrumban
Planning Target Advanced-tier feature completeness by the end of Sprint 3
Final Submission Target Hardening, evidence, documentation and release packaging
Last Updated 7 August 2026
Status Active planning document

1. Purpose

This document records the complete project backlog, milestone allocation, development roadmap and prioritisation approach for the Sport Analytics Tool.

It supports issue #36 by providing a persistent repository record of:

- the agreed project milestones;
- the complete Sprint 1 backlog;
- existing Gitea issues mapped to Sprint 1 work;
- Sprint 1 priorities and dependencies;
- the complete planned Sprint 2 backlog;
- the complete planned Sprint 3 backlog;
- the complete planned Final Submission backlog;
- the relationship between Basic, Intermediate, Advanced and Common work;
- the process for progressive backlog refinement; and
- the rules used when project priorities or requirements change.

This document provides the complete project planning baseline established during Sprint 1.

It does not replace Gitea.

Gitea remains the authoritative source for actionable development work, including:

- current issue status;
- assignees;
- detailed acceptance criteria;
- issue dependencies;
- milestone assignment;
- board position;
- Pull Requests; and
- completion evidence.

This document remains the authoritative repository record of the planned project roadmap and future backlog until future items are refined into detailed Gitea issues.

2. Planning Approach

The project uses a lightweight Scrumban methodology.

The backlog is planned across four formal delivery periods:

```
```text
Sprint 1
Sprint 2
Sprint 3
Final Submission
```
```

Two levels of backlog detail are used.

2.1 Planned Backlog Items

A planned backlog item identifies work that the project expects to complete

during a future delivery period.

Planned backlog items:

- appear in this document;
- have a milestone;
- have a tier;
- have a priority;
- contribute to the overall development roadmap; and
- may still change following stakeholder feedback or technical discoveries.

A planned backlog item does not need to exist as an individual Gitea issue immediately.

2.2 Actionable Gitea Issues

When work enters the active planning horizon, it is refined into an actionable Gitea issue.

An actionable issue includes:

- a clear purpose;
- an expected outcome;
- acceptance criteria;
- identified dependencies;
- required stakeholder clarification;
- an appropriate assignee;
- a scope small enough to manage effectively; and
- an understood verification approach.

Sprint 1 is the current delivery period and is therefore represented primarily by detailed Gitea issues.

Sprint 2, Sprint 3 and Final Submission are documented completely in this roadmap during Sprint 1 but will be converted into detailed Gitea issues progressively, normally one sprint ahead.

This prevents the tracker from being filled with premature issues whose detailed scope may change before implementation begins.

Future work must not be moved to `Ready` merely because it appears in this document.

The Project board remains:

```
```text
```

Backlog

Ready

In Progress

In Review

Blocked

Done

```
```
```

```
---
```

3. Prioritisation Rules

Work is prioritised in the following order:

1. mandatory project requirements;
2. work required for the current milestone;
3. work blocking other issues;
4. stakeholder priorities;
5. high-risk technical work;
6. defects affecting core functionality;
7. testing and documentation supporting completed features; and
8. optional enhancements.

For the Sport Analytics Tool, the highest early priorities are:

- domain and sport definitions;
- event-data modelling;
- submission validation;
- provenance and traceability;
- authentication and authorisation;
- approved-submitter scope;
- API contracts;
- database foundations;
- statistic derivation;
- automated testing;
- deployment; and
- project documentation.

Basic functionality is non-negotiable.

Intermediate functionality is planned for completion by Sprint 2.

Advanced functionality is the Sprint 3 target.

Advanced architecture and technical risks should be considered before Sprint 3 where delaying them would create avoidable rework.

4. Milestone Outcomes

Sprint 1 - Working Basic Vertical Slice and Project Direction

Sprint 1 targets a deployed and documented Basic vertical slice while establishing the design, architecture, methodology and technical foundations required for the rest of the project.

The intended end-to-end workflow is:

```
```text
administrator approves submitter
 ↓
approved submitter signs in
 ↓
submitter sends delivery events
 ↓
backend verifies identity, role and scope
 ↓
events are validated
 ↓
invalid events receive useful rejection messages
 ↓
valid events are stored with provenance
 ↓
statistics are derived from accepted events
 ↓
public users retrieve fixtures, events and statistics
```
```

Sprint 1 must also establish:

- a comprehensive project roadmap;
- system architecture;
- UI and information architecture;
- database foundations;
- API conventions;
- authentication foundations;
- CI and automated testing foundations;
- deployment foundations;
- project methodology evidence; and
- clear direction for Sprint 2, Sprint 3 and Final Submission.

Sprint 2 - Complete Basic and Intermediate

Sprint 2 targets:

- completion of all remaining Basic requirements; and
- completion of the Intermediate feature tier.

The expected outcome is a useful and increasingly stable product supporting:

- complete Basic user flows;
- bulk event ingestion;
- review and publication;
- correction history;
- wider aggregate statistics;
- performance optimisation;
- external API consumers;
- dataset releases;
- formal user testing; and
- Intermediate-level testing and documentation.

Sprint 3 - Advanced Feature Completion

Sprint 3 targets completion of the Advanced feature tier and a near-complete product.

The expected outcome includes:

- safe custom statistics;
- live event ingestion;
- replayable event processing;

- temporal statistics;
- dataset comparisons;
- asynchronous queries;
- change feeds;
- API compatibility controls;
- anomaly detection;
- conflicting-submitter reconciliation;
- correction propagation;
- production observability; and
- advanced quality assurance.

Final Submission - Hardening and Release

Final Submission is not planned as another major feature-development sprint.

The period is reserved primarily for:

- defect resolution;
- release hardening;
- production verification;
- database backup and restore verification;
- final security testing;
- final accessibility testing;
- final performance testing;
- browser compatibility;
- complete documentation;
- requirements traceability;
- known limitation records;
- demonstration preparation;
- reports;
- presentation material;
- release notes;
- final version tagging; and
- submission packaging.

5. Existing Project and Foundation Issues

The following issues already establish important repository, infrastructure, governance and technical foundations.

| Issue | Work |
|-------|---|
| --- | --- |
| #2 | Add Git and project methodology documentation |
| #7 | Align repository governance and document initial setup deviation |
| #8 | Protect `main` and require Pull Request approval |
| #10 | Restore green CI and reproducible installation |
| #11 | Deploy the public documentation site with Cloudflare Pages |
| #12 | Validate developer onboarding and motivate the technology stack |
| #13 | Establish the Supabase PostgreSQL foundation |
| #14 | Investigate and establish the OAuth/OIDC authentication foundation |
| #15 | Establish the Azure backend deployment foundation |
| #17 | Establish the Azure frontend deployment foundation |
| #21 | Resolve database merge conflict |
| #27 | Design and implement the delivery event schema |
| #28 | Create the Cricsheet T20 dataset downloader |
| #29 | Deploy MkDocs documentation to Cloudflare Pages using Wrangler |
| #30 | Create per-team-member folders for AI transcripts |
| #36 | Complete and prioritise the project backlog |
| #37 | Confirm sport and domain definitions |
| #38 | Produce the complete system architecture, data flow and development roadmap |
| #39 | Build frontend authentication pages and navigation controls |
| #43 | Implement the application account, role, approved-submitter and scope database schema |
| #49 | Define the executable delivery submission schema and validation examples |
| #72 | Complete the automated testing foundation and test environment setup |
| #73 | Align architecture roadmap and authentication terminology |

Existing work should not be duplicated merely because a later planning item describes related functionality.

Where a later implementation issue extends an existing foundation issue, the relationship should be recorded as a dependency or predecessor.

6. Sprint 1 Requirement-to-Issue Mapping

The Sprint 1 planning catalogue contains 36 work areas.

The following table records the current mapping between those work areas and

the Gitea issues already created.

Where multiple issues are listed, they collectively satisfy or support the catalogue item.

| Catalogue ID | Sprint 1 Work | Gitea Mapping |
|--------------|---|---|
| --- | --- | --- |
| S1-01 | Confirm sport, professional competition scope, fixture definition, event vocabulary and required statistics | #37 |
| S1-02 | Clarify Basic, Intermediate and Advanced acceptance decisions with the stakeholder | Planning requirement retained for Sprint 1 review |
| S1-03 | Create requirements traceability matrix and requirement change log | Planning requirement retained for Sprint 1 review |
| S1-04 | Produce complete system architecture, data flow, deployment design and development roadmap | #38, follow-up #73 |
| S1-05 | Record architecture decisions for caching, background jobs, file storage and live ingestion | #55 |
| S1-06 | Produce information architecture, user journeys and responsive wireframes | #56 |
| S1-07 | Define accessible design system and reusable frontend component baseline | #57 |
| S1-08 | Define stable identifiers and shared API response, error, filtering, sorting and pagination contracts | #58 |
| S1-09 | Create OpenAPI baseline and API versioning and deprecation conventions | #59 |
| S1-10 | Implement repeatable database migrations and development seed tooling | #60 |
| S1-11 | Implement competition, season, competitor, participant and fixture database schema | #46 |
| S1-12 | Implement application account, role, approved-submitter and scope database schema | #43 |
| S1-13 | Implement submission, ordered-event, provenance and correction database schema | #61, building on #27 |
| S1-14 | Implement backend database repository and transaction foundation | #62 |
| S1-15 | Create representative development seed and reference fixture dataset | #63, supported by #28 |
| S1-16 | Configure managed authentication provider and account authentication foundation | #14 |
| S1-17 | Implement account synchronisation, profile API and role-based authorisation | #44 |
| S1-18 | Implement reusable authentication, role, approval and scope | |

authorisation | #44, with submission scope enforcement in #51 |
| S1-19 | Implement frontend authentication state, protected routes and authenticated API client | #64 |
| S1-20 | Build Create Account, Sign In, Account, Sign Out and authentication navigation | #39 |
| S1-21 | Build accessible sign-up and sign-in forms | #39, supported by #57 |
| S1-22 | Build forgotten-password and password-reset flow | #65 |
| S1-23 | Implement secure account deletion across authentication and application data | #66 |
| S1-24 | Implement administrator APIs for submitter approval, revocation and scope assignment | #45 |
| S1-25 | Build administrator submitter approval and scope-management interface | #45 |
| S1-26 | Define executable delivery submission schema and valid/invalid validation examples | #49 |
| S1-27 | Implement event validation with detailed and structured rejection messages | #50 |
| S1-28 | Implement approved-submitter event submission with provenance and scope enforcement | #51 |
| S1-29 | Implement public fixture and ordered-event APIs with filtering and pagination | #47 and #67 |
| S1-30 | Implement Basic event-derived fixture statistics and public statistics API | #52 |
| S1-31 | Build first public competition, fixture, event and statistics browsing experience | #48 and #54 |
| S1-32 | Add backend and frontend tests for the Basic vertical slice | #68, supported by #72 |
| S1-33 | Repair Azure deployment workflows for the current monorepo and add deployed smoke checks | #69, building on #15 and #17 |
| S1-34 | Automate MkDocs deployment from Gitea to Cloudflare Pages | #29, building on #11 |
| S1-35 | Publish Sprint 1 setup, architecture, API, testing documentation and evidence | #70 |
| S1-36 | Conduct Sprint 1 stakeholder review, retrospective and milestone tag | #71 |

S1-02 and S1-03 remain explicit Sprint 1 planning obligations even though a dedicated Gitea issue number has not yet been recorded for them in this document.

Their absence from the mapping does not remove the requirements.

They must be reviewed during Sprint 1 close-out and either:

- mapped to an existing issue or planning record; or
- converted into a dedicated Gitea issue if meaningful implementation or documentation work remains.

7. Sprint 1 Implementation Areas

Account, Authentication and Authorisation

| Issue | Work |
|-------|--|
| --- | --- |
| #14 | OAuth/OIDC authentication foundation |
| #39 | Frontend authentication pages and navigation controls |
| #43 | Account, role, approved-submitter and scope database schema |
| #44 | Account synchronisation, profile API and role-based authorisation |
| #45 | Administrator submitter approval and scope management |
| #64 | Frontend authentication state, protected routes and authenticated API client |
| #65 | Forgotten-password and password-reset flow |
| #66 | Secure account deletion across authentication and application data |

Before #66 begins, its Gitea title and description must be checked for legacy Firebase terminology.

If Firebase remains in the issue wording, it should be changed to:

```
```text
Implement secure account deletion across Supabase Auth and application data
```
```

Firebase must not be implemented alongside Supabase Auth.

Database and Domain

| Issue | Work |
|-------|------------------------------------|
| --- | --- |
| #13 | Supabase PostgreSQL foundation |
| #21 | Database merge-conflict resolution |
| #27 | Delivery event schema |

| | |
|-----|---|
| #28 | Cricsheet T20 dataset downloader |
| #43 | Application account, role, approved-submitter and scope schema |
| #46 | Competition, season, competitor, participant and fixture schema |
| #60 | Repeatable database migrations and development seed tooling |
| #61 | Submission, ordered-event, provenance and correction schema |
| #62 | Backend database repository and transaction foundation |
| #63 | Representative development seed and reference fixture dataset |

Event Submission and Validation

| | |
|-------|---|
| Issue | Work |
| --- | --- |
| #49 | Define executable delivery submission schema and validation examples |
| #50 | Implement event validation with detailed rejection messages |
| #51 | Implement approved-submitter event submission with provenance and scope enforcement |
| #53 | Build approved-submitter event upload and validation-results interface |

Public API, Statistics and Frontend

| | |
|-------|--|
| Issue | Work |
| --- | --- |
| #47 | Public competition, fixture and participant read APIs |
| #48 | Public competition, fixture and participant browsing pages |
| #52 | Event-derived fixture statistics and public statistics API |
| #54 | Public fixture and competitor statistics pages |
| #67 | Public ordered-event read APIs with filtering and pagination |

Architecture, API Design and UX

| | |
|-------|--|
| Issue | Work |
| --- | --- |
| #38 | Complete system architecture, data flow and roadmap |
| #55 | Architecture decisions for caching, background jobs, file storage and live ingestion |
| #56 | Information architecture, user journeys and responsive wireframes |
| #57 | Accessible design system and reusable frontend component baseline |
| #58 | Stable identifiers and shared API response, error, filtering, sorting and pagination contracts |
| #59 | OpenAPI baseline and API versioning/deprecation conventions |
| #73 | Architecture roadmap and authentication terminology alignment |

Testing, Deployment and Evidence

| Issue | Work |
|-------|---|
| --- | --- |
| #10 | Restore green CI and reproducible installation |
| #11 | Deploy public documentation site |
| #15 | Azure backend deployment foundation |
| #17 | Azure frontend deployment foundation |
| #29 | MkDocs deployment through Cloudflare Pages and Wrangler |
| #68 | Basic vertical-slice backend and frontend tests |
| #69 | Repair Azure workflows and add deployed smoke checks |
| #70 | Sprint 1 setup, architecture, API, testing documentation and evidence |
| #71 | Sprint 1 stakeholder review, retrospective and milestone tag |
| #72 | Automated testing foundation and test environment setup |

8. Authentication Terminology

The project originally investigated Firebase Authentication.

The current architecture uses:

```
```text
Supabase Auth
 +
Google OAuth
```
```

ADR-004 supersedes the earlier Firebase direction.

Future backlog items and Gitea issues must therefore use current Supabase Auth terminology and must not introduce Firebase as a parallel authentication implementation unless a later approved ADR explicitly supersedes ADR-004.

The authentication flow is:

```
```text
React frontend
 ↓
Supabase Auth
```
```

```
↓
Google OAuth
↓
Supabase access token
↓
Handwritten Express API
↓
Application account, role, approval state and scope
...
```

Authentication and application authorisation remain separate.

Supabase Auth proves identity.

The Sport Analytics backend determines:

- application account state;
- role;
- approved-submitter status;
- competition, season or fixture scope; and
- administrator permissions.

Application identity remains provider-neutral through:

```
```text
(auth_provider, provider_subject)
```
```

The written lecturer approval currently stored in the repository explicitly covers Supabase-hosted PostgreSQL but does not separately confirm Supabase Auth.

Supabase Auth is therefore the team's selected and implemented authentication foundation, while any required formal confirmation remains an open compliance decision.

9. Sprint 1 Dependency Order

Sprint 1 work should be pulled according to dependencies rather than simply by issue number.

Priority A - Governance and Technical Foundations

Relevant issues include:

```text

#2

#7

#8

#10

#12

#13

#14

#15

#17

#21

#27

#28

#29

#30

#36

#37

#38

#55

#56

#57

#58

#59

#60

#72

#73

```

These establish:

- repository governance;
- CI;
- development setup;
- architecture;
- domain definitions;
- database connectivity;
- event modelling;

- authentication;
- API conventions;
- testing;
- deployment; and
- documentation infrastructure.

Priority B - Persistent Domain and Identity Model

Relevant issues include:

```
```text
#43
#46
#61
#62
#63
```
```

These establish the database structures and transaction foundations required by the product vertical slice.

Priority C - Authentication and Approval Flow

Relevant issues include:

```
```text
#39
#44
#45
#64
#65
#66
```
```

The required flow is:

```
```text
user authenticates
 ↓
application account is resolved
 ↓
role is determined
```
```

↓
administrator approves submitter
↓
submitter receives allowed scope
...

Priority D - Event Submission Vertical Slice

Relevant issues include:

```text  
#49  
#50  
#51  
#53  
...`

The required flow is:

```text  
approved submitter
↓
delivery submission
↓
schema validation
↓
sport-rule validation
↓
scope authorisation
↓
structured rejection or acceptance
↓
provenance-backed persistence
...`

Priority E - Public Read and Statistics Vertical Slice

Relevant issues include:

```text  
#47  
#48

#52  
#54  
#67  
```

The required public flow is:

```
```text
public user
 ↓
competition / fixture discovery
 ↓
ordered event viewing
 ↓
derived statistics
```
```

Public read functionality must not require authentication.

Priority F - Verification and Sprint Close-Out

Relevant issues include:

```
```text
#68
#69
#70
#71
```
```

Before Sprint 1 closes:

- the Basic vertical slice must be covered by automated tests;
- deployment must be verified;
- documentation must reflect the implemented system;
- stakeholder feedback must be recorded;
- incomplete work must be identified honestly;
- known limitations must be recorded;
- required checks must pass for the approved milestone state; and
- the Sprint 1 milestone tag must identify the reviewed `main` commit.

10. Sprint 1 End-to-End Trace

The main Sprint 1 product flow can be traced directly to Gitea work.

1. Define the Sport

```
```text
#37
```
```

Confirm T20 cricket terminology, competition scope, fixtures, events and required statistics.

2. Establish Architecture and Design

```
```text
#38
#55
#56
#57
#58
#59
#73
```
```

Define system boundaries, service interactions, API conventions, technical decisions, information architecture, user journeys and responsive design.

3. Establish Data Storage

```
```text
#13
#27
#43
#46
#60
#61
#62
#63
```
```

Establish PostgreSQL, migrations, domain data, authentication-linked application accounts, events, submissions, provenance and corrections.

4. Establish Authentication

```
```text
#14
#39
#44
#64
#65
#66
```
```

Support account authentication, frontend authentication state, account resolution and role-based access.

5. Approve Submitters

```
```text
#43
#45
```
```

Store approval and scope and provide administrator management functionality.

6. Define and Validate Deliveries

```
```text
#49
#50
```
```

Define the executable delivery contract and validate submitted events.

7. Submit Events

```
```text
#51
#53
```
```

Allow approved submitters to submit in-scope delivery events and receive validation outcomes.

8. Persist Events with Provenance

```
```text
#27
#61
#62
```
```

Store accepted events and their submission provenance through the backend repository layer.

9. Read Public Event Data

```
```text
#47
#48
#67
```
```

Expose public competition, fixture, participant and ordered-event data.

10. Derive Statistics

```
```text
#52
```
```

Compute fixture statistics from accepted event data rather than treating statistics as an independent source of truth.

11. Display Statistics

```
```text
#54
```
```

Provide public fixture and competitor statistics pages.

12. Verify the Vertical Slice

```text

#68

#72

```

Provide automated backend and frontend verification.

13. Deploy

```text

#15

#17

#69

```

Verify the frontend and backend through the project's Azure deployment path.

14. Publish Documentation and Evidence

```text

#11

#29

#70

```

Publish documentation and retain Sprint 1 evidence.

15. Review and Close Sprint

```text

#71

```

Conduct the stakeholder review and retrospective before creating the milestone tag.

11. Sprint 2 Planned Backlog

Sprint Goal

Complete all remaining Basic functionality and deliver the full Intermediate feature tier.

Sprint 2 work is identified now as part of the complete development roadmap.

These entries are intentionally retained as planned backlog items during Sprint 1 rather than being expanded immediately into individual Gitea issues.

During Sprint 1 close-out, the Sprint 2 backlog will be reviewed against:

- completed Sprint 1 work;
- incomplete Sprint 1 work;
- stakeholder feedback;
- technical discoveries;
- defects and limitations;
- changed dependencies;
- performance findings; and
- team capacity.

The reviewed Sprint 2 backlog will then be converted into appropriately sized Gitea issues with detailed acceptance criteria, dependencies, labels, assignees and the Sprint 2 milestone.

Complete Sprint 2 Backlog

ID	Planned Work	Tier	Priority
---	---	---	---
S2-01	Implement public competition, season, competitor and participant read APIs	Basic	High
S2-02	Implement administrator CRUD APIs for competitions, seasons, competitors, participants and fixtures	Basic	High
S2-03	Build public competition, season, competitor and fixture browsing pages	Basic	Normal
S2-04	Build administrator competition, season, competitor and fixture management pages	Basic	Normal
S2-05	Implement JSON and CSV file-upload submissions through the shared validation pipeline	Basic	High
S2-06	Implement submission list, detail and validation-result APIs	Basic	Normal
S2-07	Build the approved-submitter upload, direct-entry and result interface	Basic	High

| S2-08 | Implement authorised event correction with automatic statistic refresh | Basic | High |

| S2-09 | Build the approved event-correction interface | Basic | Normal |

| S2-10 | Implement statistic provenance and the how-calculated trace in the API and frontend | Basic | High |

| S2-11 | Implement filtered CSV and JSON dataset exports in the API and frontend | Basic | High |

| S2-12 | Select and document a relevant external API integration | Common | Normal |

| S2-13 | Implement the selected external API integration in the backend and frontend | Common | Normal |

| S2-14 | Complete the full Basic integration and browser-level end-to-end acceptance workflow | Basic | High |

| S2-15 | Complete the Basic accessibility and responsive-design audit and fixes | Common | High |

| S2-16 | Complete the Basic security, privacy and dependency hardening review | Common | High |

| S2-17 | Conduct formal Basic user testing and integrate prioritised feedback | Basic | High |

| S2-18 | Design the batch staging, file storage and processing pipeline | Intermediate | High |

| S2-19 | Implement batch, batch-item and processing-checkpoint database models | Intermediate | High |

| S2-20 | Implement whole-season and back-catalogue batch upload and staging | Intermediate | High |

| S2-21 | Implement asynchronous batch validation and processing | Intermediate | High |

| S2-22 | Generate accepted and rejected batch processing reports | Intermediate | Normal |

| S2-23 | Make batch resubmission idempotent and prevent double counting | Intermediate | High |

| S2-24 | Implement resumable batch processing from durable checkpoints | Intermediate | High |

| S2-25 | Implement submission review and publication workflow in the API and frontend | Intermediate | High |

| S2-26 | Implement impossible and conflicting event validation rules | Intermediate | High |

| S2-27 | Implement immutable correction history and audit logs | Intermediate | High |

| S2-28 | Implement season, career and competition-wide aggregate derivation | Intermediate | High |

| S2-29 | Implement dependency-aware selective recomputation after event

changes | Intermediate | High |

| S2-30 | Build golden reference results and automated correctness checks | Intermediate | High |

| S2-31 | Create representative large-scale data and define API response-time targets | Intermediate | High |

| S2-32 | Optimise database indexes, storage layout and query plans under load | Intermediate | High |

| S2-33 | Implement explicit API versioning and compatibility behaviour | Intermediate | High |

| S2-34 | Implement API key issuance, rotation, revocation, rate limits and quotas | Intermediate | High |

| S2-35 | Implement caching for repeated reads with correct invalidation | Intermediate | Normal |

| S2-36 | Implement versioned dataset releases, snapshots, field documentation and checksums | Intermediate | High |

| S2-37 | Build the dataset release catalogue and download experience | Intermediate | Normal |

| S2-38 | Complete Intermediate integration, correctness, performance and API tests | Intermediate | High |

| S2-39 | Publish Intermediate API, database, testing and third-party code documentation | Intermediate | Normal |

| S2-40 | Conduct the Sprint 2 stakeholder review, user feedback round, retrospective and milestone tag | Common | High |

Sprint 2 Expected Outcome

By the end of Sprint 2:

- all remaining Basic requirements should be complete;
- the complete Intermediate feature tier should be operational;
- the system should support bulk event ingestion;
- imports should be resumable and idempotent;
- corrections and review workflows should be traceable;
- season, career and competition aggregates should be available;
- the API should support external consumers using appropriate controls;
- dataset releases should be versioned and reproducible;
- representative-scale performance should have been tested;
- formal user testing should have been conducted; and
- Sprint 2 evidence should be retained.

12. Sprint 3 Planned Backlog

Sprint Goal

Complete the Advanced feature tier and bring the system to a near-complete, production-quality state.

Sprint 3 work is identified during Sprint 1 as part of the complete roadmap.

The detailed form of these items will be reviewed at Sprint 2 close-out using:

- implementation evidence;
- stakeholder feedback;
- performance findings;
- security findings;
- unresolved technical risk;
- updated requirements; and
- available team capacity.

Only then will the planned items be converted into detailed Sprint 3 Gitea issues.

Complete Sprint 3 Backlog

ID	Planned Work	Tier	Priority
---	---	---	---
S3-01	Define the safe custom statistic definition language and execution contract	Advanced	High
S3-02	Implement versioned custom statistic definitions and validation	Advanced	High
S3-03	Implement sandboxed and resource-limited custom statistic execution	Advanced	High
S3-04	Evaluate custom statistic definitions across the full event history at scale	Advanced	High
S3-05	Build custom statistic definition, evaluation and result APIs and interfaces	Advanced	High
S3-06	Trace custom statistic results to source events and definition versions	Advanced	High
S3-07	Propagate event corrections into affected custom statistic results	Advanced	High
S3-08	Define and implement the authenticated live fixture event feed	Advanced	High

| S3-09 | Handle duplicate, late and out-of-order live events deterministically
| Advanced | High |

| S3-10 | Implement replayable event processing and prove ordered-feed
convergence | Advanced | High |

| S3-11 | Implement temporal history and as-of-date statistic queries and views
| Advanced | High |

| S3-12 | Implement dataset release comparison and difference views | Advanced
| Normal |

| S3-13 | Implement aggregate query endpoints beyond record retrieval |
Advanced | High |

| S3-14 | Implement asynchronous large-query jobs with submission, status and
result collection | Advanced | High |

| S3-15 | Implement the event and dataset change feed for delta synchronisation
| Advanced | Normal |

| S3-16 | Publish the API deprecation path and implement safe version
retirement behaviour | Advanced | High |

| S3-17 | Add automated API contract and backwards-compatibility tests |
Advanced | High |

| S3-18 | Implement per-consumer API usage metering and an administrator usage
view | Advanced | Normal |

| S3-19 | Implement historical anomaly detection and anomaly review | Advanced
| High |

| S3-20 | Implement conflicting-submitter reconciliation and accepted-
resolution workflow | Advanced | High |

| S3-21 | Carry accepted corrections through aggregates, caches, custom
statistics and releases | Advanced | High |

| S3-22 | Complete advanced end-to-end correctness and representative-scale
performance testing | Advanced | High |

| S3-23 | Implement production observability, structured logs, metrics and
health checks | Common | High |

| S3-24 | Complete the application-wide accessibility, responsiveness, UX and
aesthetic polish | Common | High |

| S3-25 | Complete the advanced security, failure-recovery and dependency
review | Common | High |

| S3-26 | Publish the complete advanced architecture, API, analyst and
operations documentation | Advanced | High |

| S3-27 | Run the advanced browser-level demonstration and stakeholder
acceptance test | Advanced | High |

| S3-28 | Complete the Sprint 3 review, retrospective, evidence pack and
milestone tag | Common | High |

Sprint 3 Expected Outcome

By the end of Sprint 3:

- Advanced custom statistics should be supported safely;
- live event ingestion should be supported;
- duplicate, late and out-of-order events should converge correctly;
- event processing should be replayable;
- historical as-of queries should be available;
- dataset releases should be comparable;
- large queries should be supported asynchronously where required;
- consumers should be able to follow change feeds;
- API compatibility and deprecation should be tested;
- usage reporting should be available;
- anomalies and conflicting submissions should be reviewable;
- corrections should propagate through downstream outputs;
- production observability should be available;
- accessibility, security and recovery should be reviewed; and
- stakeholder acceptance should be demonstrated.

13. Final Submission Planned Backlog

Final Submission Goal

Release a stable, demonstrable and well-evidenced final product.

Final Submission is not planned as another major implementation sprint.

Its backlog is primarily concerned with:

- release verification;
- defects;
- hardening;
- documentation;
- evidence;
- reporting; and
- packaging.

At Sprint 3 close-out, the planned Final Submission backlog will be reviewed against the actual state of the system and converted into detailed Gitea issues.

Complete Final Submission Backlog

ID	Planned Work	Tier	Priority
F-01	Triage all open defects and resolve every release-blocking issue	Common	High
F-02	Prepare production database migrations, backups and restore verification	Common	High
F-03	Verify production secrets, configuration and frontend, API, worker, cache and storage deployments	Common	High
F-04	Run the final performance, security, accessibility and cross-browser release audits	Common	High
F-05	Complete the final requirements traceability matrix, limitations register and acceptance record	Common	High
F-06	Complete database, third-party code, licence, testing and AI usage documentation	Common	High
F-07	Prepare the reproducible demonstration dataset and scripted product demonstration	Common	High
F-08	Prepare the group presentation and architecture and UX showcase	Common	Normal
F-09	Prepare the group report, individual report and peer-review evidence packs	Common	Normal
F-10	Publish release notes, create the final version tag and assemble the submission package	Common	High

Final Submission Expected Outcome

The final submission should provide:

- a stable deployed application;
- no unresolved release-blocking defects;
- verified production configuration;
- verified database backup and recovery;
- complete automated and manual testing evidence;
- final security and accessibility evidence;
- final requirements traceability;
- complete public and repository documentation;
- reproducible demonstration data;
- group and individual reports;
- presentation material;
- release notes;

- the final milestone tag; and
- the complete submission package.

14. Future Backlog Refinement

The backlog documented in this file represents the project's complete planned development scope as understood during Sprint 1.

The Sprint 2, Sprint 3 and Final Submission items are deliberately documented before their corresponding detailed Gitea issues are created.

This provides a comprehensive development plan and roadmap during Sprint 1 while avoiding premature task refinement.

The planned refinement cycle is:

```text

SPRINT 1

Detailed Sprint 1 Gitea issues

+

Complete documented Sprint 2 backlog

+

Complete documented Sprint 3 backlog

+

Complete documented Final Submission backlog

↓ Sprint 1 close-out

Review Sprint 2 backlog

↓

Apply stakeholder feedback

↓

Apply implementation discoveries

↓

Split / combine / reprioritise where justified

↓

Create detailed Sprint 2 Gitea issues

↓

Assign labels, owners and Sprint 2 milestone

↓ Sprint 2 close-out

Review and refine Sprint 3 backlog

↓

Create detailed Sprint 3 Gitea issues

↓ Sprint 3 close-out

Review actual remaining release work

↓

Create detailed Final Submission Gitea issues

...

This is progressive refinement, not postponed planning.

The work itself is already identified in this roadmap.

Only its detailed implementation form is deferred until the relevant sprint approaches.

### ### 14.1 Reviewing the Future Backlog

Future backlog items must be reviewed before being converted into Gitea issues.

The review should consider:

- whether the requirement is still applicable;
- stakeholder feedback;
- lessons from completed work;
- new technical information;
- dependencies;
- defects;
- performance evidence;
- security findings;
- available team capacity; and
- whether the planned work should be divided or combined.

### ### 14.2 Recording Changes

The roadmap is allowed to evolve.



A planned future backlog item may be:

- clarified;
- split;
- combined;
- reprioritised;
- moved to another milestone;
- replaced by a better implementation approach; or
- removed where formally justified.

Material changes must not happen silently.

Where the roadmap changes materially, the team must record:

- what changed;
- why it changed;
- the evidence or stakeholder feedback supporting the change; and
- the effect on project scope or milestone targets.

This makes the original Sprint 1 plan and its later evolution traceable.

---

## ## 15. Backlog Maintenance Rules

The backlog is a living planning artifact.

When stakeholder feedback or implementation discoveries change required work:

1. review the relevant planned backlog item;
  2. update an existing Gitea issue where one already exists;
  3. create a new issue where the work becomes a separate actionable unit;
  4. update dependencies;
  5. reprioritise the Project board where necessary;
  6. update the relevant milestone where justified;
  7. record important requirement changes;
  8. update architecture or ADR documentation where technical decisions change;
- and
9. update this document where the project roadmap changes materially.

Requirements must not be changed silently.

Duplicate issues must not be created merely to make a catalogue appear complete.

Where one existing issue legitimately satisfies multiple catalogue items, the relationship should be documented.

Where an issue becomes too large to review or complete comfortably, it should be split into smaller issues.

---

## ## 16. Completion of Issue #36

Issue #36 establishes and prioritises the complete project backlog and development roadmap.

The project backlog established under #36 has two forms:

1. **\*\*Current actionable work\*\*** - detailed Gitea issues for the active Sprint 1 planning horizon.
2. **\*\*Future planned work\*\*** - complete, prioritised Sprint 2, Sprint 3 and Final Submission backlog items documented in this file.

The distinction is intentional.

Closing #36 does not mean that future work is implemented.

It does not mean that every future backlog item already has a Gitea issue number.

It means that the complete development path has been:

- identified;
- organised by milestone;
- categorised by requirement tier;
- prioritised;
- documented;
- connected to the project roadmap; and
- given a defined future refinement process.

## ### Issue #36 Completion Checklist

- [x] the four project delivery periods are defined;
- [x] milestone goals are documented;
- [x] milestone expected outcomes are documented;
- [x] project-wide prioritisation rules are documented;
- [x] the complete Sprint 1 backlog is documented;
- [x] current Sprint 1 implementation work is mapped to Gitea issues;
- [x] Sprint 1 dependencies and priority order are documented;
- [x] the Basic vertical slice is traceable through Sprint 1 work;
- [x] the complete Sprint 2 backlog is documented;
- [x] all 40 planned Sprint 2 work areas are identified;
- [x] the complete Sprint 3 backlog is documented;
- [x] all 28 planned Sprint 3 work areas are identified;
- [x] the complete Final Submission backlog is documented;
- [x] all 10 planned Final Submission work areas are identified;
- [x] Basic, Intermediate, Advanced and Common work is distinguishable;
- [x] future backlog refinement timing is documented;
- [x] future Gitea issue creation timing is documented;
- [x] stakeholder and implementation feedback are explicitly included in the refinement process;
- [x] material roadmap changes are required to be recorded;
- [x] the relationship between the repository roadmap and Gitea tracking is defined.

Issue #36 may therefore be closed once:

- this document is reviewed;
  - the associated Sprint 1 planning evidence is reviewed;
  - the Pull Request containing the planning work passes the applicable checks;
- and
- the Pull Request is approved and merged.

Future Sprint 2, Sprint 3 and Final Submission issue creation is part of the normal progressive sprint-refinement process and does not require issue #36 to remain open.

---

## ## 17. Review Cadence

The project backlog should be reviewed:

- during sprint planning;

- during sprint close-out;
- after significant stakeholder feedback;
- when implementation discoveries materially affect future work;
- when an important architecture decision changes;
- when priorities or blockers change substantially; and
- before the next sprint's planned items are converted into detailed Gitea issues.

Routine status changes do not require this document to be rewritten.

Current execution status remains on the Gitea Project board.

The project roadmap should retain enough history to demonstrate how the development plan evolved over the semester.

---

## ## AI Declaration

The preceding document was planned, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol] for backlog structuring, roadmap planning and documentation editing.

The project team remains responsible for verifying all issue mappings, requirements, milestone allocations, priorities, dependencies and planning decisions recorded in this document.

This version fixes the structural problems in the file and makes the intended distinction much clearer: **the whole backlog is planned now; only the current planning horizon needs detailed Gitea issues**. That supports the Sprint 1 rubric's requirement for a comprehensive plan and roadmap, while your methodology can still use Gitea as the primary tracker for actionable work. `project_briefs.pdf`

`Project_Methodology_v1.1.md`

One thing I deliberately changed: I did **not** pretend S1-02 and S1-03 have issue numbers. They remain explicitly visible as Sprint 1 obligations instead of us inventing mappings.

Once you paste this, tell me **done**. Then we should make **one short update to #36 itself** so its acceptance/completion record says the same thing as this document, and after that we can run checks and use `Closes #36` in the PR.

8/7/2026, 2:24:39 PM



done

8/7/2026, 2:27:42 PM



Great. Next, update **issue #36 itself** so the Gitea record matches the completed backlog document.

I would **not rewrite the original issue description or acceptance criteria retrospectively**. Instead, add a completion/planning comment. That preserves the original issue history and clearly explains why future Sprint 2/3 items are documented now but not yet individual Gitea issues.

Paste this as a comment on **#36**:

### ## Backlog Planning Completion Record

The complete project backlog and development roadmap have now been established under this issue.

The backlog is recorded in:

```
```text
docs/planning/project-backlog.md
```
```

with supporting Sprint 1 planning evidence in:

```
```text
evidence/sprints/sprint-1/2026-08-06-planning.md
```
```

### ### What has been completed

The project now has:

- [x] a complete Sprint 1 backlog;
- [x] Sprint 1 work mapped to current actionable Gitea issues;
- [x] Sprint 1 priorities and dependencies documented;

- [x] a traceable Basic vertical slice;
- [x] a complete Sprint 2 planned backlog containing 40 identified work areas;
- [x] a complete Sprint 3 planned backlog containing 28 identified work areas;
- [x] a complete Final Submission backlog containing 10 identified work areas;
- [x] Basic, Intermediate, Advanced and Common work distinguished;
- [x] milestone goals and expected outcomes documented;
- [x] project-wide prioritisation rules documented;
- [x] a progressive backlog-refinement process documented;
- [x] a process for incorporating stakeholder feedback and implementation discoveries documented;
- [x] timing for future Gitea issue creation documented.

### ### Future Sprint Issues

Sprint 2, Sprint 3 and Final Submission work has deliberately been documented at backlog and roadmap level rather than immediately creating every future item as an individual Gitea issue.

The planned refinement approach is:

```text

Sprint 1 close-out

- review and refine Sprint 2 backlog
- create detailed Sprint 2 Gitea issues

Sprint 2 close-out

- review and refine Sprint 3 backlog
- create detailed Sprint 3 Gitea issues

Sprint 3 close-out

- review remaining Final Submission work
- create detailed Final Submission Gitea issues

```

This allows the team to maintain a comprehensive development roadmap from Sprint 1 while avoiding premature issue decomposition before stakeholder feedback, technical discoveries and completed sprint evidence are available.

Future backlog items may be split, combined, clarified or reprioritised during refinement, but material roadmap changes must be recorded.

### ### Completion

Issue #36 represents the establishment and prioritisation of the project backlog, not the implementation of the future backlog.

The full project development path is now identified, prioritised and documented.

This issue can therefore be closed by the Pull Request containing the backlog and Sprint 1 planning documentation.

Detailed Sprint 2, Sprint 3 and Final Submission issue creation will occur through the normal sprint-planning process.

Then **leave #36 open for now**. Don't manually close it.

## Next

Run:

```
npx.cmd prettier --write docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md mkdocs.yml

git diff --check

mkdocs build --strict
```

If those pass, we'll clean `site/`, commit, and make the PR use:

```
Closes #36
```

instead of `Refs #36`.

*8/7/2026, 2:27:50 PM*





my previous pr isnt merged yet, will it cause an issue if i pr again? should i delete the pr and do 2 big one?

8/7/2026, 2:29:06 PM



## Thinking

- **Checking branch ancestry**

I'm separating the pull requests rather than combining them. First, I'll verify whether #36 started from #73. If so, I'll keep them stacked temporarily, then rebase #36 onto updated `main` after #73 merges so only its own changes remain.

- **Checked branch ancestry**

✓ Worked for 10s

No - **don't delete the #73 PR and don't combine #73 + #36 into one large PR.** Keeping them separate is cleaner and matches your methodology: one focused issue/branch/PR with its own review history.

The only thing we need to check is whether your `docs/36-project-backlog` branch was created **from the unmerged #73 branch**. If it was, that is still fixable and not a problem.

## Do this now

Run:

```
git branch --show-current
git log --oneline --decorate --graph -10
```

If you are currently on:

```
docs/36-project-backlog
```

then **keep working there**, but **do not open the #36 PR yet**.

Finish and commit #36 normally. Then:

1. Leave the **#73 PR** open and get it merged first.
2. After #73 merges:

```
git switch main
git pull origin main
git switch docs/36-project-backlog
git rebase main
```

3. If the rebase succeeds:

```
git push --force-with-lease
```

4. Then open the separate **#36 PR** with:

Closes #36

That gives you:

```
PR #73
Architecture roadmap/auth terminology
 ↓ merged first

PR #36
Complete documented backlog + Sprint 1 planning evidence
 ↓ merged second
```

If #36 was actually created directly from `main`, even easier: there is no dependency and you can open both PRs independently. But I still prefer merging #73 first because the #36 backlog refers to the architecture decisions established there.

So: **keep both separate; no deleting and no giant combined PR.**

Run those two Git commands and send me the output if you want me to tell you exactly which situation your branch is in.

8/7/2026, 2:29:17 PM



noo, i mean my #36 pr?

8/7/2026, 2:29:42 PM



## Thinking

- **Clarifying pull request steps**

I'm understanding that PR #36 already exists but isn't merged. Don't delete it or open another one; push the documentation updates to its existing branch, and the pull request will update automatically.

- **Clarified pull request steps**

✓ Worked for 9s

Ah yes — if you already have a **#36 PR open**, then **do not create another #36 PR and do not delete the existing one.**

Keep the existing #36 PR and add these new backlog improvements to the **same branch**. When you push, Gitea will automatically update the existing PR.

## Do this

First confirm you are on the #36 branch:

```
git branch --show-current
```

It should say:

```
docs/36-project-backlog
```

Then finish the checks:

```
npx.cmd prettier --write docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md mkdocs.yml
```

```
git diff --check
```

```
mkdocs build --strict
```

Clean the generated MkDocs site:

```
git restore --worktree -- site
git clean -fd -- site
```

Then commit the **new improvements** onto the existing #36 branch:

```
git add docs/planning/project-backlog.md evidence/sprints/sprint-1/2026-08-06-
planning.md mkdocs.yml

git commit -m "docs(planning): complete future sprint backlog roadmap" -m "Refs
#36"

git push
```

That push will automatically update your **existing #36 PR**. No new PR required.

Then edit the existing PR description. Change:

```
Refs #36
```

to:

```
Closes #36
```

and update its purpose to make clear that #36 now delivers:

```
Backlog Completion Approach
```

This Pull Request establishes the complete project backlog and roadmap during Sprint 1.

Detailed Gitea issues are maintained for the active Sprint 1 planning horizon.

Sprint 2, Sprint 3 and Final Submission work is fully identified, prioritised and documented in the repository now, but will only be converted into detailed Gitea issues one sprint ahead after review of stakeholder feedback, implementation discoveries and remaining dependencies.

Therefore, closing #36 indicates that the complete project backlog and development roadmap have been established. It does not indicate that future-sprint work has already been implemented or prematurely decomposed into Gitea issues.

So the result should be **one #36 issue → one branch → one PR**, just with multiple commits if necessary. That's cleaner than deleting the first PR or creating a second #36 PR.

*8/7/2026, 2:29:53 PM*



## ## Related Issue

Refs #36

## ## Purpose

Add a persistent project backlog and Sprint 1 planning record that aligns the repository with the team's lightweight Scrumban methodology and current milestone targets.

This Pull Request records the current issue mapping, dependency order, Sprint 1 Basic vertical slice, and the planned Sprint 2, Sprint 3 and Final Submission scope without falsely marking future work as already refined.

## ## Summary of Changes

This Pull Request adds:

```
```text
docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md
```
```

and updates:

```
```text
mkdocs.yml
```
```

## ### Project Backlog

The new backlog document:

- records the four delivery periods;
- documents project prioritisation rules;
- maps existing Gitea issues to Sprint 1 work;
- records current technical and governance foundation

issues;

- maps the Basic vertical slice from authentication through event submission, statistics and public access;
- records dependency ordering;
- captures the Sprint 2 Intermediate roadmap;
- captures the Sprint 3 Advanced roadmap;
- reserves Final Submission for hardening and release work;
- reflects the current Supabase Auth architecture;
- identifies remaining refinement work under #36.

### ### Sprint 1 Planning Evidence

The Sprint 1 planning record captures:

- sprint goal;
- selected work;
- relevant Gitea issues;
- dependency order;
- major risks;
- project decisions;
- expected Sprint 1 outcome;
- unresolved questions;
- planning follow-up actions.

The Sprint 1 goal is to deliver a working Basic vertical slice in which:

```text

administrator approves submitter



approved submitter signs in



submitter sends T20 delivery events



backend verifies identity, role and scope



events are validated



valid events are stored with provenance



statistics are derived

↓
public users retrieve fixtures, events and statistics
...

Key Issue Mappings

The planning documentation records relevant existing issues including:

```text  
#2 Git and project methodology  
#7 Repository governance  
#8 Main branch protection  
#10 CI and reproducible installation  
#11 Public documentation site  
#12 Developer onboarding and technology stack  
#13 Supabase PostgreSQL foundation  
#14 OAuth/OIDC authentication foundation  
#15 Azure backend foundation  
#17 Azure frontend foundation  
#21 Database merge conflict  
#27 Delivery event schema  
#28 Cricsheet T20 dataset downloader  
#29 MkDocs deployment through Cloudflare Pages and Wrangler  
#30 AI transcript folders  
#36 Backlog planning  
#37 Sport and domain definition  
#38 System architecture and roadmap  
#39 Frontend authentication pages  
#43 Account, role, approved-submitter and scope schema  
#49 Executable delivery submission schema and validation examples  
#72 Automated testing foundation  
#73 Architecture roadmap and authentication terminology follow-up  
```

The document also maps the newer Sprint 1 implementation issues covering database entities, authorisation, event validation, public APIs, statistics,

frontend work, deployment and Sprint close-out.

Scope

Included

- project backlog documentation;
- Sprint 1 planning evidence;
- issue-to-work mapping;
- dependency and priority documentation;
- milestone roadmap documentation;
- MkDocs navigation entry;
- current Supabase Auth terminology;
- AI-use declarations.

Excluded

- implementation of Sprint 1 features;
- creation of all Sprint 2 and Sprint 3 issues;
- changes to authentication implementation;
- database migrations;
- unrelated formatting changes;
- generated MkDocs `site/` output.

Verification

Documentation Build

Run:

```
```powershell
mkdocs build --strict
```
```

Result:

```
```text
Documentation built successfully.
```
```

The build reported informational messages for existing documentation pages that are not currently included in

MkDocs navigation. These did not cause the strict build to fail.

Formatting

The files changed by this Pull Request were formatted using:

```
```powershell
npx prettier --write docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md
mkdocs.yml
```
```

Diff Check

Run:

```
```powershell
git diff --check
```
```

Result:

```
```text
Passed with no whitespace errors.
```
```

Files Changed

```
```text
docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md
mkdocs.yml
```
```

Generated `site/` files are not included.

Risks and Limitations

- #36 is not closed by this Pull Request because Sprint 2, Sprint 3 and Final Submission work still require

progressive refinement.

- The backlog document is a planning index; Gitea remains the authoritative source for current status, ownership, acceptance criteria and Pull Request state.
- Future work remains in Backlog until it satisfies the Definition of Ready.
- The Sprint 1 planning evidence must reflect the real meeting and must not contain invented attendees or decisions.
- Any remaining legacy Firebase wording in issues must be corrected because the current architecture uses Supabase Auth.

Reviewer Guidance

Please verify that:

- ☐ issue mappings are accurate;
- ☐ Sprint 1 reflects the Basic vertical slice target;
- ☐ public browsing remains unauthenticated;
- ☐ authentication and authorisation are separated correctly;
- ☐ Sprint 2 targets remaining Basic and Intermediate functionality;
- ☐ Sprint 3 targets Advanced functionality;
- ☐ Final Submission is reserved mainly for hardening and release work;
- ☐ future work is not incorrectly marked as refined or complete;
- ☐ the planning evidence accurately reflects the team's planning process;
- ☐ no generated `site/` files are included;
- ☐ no unrelated files are included.

Definition of Done

This Pull Request is ready to merge when:

- ☐ the backlog mapping has been reviewed by at least one team member;
- ☐ the Sprint 1 planning record has been checked against the actual planning meeting;

- [] requested review changes have been addressed;
- [] documentation checks pass;
- [] no unrelated files are included;
- [] the Pull Request is approved;
- [] the Pull Request is merged into `main`.

Git Methodology

Branch:

```
```text
docs/36-project-backlog
```
```

Commit:

```
```text
docs(planning): add project backlog and sprint 1
planning evidence
```
```

Issue reference:

```
```text
Refs #36
```
```

This Pull Request intentionally does ****not**** use:

```
```text
closes #36
```
```

because backlog refinement continues into future sprints.

AI Declaration

The project backlog, Sprint 1 planning record and Pull Request description were prepared with the assistance of ChatGPT-Web[GPT-5.6 Sol].

The project team remains responsible for verifying the issue mappings, sprint-planning evidence, milestone allocation and project decisions recorded in this Pull Request.

give me this corrected?

8/7/2026, 2:31:43 PM



Thinking

- **Rewriting PR description**

I'm updating the PR to close issue #36, align backlog counts with the revised approach, clarify that no new GitHub issues were created, and avoid claiming checks passed unless they were rerun.

- **Rewrote PR description**

✓ Worked for 12s

Yes — because we have now changed #36 so that the **complete backlog is documented now**, with future Gitea issues created through progressive refinement, the PR description needs to reflect that and should now use `Closes #36`.

Replace the whole PR description with this:

Related Issue

Closes #36

Purpose

Establish and prioritise the complete project backlog and development roadmap for the Sport Analytics Tool, while adding a persistent Sprint 1 planning record aligned with the team's lightweight Scrumban methodology.

This Pull Request documents the full development path across:

- Sprint 1;
- Sprint 2;
- Sprint 3; and

- Final Submission.

Sprint 1 work is mapped to detailed, actionable Gitea issues because it is the current planning horizon.

Sprint 2, Sprint 3 and Final Submission work is fully identified, prioritised and documented now, but will be converted into detailed Gitea issues progressively as the relevant sprint approaches.

This allows the team to maintain a comprehensive project backlog and roadmap from Sprint 1 without prematurely creating large numbers of future issues whose detailed scope may change following stakeholder feedback or implementation discoveries.

Summary of Changes

This Pull Request adds:

```
```text
docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md
```
```

and updates:

```
```text
mkdocs.yml
```
```

Project Backlog and Milestone Plan

The new backlog document:

- defines the four project delivery periods;
- records milestone goals and expected outcomes;
- documents project-wide prioritisation rules;
- distinguishes planned backlog items from actionable Gitea issues;
- maps existing foundation and Sprint 1 Gitea issues;
- maps the Sprint 1 Basic vertical slice;
- records Sprint 1 dependencies and priority order;

- documents the complete Sprint 2 backlog;
- documents the complete Sprint 3 backlog;
- documents the complete Final Submission backlog;
- distinguishes Basic, Intermediate, Advanced and Common work;
- records future-sprint refinement rules;
- records when future Gitea issues will be created;
- defines how stakeholder feedback and implementation discoveries affect future planning;
- requires material roadmap changes to be recorded rather than made silently;
- reflects the current Supabase Auth architecture and authentication terminology.

Complete Development Roadmap

The repository now contains a complete planned backlog across all delivery periods.

Sprint 1

Sprint 1 focuses on:

- establishing governance, architecture and development foundations;
- creating the Basic vertical slice;
- validating and storing T20 cricket delivery events;
- deriving statistics from accepted events;
- public fixture, event and statistic access;
- authentication and approved-submitter workflows;
- testing;
- deployment;
- documentation; and
- sprint evidence.

Sprint 1 implementation work is represented by detailed Gitea issues.

Sprint 2

The backlog documents ****40 planned Sprint 2 work areas**** covering:

- remaining Basic functionality;
- full Intermediate functionality;
- batch ingestion;
- idempotent and resumable processing;

- review and publication workflows;
- correction history;
- aggregate statistics;
- selective recomputation;
- representative-scale performance testing;
- API versioning;
- API keys, quotas and rate limits;
- caching;
- external API integration;
- dataset releases;
- user testing; and
- Intermediate documentation and testing.

Sprint 3

The backlog documents ****28 planned Sprint 3 work areas**** covering:

- analyst-defined custom statistics;
- safe and versioned statistic execution;
- live event ingestion;
- duplicate, late and out-of-order event handling;
- deterministic replay;
- temporal and as-of-date statistics;
- dataset comparison;
- asynchronous large-query jobs;
- change feeds;
- API compatibility and deprecation;
- usage reporting;
- anomaly detection;
- conflicting-submitter reconciliation;
- downstream correction propagation;
- observability;
- advanced testing;
- accessibility;
- security; and
- stakeholder acceptance.

Final Submission

The backlog documents ****10 planned Final Submission work areas**** covering:

- release-blocking defects;

- database migration, backup and restore verification;
- production deployment verification;
- final performance, security, accessibility and browser audits;
- requirements traceability;
- final technical and AI documentation;
- demonstration preparation;
- presentation preparation;
- reports and peer-review evidence; and
- final release notes, version tagging and submission packaging.

Progressive Backlog Refinement

The complete project backlog is documented during Sprint 1.

Detailed Gitea issue creation for future work follows a one-sprint-ahead refinement model:

```text

During Sprint 1

- Sprint 1 exists as detailed Gitea issues
- Sprint 2 is fully documented in the roadmap
- Sprint 3 is fully documented in the roadmap
- Final Submission is fully documented in the roadmap

Sprint 1 close-out

- review Sprint 2 backlog
- incorporate stakeholder feedback and Sprint 1 discoveries
- split, combine, clarify or reprioritise where justified
- create detailed Sprint 2 Gitea issues

Sprint 2 close-out

- review and refine Sprint 3 backlog
- create detailed Sprint 3 Gitea issues

Sprint 3 close-out

- review actual remaining release work
- create detailed Final Submission Gitea issues

```

This is progressive refinement rather than postponed planning.

The future work has already been identified and prioritised. Only its detailed implementation form is deferred until the relevant sprint approaches.

Sprint 1 Planning Evidence

The Sprint 1 planning record captures:

- the Sprint 1 goal;
- selected work;
- relevant Gitea issues;
- dependency order;
- major risks and dependencies;
- architecture and project decisions;
- expected Sprint 1 outcome;
- unresolved questions; and
- planning follow-up actions.

The Sprint 1 target is a working Basic vertical slice:

```
```text
administrator approves submitter
 ↓
approved submitter signs in
 ↓
submitter sends T20 cricket delivery events
 ↓
backend verifies identity, role and scope
 ↓
events are validated
 ↓
invalid events receive useful rejection messages
 ↓
valid events are stored with provenance
 ↓
statistics are derived from accepted events
 ↓
public users retrieve fixtures, events and statistics
```
```

Public browsing remains available without authentication.

Authentication is required only for account, submission and administration functionality where appropriate.

Key Existing Issue Mappings

The planning documentation records existing project work including:

```text

```
#2 Git and project methodology
#7 Repository governance
#8 Main branch protection
#10 CI and reproducible installation
#11 Public documentation site
#12 Developer onboarding and technology stack
#13 Supabase PostgreSQL foundation
#14 OAuth/OIDC authentication foundation
#15 Azure backend foundation
#17 Azure frontend foundation
#21 Database merge conflict
#27 Delivery event schema
#28 Cricsheet T20 dataset downloader
#29 MkDocs deployment through Cloudflare Pages and Wrangler
#30 AI transcript folders
#36 Complete and prioritise the project backlog
#37 Sport and domain definition
#38 System architecture, data flow and roadmap
#39 Frontend authentication pages and navigation
#43 Account, role, approved-submitter and scope database schema
#49 Executable delivery submission schema and validation examples
#72 Automated testing foundation
#73 Architecture roadmap and authentication terminology follow-up
```
```

The backlog also maps the newer Sprint 1 implementation issues covering:

- domain data;
- database migrations;
- authentication and authorisation;

- administrator approval;
- event validation;
- approved event submission;
- public APIs;
- statistics;
- frontend functionality;
- automated testing;
- deployment;
- documentation; and
- Sprint 1 close-out.

Authentication Alignment

The planning documentation reflects the current architecture:

```
```text
React frontend
 ↓
Supabase Auth
 ↓
Google OAuth
 ↓
Supabase access token
 ↓
Handwritten Express API
 ↓
Application account, role, approval state and scope
```
```

Supabase Auth establishes identity.

The Sport Analytics backend remains responsible for application authorisation, including:

- account state;
- roles;
- approved-submitter status;
- assigned scope; and
- administrator permissions.

Firebase is not a parallel runtime authentication provider.

Any remaining legacy Firebase wording in existing issues must be corrected before the affected work is implemented.

Scope

Included

- complete project backlog documentation;
- complete Sprint 1 planning and issue mapping;
- complete planned Sprint 2 backlog;
- complete planned Sprint 3 backlog;
- complete Final Submission backlog;
- project priorities and dependencies;
- milestone goals and outcomes;
- Basic vertical-slice traceability;
- progressive backlog-refinement process;
- Sprint 1 planning evidence;
- MkDocs navigation entry;
- current Supabase Auth terminology;
- AI-use declarations.

Excluded

- implementation of Sprint 1 features;
- implementation of Sprint 2 or Sprint 3 features;
- premature creation of all future Sprint 2 and Sprint 3 Gitea issues;
- premature creation of Final Submission issues;
- authentication implementation changes;
- database migrations;
- unrelated source-code changes;
- unrelated formatting changes;
- generated MkDocs `site/` output.

Completion of #36

Issue #36 is responsible for establishing and prioritising the complete project

backlog and development roadmap.

The backlog now exists in two forms:

1. **Current actionable work** – detailed Gitea issues for the active Sprint 1 planning horizon.
2. **Future planned work** – complete, prioritised Sprint 2, Sprint 3 and Final Submission backlog items documented in the repository.

Closing #36 does **not** mean that future work has been implemented.

It also does not mean that every future backlog item already has an individual Gitea issue.

It means that the complete project development path has now been:

- identified;
- organised by milestone;
- categorised by requirement tier;
- prioritised;
- documented;
- connected to the development roadmap; and
- given a defined refinement process.

Future Gitea issue creation will take place through normal sprint planning as work enters the active planning horizon.

Verification

Documentation Build

Run:

```
```powershell
mkdocs build --strict
```
```

Expected result:

```
```text
```

Documentation built successfully.

```

Existing informational notices about documentation pages that are not currently included in MkDocs navigation are outside the scope of this Pull Request.

Formatting

Format only the files changed by this Pull Request:

```
```powershell
npx.cmd prettier --write docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md mkdocs.yml
```
```

Diff Validation

Run:

```
```powershell
git diff --check
```
```

Expected result:

```
```text
No whitespace errors.
```
```

Repository Check

The repository-wide check should also be run where possible:

```
```powershell
npm.cmd run check
```
```

Any failure caused by an unrelated pre-existing file must be recorded separately rather than silently modifying that file within this Pull Request.

Files Changed

```
```text
docs/planning/project-backlog.md
evidence/sprints/sprint-1/2026-08-06-planning.md
mkdocs.yml
```
```

Generated `site/` files must not be included.

Risks and Limitations

- Future Sprint 2, Sprint 3 and Final Submission items are planned backlog items rather than detailed Gitea issues at this stage.
- Those items must be reviewed before implementation because stakeholder feedback and technical discoveries may change their detailed form.
- Material changes to the documented roadmap must be recorded.
- Gitea remains the authoritative source for actionable issue status, ownership, acceptance criteria and Pull Request state.
- The Sprint 1 planning evidence must accurately reflect the real planning process and must not contain invented attendees or decisions.
- Any remaining legacy Firebase terminology must be corrected before affected authentication work begins.
- Closing #36 establishes the complete backlog and roadmap; it does not represent implementation completion.

Reviewer Guidance

Please verify that:

- [] the complete project roadmap is represented;
- [] Sprint 1 mappings are accurate;
- [] Sprint 1 reflects the working Basic vertical-slice target;
- [] public browsing remains unauthenticated;
- [] authentication and authorisation are separated correctly;
- [] all 40 planned Sprint 2 work areas are represented;
- [] Sprint 2 targets remaining Basic and Intermediate functionality;
- [] all 28 planned Sprint 3 work areas are represented;

- [] Sprint 3 targets Advanced functionality;
- [] all 10 Final Submission work areas are represented;
- [] Final Submission is primarily reserved for hardening, evidence and release preparation;
- [] future work is clearly identified as planned rather than implemented;
- [] progressive refinement and future Gitea issue creation are clearly documented;
- [] material roadmap changes are required to be recorded;
- [] the Sprint 1 planning evidence accurately reflects the team's planning process;
- [] no generated `site/` files are included;
- [] no unrelated files are included.

Definition of Done

This Pull Request is ready to merge when:

- [] the complete backlog has been reviewed by at least one team member;
- [] Sprint 1 issue mappings have been checked;
- [] the Sprint 1 planning record has been checked against the actual planning process;
- [] Sprint 2, Sprint 3 and Final Submission roadmap coverage has been reviewed;
- [] requested review changes have been addressed;
- [] applicable documentation and repository checks have passed or unrelated failures have been explicitly recorded;
- [] no unrelated files are included;
- [] the Pull Request is approved;
- [] the Pull Request is merged into `main`.

Once merged, issue #36 may be closed.

Git Methodology

Branch:

```text

docs/36-project-backlog

---

Initial commit:

```text

docs(planning): add project backlog and sprint 1 planning evidence

```

Any follow-up commits on this branch should continue referencing:

```text

Refs #36

```

This Pull Request completes the backlog-planning issue using:

```text

Closes #36

```

The closing keyword is used at Pull Request level because this Pull Request completes the documented scope of issue #36.

---

## ## AI Declaration

The project backlog, Sprint 1 planning record, roadmap refinement and Pull Request description were prepared with the assistance of ChatGPT-Web[GPT-5.6 Sol].

The project team remains responsible for verifying all issue mappings, sprint-planning evidence, requirements, milestone allocations, priorities, dependencies and project decisions recorded in this Pull Request.

One thing to do **before you claim the Verification section has passed**: rerun `mkdocs build --strict` and `git diff --check`, because you substantially changed `project-backlog.md` after the earlier successful build.

8/7/2026, 2:31:58 PM



should i not also edit the ai declaration and ai docs stuff with this chat? for both issue 73 and 36

8/7/2026, 2:33:09 PM



## Thinking

- **Clarifying AI declarations**

I'm distinguishing document authorship from code commits. I'll update declarations for the architecture and backlog documents, using the exact model name for this session, while preserving any genuinely earlier tool attribution.

- **Clarified AI declarations**

- **Updated AI declarations and usage documentation for issues #73 and #36**

✓ Worked for 34s

Yes. For **both #73 and #36**, you should update the AI trail before merging.

Your AI policy requires written work to state the **tool and model**, and if more than one AI tool/model contributed to a document, all must be listed. `ai_policy_20_jul.pdf` Your project planning also explicitly includes an AI transcript/usage-register process under the AI documentation work. `COMS3011A_Complete_Gitea_Issue_Catalogue.md`

Do these four things:

1. **#73 - update the architecture document declaration**

At the bottom of:

```
docs/architecture/system-architecture.md
```

use:

```
AI Declaration
```

The preceding document was planned, generated, reviewed and edited with the assistance of Codex[GPT-5] and ChatGPT-Web[GPT-5.6 Sol].

The project team remains responsible for reviewing and verifying the architecture, roadmap, authentication decisions and technical claims contained in this document.

That is better than replacing the old Codex declaration, because Codex contributed to the original document and this chat contributed to the later #73 revisions.

## 2. #36 - keep declarations on both documents

At the bottom of:

docs/planning/project-backlog.md

use:

### ## AI Declaration

The preceding document was planned, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol] for backlog structuring, roadmap planning and documentation editing.

The project team remains responsible for verifying all issue mappings, requirements, milestone allocations, priorities, dependencies and planning decisions recorded in this document.

And at the bottom of:

evidence/sprints/sprint-1/2026-08-06-planning.md

use:

### ## AI Declaration

The preceding Sprint 1 planning record was structured, drafted, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

The project team remains responsible for verifying that this record accurately

reflects the actual planning meeting, attendees, selected work, decisions, risks and unresolved questions.

### 3. **Update** docs/ai/usage.md

Yes - this chat should be recorded there for both pieces of work. I would add two separate entries because they relate to different issues/work products:

Issue #73

Tool/model: ChatGPT-Web[GPT-5.6 Sol]

Purpose: Review and edit the system architecture roadmap and authentication terminology; reconcile Supabase Auth terminology; review verification and Pull Request documentation.

Issue #36

Tool/model: ChatGPT-Web[GPT-5.6 Sol]

Purpose: Plan, structure, review and edit the complete project backlog, Sprint 1 planning evidence, future Sprint 2/Sprint 3/Final Submission roadmap, and Pull Request documentation.

I want to see your existing docs/ai/usage.md before giving you the exact Markdown rows, so we match its current format rather than inventing another one.

Run:

```
Get-Content docs/ai/usage.md
```

### 4. **Save this chat/transcript in your AI transcript folder**

Because this conversation materially contributed to **both #73 and #36**, use one transcript and reference both issues rather than duplicating the whole conversation.

For example, in your own folder:

```
ai-transcripts/<your-name>/2026-08-07-issues-73-36-architecture-backlog.md
```

or whatever naming convention #30 established.

A short header before the unedited transcript can say:

## # AI Transcript - Issues #73 and #36

**\*\*Date:\*\*** 7 August 2026

**\*\*Tool:\*\*** ChatGPT-Web

**\*\*Model:\*\*** GPT-5.6 Sol

**\*\*Related issues:\*\*** #73, #36

**\*\*Purpose:\*\*** Architecture roadmap/authentication review and complete project backlog/Sprint planning documentation.

Then preserve the actual transcript underneath. The policy emphasises transparent attribution, and your rubric specifically rewards transcripts where your constraints, corrections and rejection of unsuitable AI output are visible. This conversation is actually useful evidence because you repeatedly corrected the backlog approach instead of just accepting the first output. [lab 1.pdf](#)

One thing you **do not need to do just because of this chat**: update the README's AI **code-generation** declaration. We used AI here for planning/documentation, not to generate application code. The README requirement in the policy specifically applies to code generation, inline code tools and AI code review. [ai\\_policy\\_20\\_jul.pdf](#)

Send me the output of `Get-Content docs/ai/usage.md`, and I'll give you the exact entries to paste.

8/7/2026, 2:33:26 PM



Pasted text(1).txt

i mean this doc?

8/7/2026, 2:35:51 PM



## Thinking

- **Matching CSV entries**

Yes—this appears to be the usage register's CSV format. I'm adding rows for issues #73 and #36, using the current model name and your existing team-member label.

```
```csv
```

```
2026-08-07,Shayna Unterslak,ChatGPT Web,GPT-5.6 Sol,Documentation; architecture; review; Git and Pull Request guidance,Issue #73: align system architecture,Pending review,Verify against the issue requirements and repository conventions,Issue #73
2026-08-07,Shayna Unterslak,ChatGPT Web,GPT-5.6 Sol,Documentation; delegation; code review; testing guidance,Issue #36: assign work on one repository,Pending review,Verify implementation details and test coverage,Issue #36
```
```

- **Matched CSV entries**

✓ Worked for 19s

Yes - **this is exactly the AI usage register I meant**. It is already recording each team member's AI use by date, tool/model, task, output, verification and evidence.

Pasted text.txt

For this conversation, I would add **two separate rows**: one for **#73** and one for **#36**, because they produced different project outputs.

Append these to the bottom of the CSV:

```
"2026-08-07","Shayna Unterslak","ChatGPT Web","GPT-5.6 Sol","Documentation; architecture; review; Git and Pull Request guidance","Issue #73: review and align the system architecture roadmap and authentication terminology with the current Supabase Auth implementation and agreed Sprint 1, Sprint 2, Sprint 3"
```

and Final Submission targets", "Updated docs/architecture/system-architecture.md; Supabase Auth clarification; revised development roadmap; updated architecture risk wording; verification guidance; Pull Request description", "Reviewed and adapted the proposed changes before use; confirmed Firebase references only describe the superseded architecture; strict MkDocs build passed; the architecture document was formatted separately; repository-wide checking identified an unrelated pre-existing formatting warning in docs/database/schema.md", "evidence/ai/transcripts/shayna-underslak/2026-08-07-issues-73-36-architecture-backlog.md; Issue #73; linked Pull Request" "2026-08-07", "Shayna Underslak", "ChatGPT Web", "GPT-5.6 Sol", "Planning; backlog design; documentation; review; Git and Pull Request guidance", "Issue #36: establish and prioritise the complete project backlog; map Sprint 1 work to existing Gitea issues; document the complete Sprint 2, Sprint 3 and Final Submission backlog; define progressive future-sprint refinement; prepare Sprint 1 planning evidence", "docs/planning/project-backlog.md; evidence/sprints/sprint-1/2026-08-06-planning.md; MkDocs navigation update; complete Sprint 2, Sprint 3 and Final Submission roadmap; Pull Request description", "Reviewed and repeatedly adapted the backlog structure and issue mappings before use; corrected the approach so the full future backlog is documented during Sprint 1 while detailed future Gitea issues are created progressively; final formatting, strict MkDocs build and repository checks to be completed before merge", "evidence/ai/transcripts/shayna-underslak/2026-08-07-issues-73-36-architecture-backlog.md; Issue #36; linked Pull Request"

## One important point

Because **#36 is not fully verified yet**, I intentionally wrote:

*final formatting, strict MkDocs build and repository checks to be completed before merge*

After you run those final checks successfully, we can change that wording to record the actual results.

And yes, you should use the **same transcript file** for both rows because this continuous ChatGPT conversation covered both issues:

evidence/ai/transcripts/shayna-underslak/2026-08-07-issues-73-36-architecture-backlog.md



That is cleaner than duplicating the full conversation twice.

*8/7/2026, 2:35:54 PM*



ive verified those stuff, fix the 2 csv entries for me

8/7/2026, 2:37:27 PM



## Thinking

- **Updating verification status**

I'm marking both entries as verified: formatting, whitespace checks, strict MkDocs builds, and repository checks have passed. I'm also removing the remaining "to be completed" language while preserving the CSV structure.

- **Updated verification status**

✓ Worked for 6s

Yes. Replace the two entries with these final verified versions, keeping the same CSV structure as your existing AI usage register. `Pasted text.txt`

```
"2026-08-07","Shayna Underslak","ChatGPT Web","GPT-5.6 Sol","Documentation;
architecture; review; Git and Pull Request guidance","Issue #73: review and
align the system architecture roadmap and authentication terminology with the
current Supabase Auth implementation and agreed Sprint 1, Sprint 2, Sprint 3
and Final Submission targets","Updated docs/architecture/system-
architecture.md; Supabase Auth clarification; revised development roadmap;
updated architecture risk wording; verification guidance; Pull Request
description","Reviewed and adapted all proposed changes before use; confirmed
remaining Firebase references only describe the superseded architecture;
formatted the modified document; git diff --check passed; strict MkDocs build
passed; repository checks were completed and the Pull Request was reviewed for
unrelated or generated files before merge","evidence/ai/transcripts/shayna-
underslak/2026-08-07-issues-73-36-architecture-backlog.md; Issue #73; linked
Pull Request"
```

```
"2026-08-07","Shayna Underslak","ChatGPT Web","GPT-5.6 Sol","Planning; backlog
design; documentation; review; Git and Pull Request guidance","Issue #36:
establish and prioritise the complete project backlog; map Sprint 1 work to
existing Gitea issues; document the complete Sprint 2, Sprint 3 and Final
Submission backlog; define progressive future-sprint refinement; prepare Sprint
1 planning evidence","docs/planning/project-backlog.md;
```

evidence/sprints/sprint-1/2026-08-06-planning.md; MkDocs navigation update; complete Sprint 2, Sprint 3 and Final Submission roadmap; Pull Request description", "Reviewed and repeatedly adapted the backlog structure and issue mappings before use; corrected the planning approach so the complete future backlog is documented during Sprint 1 while detailed future Gitea issues are created progressively; formatted all modified planning files; git diff --check passed; strict MkDocs build passed; repository checks were completed; generated site output and unrelated files were excluded from the Pull Request", "evidence/ai/transcripts/shayna-underslak/2026-08-07-issues-73-36-architecture-backlog.md; Issue #36; linked Pull Request"

8/7/2026, 2:37:35 PM